



JAVA PROGRAMMING





Centre for Distance and Online Education

Online BCA Program

Java Programming

Semester: 4

Author

Dr. Hetal Bhaidasna,
Assistant Professor,
Computer Engineering,
PIET-DS,
Parul University.

Credits

Centre for Distance and Online Education, Parul University,
Post Limda, Waghodia,
Vadodara, Gujarat, India
391760.
Website: <https://paruluniversity.ac.in/>

Disclaimer

This content is protected by CDOE, Parul University. It is sold under the stipulation that it cannot be lent, resold, hired out, or otherwise circulated without obtaining prior written consent from the publisher. The content should remain in the same binding or cover as it was initially published, and this requirement should also extend to any subsequent purchaser. Furthermore, it is important to note that, in compliance with the copyright protections outlined above, no part of this publication may be reproduced, stored in a retrieval system, or transmitted through any means (including electronic, Mechanical, photocopying, recording, or otherwise) without obtaining the prior written permission from both the copyright owner and the publisher of this content.

Note to Students



These course notes are intended for the exclusive use of students enrolled in Online BCA Program. They are not to be shared or distributed without explicit permission from the University. Any unauthorized sharing or distribution of these materials may result in academic and legal consequences.



TABLE OF CONTENT

SUB LESSON 1.1.....	9
BASICS OF JAVA, PROCEDURE-ORIENTED VS. OBJECT-ORIENTED PROGRAMMING, DIFFERENCE BETWEEN C, C++ & JAVA.....	9
SUB LESSON 1.2.....	17
BASICS OF OOPS.....	17
SUB LESSON 1.3.....	22
FEATURES AND ADVANTAGES OF JAVA.....	22
SUB LESSON 1.4.....	28
JAVA VIRTUAL MACHINE & BYTE CODE, JAVA ENVIRONMENT SETUP, JAVA PROGRAM STRUCTURE, COMPILE & RUNNING JAVA PROGRAM.....	28
SUB LESSON 1.5.....	37
DATA TYPES OF JAVA, TYPE CONVERSION & CASTING	37
SUB LESSON 1.6.....	43
VARIABLES, SCOPE OF VARIABLE, IDENTIFIERS, CONSTANT, COMMENTS IN JAVA	43
SUB LESSON 1.7.....	55
WRAPPER CLASS.....	55
SUB LESSON 1.8.....	59
STRING IN JAVA	59
SUB LESSON 1.9.....	64
GARBAGE COLLECTION	64
SUB LESSON 1.10	68
COMMAND LINE ARGUMENTS	68
SUB LESSON 2.1.....	73



DIFFERENT OPERATORS: ARITHMETIC, BITWISE, RATIONAL, LOGICAL, ASSIGNMENT, CONDITIONAL, TERNARY, INCREMENT AND DECREMENT, MATHEMATICAL FUNCTIONS 73

SUB LESSON 2.2..... 89

ARRAYS, TYPES OF ARRAY 89

SUB LESSON 2.3..... 96

CONTROL STATEMENTS 96

SUB LESSON 3.1..... 116

CLASS, OBJECT AND METHOD..... 116

SUB LESSON 3.2..... 127

METHOD OVERLOADING 127

SUB LESSON 3.3..... 135

STATIC KEYWORD..... 135

SUB LESSON 3.4..... 143

THIS KEYWORD..... 143

SUB LESSON 3.5..... 151

CONSTRUCTOR & TYPES OF CONSTRUCTOR, CONSTRUCTOR OVERLOADING, COPY CONSTRUCTOR, CONSTRUCTOR OVERLOADING 151

SUB LESSON 4.1..... 160

INHERITANCE & TYPES OF INHERITANCE..... 160

SUB LESSON 4.2..... 174

METHOD OVERRIDING 174

SUB LESSON 4.3..... 179

DYNAMIC METHOD DISPATCH..... 179

SUB LESSON 4.4..... 184

SUPER KEYWORD 184



SUB LESSON 4.5..... 191

FINAL KEYWORD 191

SUB LESSON 4.6..... 195

ABSTRACT CLASS 195

SUB LESSON 5.1..... 200

DEFINING INTERFACE, INHERITANCE ON INTERFACES, IMPLEMENTING INTERFACE, MULTIPLE INHERITANCE USING INTERFACE 200

SUB LESSON 6.1..... 208

CREATING PACKAGE, IMPORTING PACKAGE 208

SUB LESSON 6.2..... 215

ACCESS SPECIFIER, CLASSPATH..... 215

SUB LESSON 7.1..... 222

CREATING THREAD, EXTENDING THREAD CLASS, IMPLEMENTING RUNNABLE INTERFACE, LIFE CYCLE OF A THREAD..... 222

SUB LESSON 7.2..... 229

THREAD PRIORITIES & METHOD..... 229

SUB LESSON 7.3..... 235

THREAD SYNCHRONIZATION & MESSAGING 235

SUB LESSON 7.4..... 244

COMMUNICATION, SUSPENDING, RESUMING AND STOPPING THREAD..... 244

SUB LESSON 8.1..... 250

ERRORS, EXCEPTIONS, INBUILT EXCEPTION 250

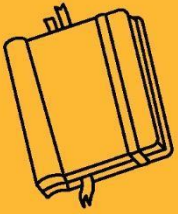
SUB LESSON 8.2..... 257

TRY..CATCH STATEMENT, MULTIPLE CATCH BLOCKS 257

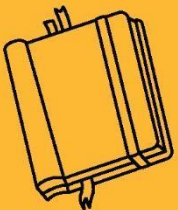
SUB LESSON 8.3..... 264



THROW AND THROWS KEYWORD, FINALLY CLAUSE.....	264
SUB LESSON 8.4.....	270
USES OF EXCEPTION, USER DEFINED EXCEPTION	270
SUB LESSON 9.1.....	275
STREAM CLASS, BYTE STREAM AND CHARACTER STREAM	275
SUB LESSON 9.2.....	280
READING CONSOLE INPUT, WRITING CONSOLE OUTPUT, PRINT WRITER CLASS, READING AND WRITING FILES	280
SUB LESSON 10.1	288
INTRODUCTION TO APPLETS, LIFECYCLE OF APPLET, WRITING JAVA APPLETS	288
SUB LESSON 10.2	297
GRAPHICS CLASS IN APPLET	297
SUB LESSON 10.3	303
IMAGE AND SOUND IN APPET	303
SUB LESSON 10.4	308
PASSING PARAMETER	308



OVERVIEW TO JAVA PROGRAMMING





SUB LESSON 1.1

BASICS OF JAVA, PROCEDURE-ORIENTED VS. OBJECT-ORIENTED PROGRAMMING, DIFFERENCE BETWEEN C, C++ & JAVA

OBJECTIVES OF CHAPTER 1

1.1 Basics of Java, Procedure-Oriented vs. Object-Oriented Programming, Difference between C, C++ & Java

- Understand the fundamentals of Java programming and its evolution.
- Compare procedural and object-oriented paradigms and distinguish between C, C++, and Java.

1.2 Basics of OOP

- Learn the key principles of Object-Oriented Programming like classes, objects, inheritance, polymorphism, encapsulation, and abstraction.
- Recognize how OOP concepts enhance modularity and reusability in Java.

1.3 Features & Advantages of Java

- Identify the key features of Java such as platform independence, robustness, and security.
- Understand how Java's features make it suitable for web, desktop, and mobile applications.

1.4 Java Virtual Machine & Byte Code, Java Environment Setup, Java Program Structure, Compile & Running Java Program

- Understand the role of JVM and how Java bytecode is executed.
- Learn the basic structure of a Java program, and how to compile and run Java programs.

1.5 Data Types of Java, Type Conversion & Casting



- Get familiar with Java's primitive and non-primitive data types.
- Understand implicit and explicit type conversions with casting examples.

1.6 Variables, Scope of Variable, Identifiers, Constant, Comments in Java

- Learn about variable types, scopes, naming rules, constants, and how to write effective comments.
- Understand how variables behave within different code blocks.

1.7 Wrapper Class

- Understand the purpose and use of wrapper classes in Java.
- Learn how to convert primitive types to objects and vice versa (autoboxing/unboxing).

1.8 String in Java

- Explore how to create, manipulate, and compare strings in Java using the String class and methods.
- Understand immutability and the use of StringBuffer and StringBuilder.

1.9 Garbage Collection

- Learn how Java handles memory management through automatic garbage collection.
- Understand how and when unused objects are cleaned up by the JVM.

1.10 Command Line Arguments

- Understand how to pass arguments to a Java program through the command line.
- Learn how to access and use command-line inputs in your code.

Java is a high-level, object-oriented programming language that is widely used for building platform-independent applications. Before diving into Java, it's important to understand how it differs from traditional procedural languages like C and C++.

BASICS OF JAVA



Java is an object-oriented programming language that is simple, secure, and easy to understand. It was developed to overcome the limitations of earlier languages like C and C++. Java is derived from C and C++, meaning its syntax is similar to C, which makes it easy for programmers who know C to learn Java.

However, unlike C++, Java removes complex features like pointers and operator overloading, making it safer and more beginner-friendly. Java uses the concepts of C++ such as classes, objects, inheritance, polymorphism, encapsulation, and abstraction but in a more structured and simplified way.

Java is a powerful, easy-to-learn programming language used to create a wide range of applications. It is an object-oriented, high-level language that follows the principle of "Write Once, Run Anywhere." This means that once you write a Java program, you can run it on any computer that has the Java Virtual Machine (JVM), regardless of the operating system.

Java is known for its simplicity, security, and platform independence. It is widely used to develop mobile apps (like Android apps), web applications, desktop software, games, and even scientific tools.

Java was originally developed by James Gosling and his team at Sun Microsystems in 1991. It was first named "Oak" after a tree outside Gosling's office but was later renamed "Java," inspired by Java coffee. Java was officially released in 1995 as a core part of Sun's Java platform. Its initial aim was to run on embedded devices, but it quickly gained popularity for internet-based applications. Over time, Java evolved with new versions and features, becoming one of the most widely used programming languages.

Java was divided into three main platforms:

1. Java SE (Standard Edition)
 - For desktop and simple server-side applications.
 - Includes core libraries and APIs like `java.lang`, `java.io`, `java.util`, etc.
2. Java EE (Enterprise Edition)
 - Used to build large-scale enterprise applications.



- Includes additional APIs like Servlets, JSP, EJB, Web Services, etc.
- 3. Java ME (Micro Edition)**
- Used for mobile and embedded devices with limited resources.

To write Java programs, we need software tools:

- JDK (Java Development Kit) – Used to write and run Java code.
- JRE (Java Runtime Environment) – Needed to run Java programs.
- JVM (Java Virtual Machine) – Converts Java bytecode into machine code.

We can use editors and IDEs like:

- Notepad / Notepad++ – Simple editors.
- Eclipse, NetBeans, IntelliJ IDEA – Powerful tools for Java development.

PROCEDURE-ORIENTED VS. OBJECT-ORIENTED PROGRAMMING

Procedure-Oriented Programming focuses on functions and step-by-step instructions to solve a problem, treating data as secondary. In contrast, Object-Oriented Programming is based on the concept of objects that combine both data and behavior. OOP is more suitable for large and complex programs, while POP is simple and works well for small tasks.

Procedure-Oriented Programming	Object-Oriented Programming
Program is divided into function.	Program is divided into Object
It deals with algorithm.	It deals with data.
It follows a top-down approach.	It follows a bottom-up approach.



It does not supports Inheritance.	It supports Inheritance.
Suited for small or simple programs.	Suited for large, complex systems.
Ex: C,FORTRAN	Ex: C++,JAVA

DIFFERENCE BETWEEN C/C++ AND DJAVA

C, C++, and Java are three widely-used programming languages, each with its own strengths and features. C is a structured, procedural language that provides low-level memory access and is mainly used for system programming. C++ builds on C by adding object-oriented programming features like classes and inheritance, making it suitable for both system and application software. Java was developed later and is a fully object-oriented language (except for primitive types), designed to be platform-independent using the Java Virtual Machine (JVM). Java also emphasizes security, simplicity, and automatic memory management through garbage collection. While C and C++ are compiled to machine code, Java is compiled to bytecode that can run on any platform

C	JAVA
C is Procedure Oriented Programming Language.	Java is Object Oriented Programming Language.
It is Platform Dependent.	It is Platform Independent.
C does not support inheritance.	Java supports inheritance.



C supports Pointers.	Java does not support pointers.
Multithreading is not supported .	Multithreading is supported .
C is not portable.	Java is Portable.
C is a Compiler Language.	Java is Compiler & Interpreter Language.
C follow top down approach.	Java follow bottom up approach.
C supports Structure & Union.	Java does not supports Structure & Union.

C++	JAVA
C++ is object Oriented Programming Language.	Java is pure Object Oriented Programming Language.
It is Platform Dependent.	It is Platform Independent.
C++ supports multiple Inheritance.	Java does not support Multiple Inheritance.
C++ supports Pointers.	Java does not support pointers.



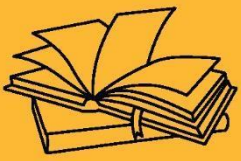
C++ is not portable.	Java is Portable.
C++ is a Compiler Language.	Java is Compiler & Interpreter Language.
C++ supports operator Overloading.	Java does not supports operator Overloading.
C++ supports Structure & Union.	Java does not supports Structure & Union.

CONCLUSION:

- C, C++, and Java are foundational programming languages, each evolving with added features and use cases.
- Java stands out with its platform independence, security features, and fully object-oriented approach, whereas C and C++ offer powerful system-level programming capabilities.

KEY TAKEAWAYS

- C is a procedural language best suited for system programming and low-level tasks.
- C++ is an extension of C that supports both procedural and object-oriented programming.
- Java is a platform-independent, object-oriented language that runs on the Java Virtual Machine (JVM).
- Java handles memory automatically using garbage collection, while C and C++ require manual memory management.
- Syntax in all three languages is similar, but Java removes complex features like pointers to make coding safer and easier. Inheritance allows code sharing between classes.



OVERVIEW TO JAVA PROGRAMMING





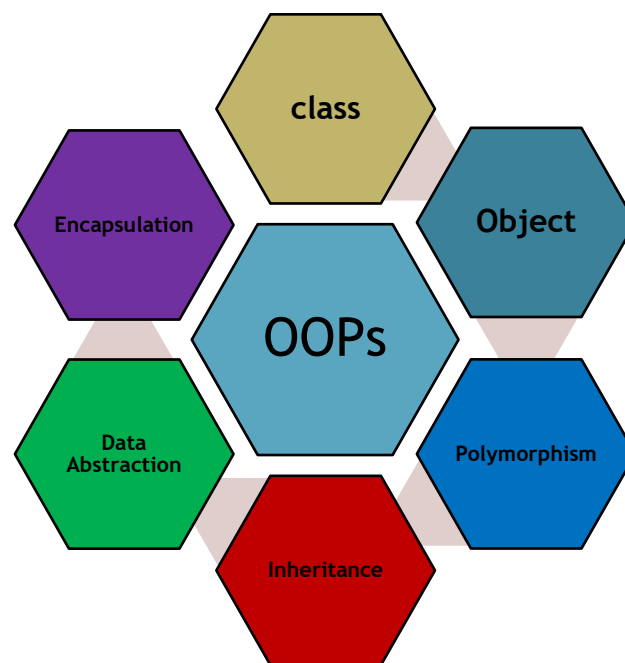
SUB LESSON 1.2

BASICS OF OOPS

Object-Oriented Programming (OOP) is a programming approach based on the concept of “objects,” which can contain data and methods. It helps in organizing complex programs, making them easier to manage, modify, and reuse. OOP promotes code clarity and reusability by combining related variables and functions into objects. Java is a purely object-oriented programming language that follows OOP principles strictly.

BASICS OF OOPS

Object-Oriented Programming (OOP) focuses on using objects to model real-world entities, making programs easier to understand and maintain. In OOP, classes and objects help organize code by grouping data and functions together in a structured way.



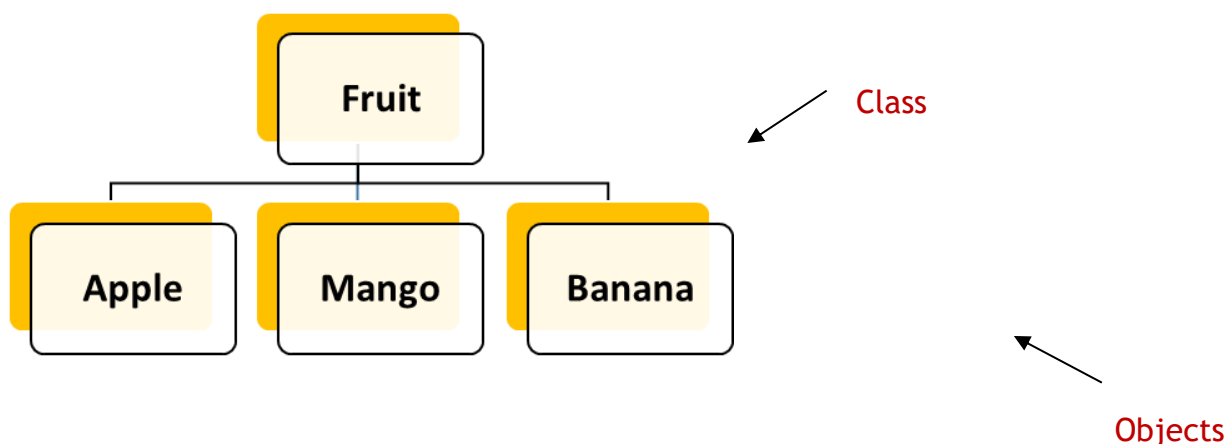


CLASS:

- A class is a blueprint or prototype from which objects are created.
- Classes are the basic blocks for object oriented programming in Java.
- Class is a use defined data type; all data and method are encapsulated within the class.
- The data and function written within the class are called as a member of a class. Class is a collection of Objects.
- A class is a blueprint for creating objects that share common properties and behaviour.
- Ex: Animal, Fruit, Students, Vehicle etc..

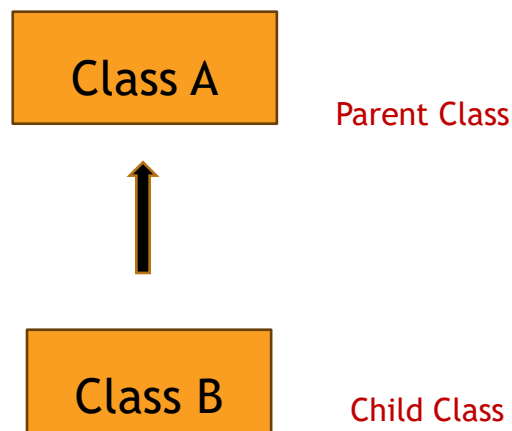
OBJECT:

- A Java object is one instance of class.
- An object is a block of memory that contains space to store all the instance variables.
- Creating an object is also referred as instantiating an object.
- Objects are created by using new operator. The new operator creates an object of the specified class and returns a reference to the object created.
- An object is an instance of a class that represents a real-world entity or concept.
- Ex: tiger, mango, car etc..



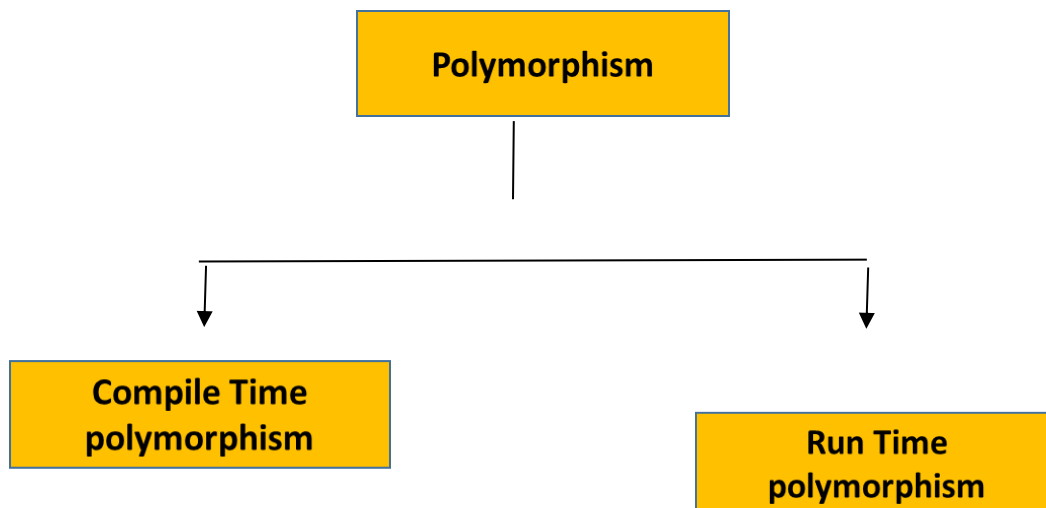
INHERITANCE:

- Inheritance allows a new class (child or subclass) to acquire the properties and behaviors of an existing class (parent or superclass).
- This promotes code reuse and establishes a natural hierarchy between classes.



POLYMORPHISM

- Polymorphism means “many forms” and refers to the ability of a method or object to behave differently based on context.
- It is mechanism in which one task is performed in different ways.
- Method overloading and method overloading are example of polymorphism.





ENCAPSULATION:

- Encapsulation is the process of hiding internal data and implementation details from the outside world.
- It is achieved by using access modifiers like private, public, and protected, and by providing public getter and setter methods.
- This keeps the data safe from unauthorized access and modification. Encapsulation improves maintainability, modularity, and security of the application.

ABSTRACTION:

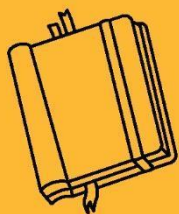
- Abstraction is the concept of hiding complex internal details and showing only essential features of an object.
- In Java, abstraction is achieved using abstract classes and interfaces. It helps in focusing on what an object does instead of how it does it.
- For example, when you use a smartphone, you interact with a simple interface without knowing the internal circuitry. Abstraction simplifies code and reduces complexity in large systems.

CONCLUSION:

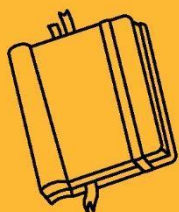
- Method overriding allows a subclass to change the behavior of a method inherited from its superclass.
- It is a key concept in runtime polymorphism, helping Java programs become more flexible and dynamic.

KEY TAKEAWAYS

- OOP is based on real-world modeling using classes and objects.
- It improves code reusability through inheritance.
- Encapsulation secures data and controls access.
- Polymorphism enhances flexibility in method usage.
- Abstraction hides complexity and shows relevant features.



OVERVIEW TO JAVA PROGRAMMING





SUB LESSON 1.3

FEATURES AND ADVANTAGES OF JAVA

Java is a powerful, high-level, object-oriented programming language developed by Sun Microsystems in 1995. It is widely used for building platform-independent, secure, and robust applications. Java's features make it ideal for developing web, mobile, and enterprise software.

FEATURES OF JAVA

There are many features that make java a very powerful tool and programming language to code with. Here are the features.

1. Simple.
2. Object Oriented Programming.
3. Robust.
4. Platform Independent.
5. Distributed.
6. Interpreted.
7. Dynamic.
8. Architecture Neutral.
9. Secure.
10. Multithreaded.



SIMPLE:

- Java has a syntax similar to C and C++ but simpler and easier to learn.
- It removes complex and error-prone features like pointers and operator overloading.
- Automatic memory management through garbage collection reduces programmer workload.
- Java enforces strong type checking which helps catch errors early.
- The language is designed to be readable and maintainable for developers of all levels.

OBJECT ORIENTED PROGRAMMING:

- Java is fully object-oriented, organizing software design around data, or objects.
- It supports key OOP concepts such as encapsulation, inheritance, and polymorphism.
- Objects interact through methods, promoting modular and reusable code.
- OOP makes Java programs easier to manage, modify, and scale.
- This approach closely models real-world entities, improving code clarity.

ROBUST:

- Java emphasizes reliability with strong compile-time and runtime checking.
- Automatic memory management and garbage collection help avoid memory leaks.
- Exception handling in Java enables graceful error detection and recovery.
- Strict type checking helps prevent many common programming errors.
- Java's platform ensures programs run without crashes caused by illegal operations.

PLATFORM INDEPENDENT:

- Java compiles source code into platform-neutral bytecode.
- Bytecode runs on any system with a Java Virtual Machine (JVM), regardless of underlying hardware or OS.
- This enables "Write Once, Run Anywhere" (WORA) functionality.
- It eliminates the need to rewrite or recompile code for different platforms.
- Platform independence makes Java ideal for web and enterprise applications.
- ensures programs run without crashes caused by illegal operations.



DISTRIBUTED:

- Java supports distributed computing with built-in network protocols like TCP/IP.
- Technologies like Remote Method Invocation (RMI) allow method calls across networks.
- Java provides support for developing applications that communicate over the internet.
- It enables the creation of distributed, scalable, and flexible software systems.
- Networking APIs in Java simplify tasks like socket programming and data sharing.

INTERPRETED:

- Java bytecode is interpreted by the JVM at runtime, making execution flexible.
- Interpretation allows Java programs to run on any device with a compatible JVM.
- It supports dynamic program execution and easier debugging.
- JVM optimizations like Just-In-Time (JIT) compilation improve performance.
- Interpreted nature simplifies development across diverse platforms.

DYNAMIC:

- Java supports dynamic loading of classes at runtime, making programs flexible.
- It can adapt to an evolving environment by linking new code while running.
- Java programs carry extensive metadata for runtime inspection and interaction.
- Reflection API allows inspection and modification of program behavior dynamically.
- This dynamic capability supports powerful frameworks and libraries.

ARCHITECTURE NEUTRAL:

- Java bytecode is independent of processor architecture or operating system.
- The JVM abstracts hardware and OS details from Java applications.
- This neutrality ensures consistent behavior and output on all platforms.
- It simplifies software distribution and reduces compatibility issues.
- Architecture neutrality helps Java thrive in heterogeneous environments.

SECURE:

- Java provides a secure execution environment with a built-in security manager.
- It restricts unauthorized access to system resources like file systems and networks.



- Bytecode verification ensures code integrity before execution.
- Java sandboxing prevents harmful programs from causing damage.

MULTITHREADED:

- Java supports multithreading to run multiple tasks concurrently within a program.
- Threads share resources but run independently, enabling efficient CPU use.
- Java provides built-in synchronization to handle thread conflicts and race conditions.
- Multithreading helps develop interactive and high-performance applications.
- It simplifies designing responsive GUIs and server-side applications.

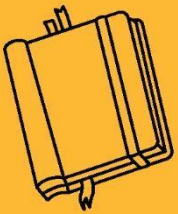
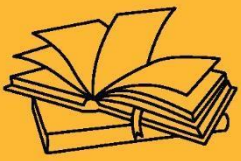
CONCLUSION:

- Java is a powerful, versatile, and easy-to-learn programming language designed with key features that promote reliability, portability, and security.
- Its object-oriented nature, platform independence, and robust architecture make it suitable for developing a wide range of applications from desktop to enterprise systems.
- Features like multithreading, dynamic class loading, and built-in security ensure Java remains a preferred choice for modern software development.



KEY TAKEAWAYS

- Java supports "Write Once, Run Anywhere" through platform-independent bytecode executed by the JVM.
- Object-oriented principles like encapsulation, inheritance, and polymorphism make Java programs modular and reusable.
- Automatic memory management and strong exception handling enhance program robustness and reduce errors.
- Java's security features protect applications from unauthorized access and vulnerabilities.
- Multithreading allows efficient execution of concurrent tasks improving application responsiveness.
- Java's extensive networking and distributed computing capabilities enable scalable and flexible software solutions.



OVERVIEW TO JAVA PROGRAMMING





SUB LESSON 1.4

JAVA VIRTUAL MACHINE & BYTE CODE, JAVA ENVIRONMENT SETUP, JAVA PROGRAM STRUCTURE, COMPILE & RUNNING JAVA PROGRAM

Before diving into the technical details of how Java programs work, it is essential to understand the internal working of Java and how it achieves platform independence. The concepts of Java Virtual Machine (JVM) and Byte Code are central to this understanding. Additionally, setting up the Java development environment properly is a foundational step for writing, compiling, and executing Java programs efficiently.

JAVA VIRTUAL MACHINE AND BYTECODE

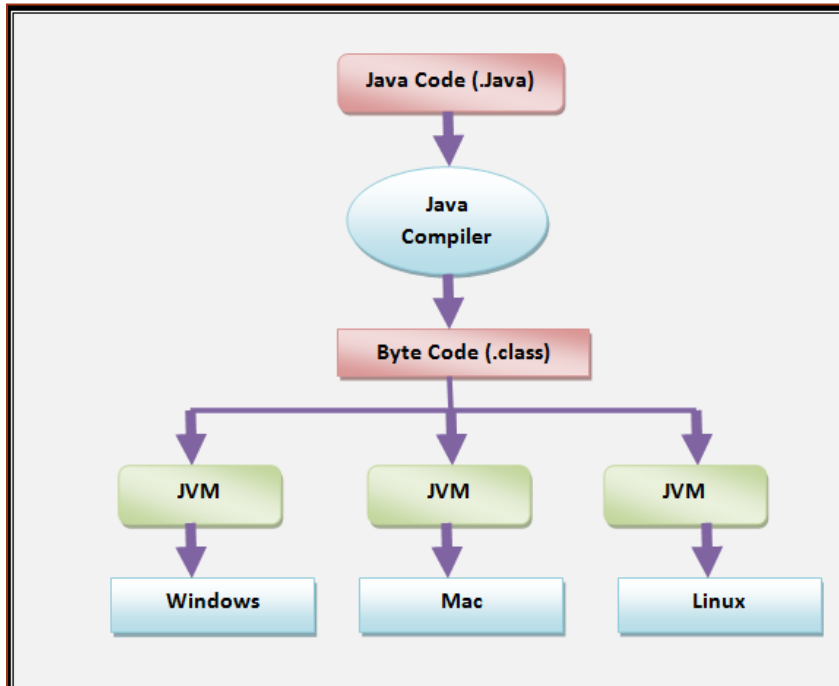
The Java Virtual Machine (JVM) is an abstract machine that enables your computer to run Java programs. It provides a platform-independent execution environment, meaning that Java code can be written once and run anywhere—on any machine that has a JVM installed.

- It is part of the Java Runtime Environment (JRE).
- JVM loads the .class file, verifies bytecode, and interprets or compiles it using JIT (Just-In-Time) compiler.
- It manages memory, garbage collection, and handles exceptions during program execution.

Bytecode is an intermediate, low-level representation of your Java source code, which the JVM can understand and execute.

- When a Java file (.java) is compiled, it is converted into bytecode (.class file).
- Bytecode is platform-independent and can be executed on any system with a JVM.

- It ensures security and portability of Java applications.



The image illustrates the platform-independent nature of Java through the compilation and execution process using the Java Virtual Machine (JVM). First, a Java program is written in a .java file, containing the source code. This code is then passed to the Java Compiler, which compiles the source code into bytecode, saved in a .class file. This bytecode is not specific to any operating system—instead, it is designed to be interpreted by any JVM, regardless of the platform.

As shown in the diagram, the same bytecode can be executed on different operating systems like Windows, Mac, or Linux, as long as they have their respective JVMs installed. Each JVM acts as a translator that converts bytecode into machine-specific instructions, enabling the code to run on various systems without modification. This entire process highlights the key Java principle: “Write Once, Run Anywhere.”



JAVA ENVIRONMENT SETUP:

The Java Environment is a combination of tools and software required to write, compile, debug, and execute Java programs. It mainly includes:

JDK (Java Development Kit)

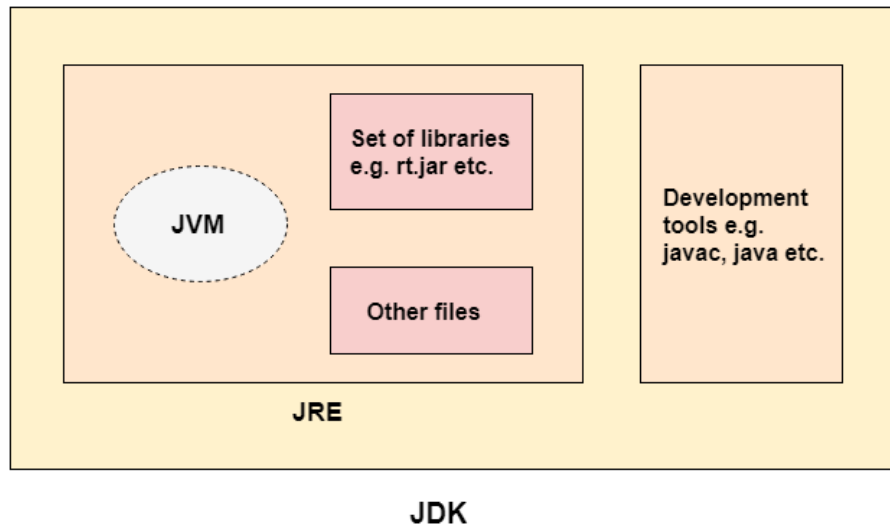
- It is a software package that includes tools for developing, debugging, and monitoring Java applications.
- It contains:
 - javac – Java Compiler
 - java – Java Application Launcher
 - javadoc – Documentation Generator
 - jar – Archive Tool
- JDK also contains the JRE.

JRE (Java Runtime Environment)

- Provides the libraries and JVM required to run Java applications.
- Does not contain development tools like compiler.
- Needed if you're only running Java programs (not writing).

JVM (Java Virtual Machine)

- Converts bytecode into machine code specific to the operating system.
- Platform-independent execution is possible through the JVM.



Java Environment Setup

- Install JDK(Java Development kit)
 - Visit the official Oracle site: <https://www.oracle.com/java>
 - Choose the correct version for your OS (Windows, Linux, macOS)
 - Install the software
- Run .exe file.
- Set the PATH variable using following instruction
 - Right click the Computer icon.
 - Choose Properties .
 - Click the Advanced system settings link.
 - Click Environment Variables and set PATH variable.
 - Run your java code.

JAVA PROGRAM STRUCTURE:

The structure of java program consists of mainly 5 different stages.

- Document Section
- Packages
- Import statements



- Classes and interface
- Main class

Structure of Java Program

Documentation Section

Package Statement

Import statement

class ClassName

{

Data Members;

user_defined Methods;

public static void main(String srgs[])

{

//statements;

}



```
}
```

Example

```
import java.io.*;
```

```
class Hello
```

```
{ //use for comment
```

```
//int a,b;
```

```
// void add();
```

```
public static void main(String srgs[])
```

```
{
```

```
System.out.println("Hello");
```

```
}
```

```
}
```

COMPILE AND RUNNING JAVAPROGRAM:

```
import java.io.*;
```



```
class Hello

{

    public static void main(String args[])

    {

        System.out.println("Hello");

    }

}
```

Step 1: Save the File

Save the program with the same name as the class (case-sensitive).

File name: HelloWorld.java

Step 2: Open Command Prompt or Terminal

Navigate to the folder where the file is saved.

Step 3: Compile the Java Program

```
javac HelloWorld.java
```

If there are no errors, it will create a file named HelloWorld.class (this is the bytecode).

Step 4: Run the Compiled Java Program

```
java HelloWorld
```

OUTPUT:

Hello, Java!



```
C:\Windows\system32\cmd.exe

- target <release>      Generate class files for specific VM version
- version              Version information
- help                 Print a synopsis of standard options
- Akey[=value]         Options to pass to annotation processors
- X                    Print a synopsis of nonstandard options
- J<flag>               Pass <flag> directly to the runtime system
- Werror               Terminate compilation if warnings occur
- R<filename>          Read options and filenames from file

C:\Users\HEIAL>cd..
C:\Users>e:
E:\>cd "A to Z JIT"
E:\A to Z JIT>cd Program
E:\A to Z JIT\Program>javac Hello.java
E:\A to Z JIT\Program>java Hello
Hello
E:\A to Z JIT\Program>
```

CONCLUSION:

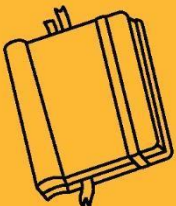
- Writing and executing a Java program involves creating a .java file, compiling it into bytecode using the javac compiler, and running it using the java command.
- Java's platform independence is achieved through the Java Virtual Machine (JVM), which executes the compiled bytecode on any operating system.

KEY TAKEAWAYS

- Java source files must be saved with .java extension.
- Class name and file name must match (especially for public classes).
- Use javac filename.java to compile and generate bytecode (.class file).
- Use java ClassName (without .class) to run the program.
- The main() method is the entry point of any Java application.
- Bytecode makes Java platform-independent, running on any JVM-supported OS.
- A simple System.out.println() is used to display output on the console.



OVERVIEW TO JAVA PROGRAMMING





SUB LESSON 1.5

DATA TYPES OF JAVA, TYPE CONVERSION & CASTING

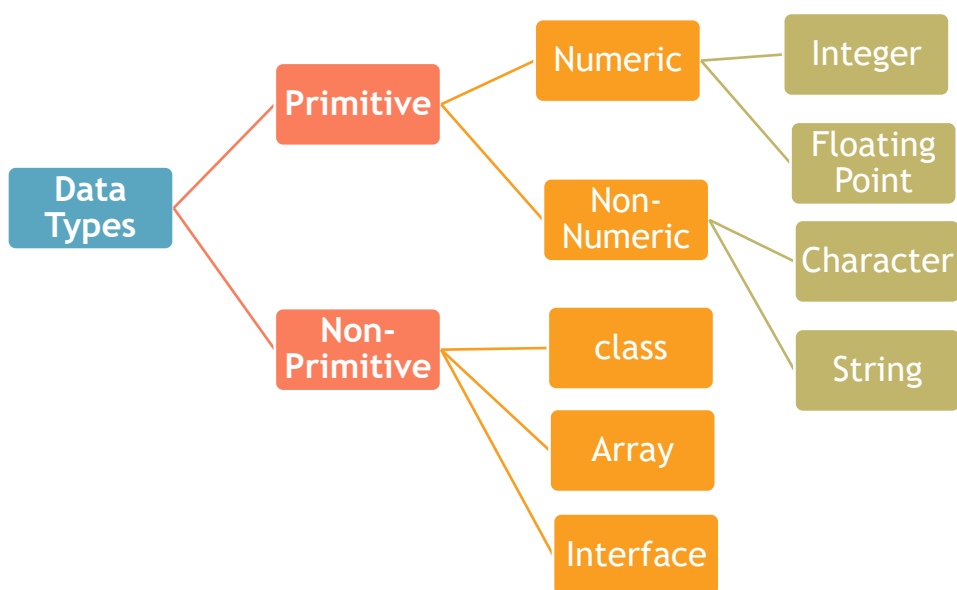
Java is a strongly typed language, which means every variable must be declared with a data type. Understanding data types, type conversion, and casting is essential for writing efficient and error-free Java programs.

METHOD OVERRIDING

Based on the data type of a variable, the operating system allocates memory and decides what can be stored in the reserved memory.

There are two data types available in Java –

- Primitive Data Types
- Non-Primitive/Object/Derived Data Types





PRIMITIVE DATA TYPES

A primitive type is predefined by the language and is named by a reserved keyword. Primitive values do not share state with other primitive values. The eight primitive data types supported by the Java programming language are:

- **Byte:** The byte data type is an 8-bit signed two's complement integer. It has a minimum value of -128 and a maximum value of 127 (inclusive). The byte data type can be useful for saving memory in large arrays, where the memory savings actually matters. They can also be used in place of int where their limits help to clarify your code; the fact that a variable's range is limited can serve as a form of documentation.
- **Short:** The short data type is a 16-bit signed two's complement integer. It has a minimum value of -32,768 and a maximum value of 32,767 (inclusive). As with byte, the same guidelines apply: you can use a short to save memory in large arrays, in situations where the memory savings actually matters.
- **int:** The int data type is a 32-bit signed two's complement integer. It has a minimum value of -2,147,483,648 and a maximum value of 2,147,483,647 (inclusive). For integral values, this data type is generally the default choice unless there is a reason (like the above) to choose something else. This data type will most likely be large enough for the numbers your program will use, but if you need a wider range of values, use long instead.
- **Long:** The long data type is a 64-bit signed two's complement integer. It has a minimum value of -9,223,372,036,854,775,808 and a maximum value of 9,223,372,036,854,775,807 (inclusive). Use this data type when you need a range of values wider than those provided by int.
- **Float:** The float data type is a single-precision 32-bit IEEE 754 floating point. As with the recommendations for byte and short, use a float (instead of double) if you need to save memory in large arrays of floating point numbers.



- Double: The double data type is a double-precision 64-bit IEEE 754 floating point.
- Boolean: The boolean data type has only two possible values: true and false. Use this data type for simple flags that track true/false conditions. This data type represents one bit of information, but its "size" isn't something that's precisely defined.
- char: The char data type is a single 16-bit Unicode character. It has a minimum value of '\u0000' (or 0) and a maximum value of '\uffff' (or 65,535 inclusive).

These are the most basic data types built into Java. There are 8 primitive types:

Data Type	Size	Default Value	Description
byte	1 byte	0	Small integer from -128 to 127
short	2 bytes	0	Larger than byte
int	4 bytes	0	Default integer type
long	8 bytes	0L	Very large integers
float	4 bytes	0.0f	Single-precision decimal numbers

NON PRIMITIVE DATA TYPES

Non-primitive data types (also called reference types) are not predefined like primitive types. They are created by the programmer and refer to objects, not actual values. These types are stored in heap memory, and their variable stores a reference (memory address), not the actual object.

1. String: A String is an object that represents a sequence of characters.

```
String name = "Java";
```

```
System.out.println(name.toUpperCase()); // Output: JAVA
```

2. Array: An array is a collection of elements of the same type.



```
int[] numbers = {10, 20, 30};
```

```
System.out.println(numbers[1]); // Output: 20
```

3. Class:

A class is a blueprint for creating objects with properties and methods.

```
class Student {  
  
    int rollNo;  
  
    String name;  
  
}
```

4. Interface: An interface is like a contract that a class agrees to follow by implementing its methods.

```
interface Animal {  
  
    void sound();  
  
}
```

5. Enum: enum is a special type used to define a collection of constants.

```
enum Day { MONDAY, TUESDAY, WEDNESDAY }
```

Type Conversion & Casting

The process of converting a value from one data type to another is known as type conversion.

- Widening or Automatic Conversion
 - Two data type are compatible.



- Assign lower value to higher data type.

Byte-short-int-long-float-double

Ex: `int i = 100; //100`

`float l = i; //100.0`

- Narrowing or Explicit Conversion

- Two data type are compatible.
- Source data type is greater than destination data type.

`double-float-long-int-short-Byte`

Ex: `double d = 100.04; //100.04`

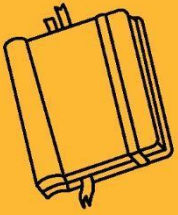
`int i = (int)d //100`

CONCLUSION:

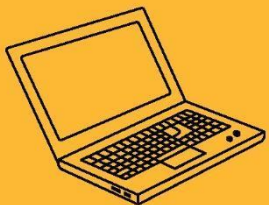
- Java provides a wide range of data types to handle different types of data effectively, ensuring memory efficiency and type safety.
- Type conversion and casting help in converting data from one type to another, allowing operations between different data types and improving code flexibility.

KEY TAKEAWAYS

- **Java** data types are broadly classified into primitive and non-primitive types.
- Primitive types (like `int`, `float`, `char`) hold simple values, while non-primitive types (like `String`, `Array`) hold references.
- Type conversion occurs in two forms: implicit (widening) and explicit (narrowing/casting).
- Implicit conversion happens automatically when moving from smaller to larger types (e.g., `int` to `float`).
- Explicit casting is required when converting from a larger to a smaller type (e.g., `double` to `int`) to avoid data loss.



OVERVIEW TO JAVA PROGRAMMING





SUB LESSON 1.6

VARIABLES, SCOPE OF VARIABLE, IDENTIFIERS, CONSTANT, COMMENTS IN JAVA

In Java, a variable is a container used to store data that can change during program execution, and it must be declared with a specific data type like int, float, or String. The scope of a variable refers to the region in the program where the variable is accessible, such as local, instance, or class scope. An identifier is the name given to variables, classes, or methods. A constant is a value that remains fixed throughout the program and is typically declared using the final keyword.

JAVA VARIABLES

In Java programming, a variable is a name given to a memory location that stores a value. It is called a variable because the value stored in it can be changed during the execution of the program. Think of a variable as a box with a label — you can put something inside it (like a number or word), take it out, or replace it with something else.

Variables help us write programs that can work with data. Instead of hardcoding values, we can store them in variables and use those variables wherever needed. This makes the code easier to read, write, and modify.

HOW TO DECLARE A VARIABLE IN JAVA?

Before using a variable, we must declare it by telling Java the type of data the variable will hold, followed by the variable name.

```
dataType variableName;
```

```
int age;
```



In the above line, we are telling Java that age is a variable that will store integer values (like 18, 25, 40, etc.).

RULES FOR NAMING VARIABLES (IDENTIFIERS)

- Names must begin with a letter, underscore (_), or dollar sign (\$).
- They cannot begin with a number.
- No spaces are allowed.
- They should not be Java keywords (like int, class, public).

```
int age;
```

```
float _height;
```

```
String $name;
```

Examples of invalid variable names:

```
int 2days;    // starts with number – wrong
```

```
float full name; // contains space – wrong
```

ASSIGN AND CHANGE VALUES TO VARIABLE

Once you declare a variable, you can give it a value using the assignment operator (=).

```
age = 25;
```

You can also declare and assign a value in one line:

```
int age = 25;
```

You can change the value of a variable anytime in the program.

```
int age = 20;
```

```
age = 21;
```



```
System.out.println("Age is: " + age);
```

TYPES OF VARIABLE

There are three types of variables in Java based on where they are declared:

1. LOCAL VARIABLES

Declared inside a method or block.

Only available within that block.

Not accessible from outside.

```
public void show() {  
  
    int number = 10; // Local variable  
  
    System.out.println(number);  
  
}
```

2. INSTANCE VARIABLES

Declared inside a class, but outside any method.

Belongs to an object of the class.

Each object gets its own copy.

```
public class Student {  
  
    String name; // Instance variable  
  
    void setName(String newName) {  
  
        name = newName;  
  
    }  
  
}
```



3. STATIC VARIABLES

Declared with the static keyword.

Shared by all objects of the class.

Memory is allocated only once.

```
public class School {  
  
    static String schoolName = "Greenwood High"; // Static variable  
  
    void showSchool() {  
  
        System.out.println(schoolName);  
  
    }  
  
}
```

SCOPE OF VARIABLE

In Java, the scope of a variable means where in the program you can use that variable. In other words, it defines how long the variable exists in memory and which parts of your code can access it.

Just like in real life, if something is available only in a particular room, you can't use it outside that room. Similarly, in Java, if a variable is declared in one part of the code, it might not be accessible in another part. This area of visibility is called the scope of the variable.

Understanding the scope of variables helps you:

1. Avoid name conflicts.
2. Write clean and error-free code.
3. Control memory usage by limiting variable life span.

TYPES OF VARIABLE SCOPE IN JAVA



Java has three main types of variable scope:

1. Local Scope
2. Instance Scope (Object Level)
3. Class Scope (Static Level)

1. LOCAL VARIABLE (LOCAL SCOPE)

A local variable is declared inside a method, constructor, or block. It can only be used inside that method or block. Once the method ends, the variable is destroyed automatically.

```
public class Example {  
  
    public void showMessage() {  
  
        String message = "Hello, Java!"; // Local variable  
  
        System.out.println(message);  
  
    }  
  
    public void anotherMethod() {  
  
        // System.out.println(message); // Error: message is not visible here  
  
    }  
  
}
```

2. INSTANCE VARIABLE (OBJECT SCOPE)

An instance variable is declared inside a class but outside all methods. It is part of the object created from the class, so each object has its own copy of the variable.

```
public class Student {  
  
    String name; // Instance variable
```



```
public void setName(String newName) {  
  
    name = newName;  
  
}  
  
public void printName() {  
  
    System.out.println("Student Name: " + name);  
  
}  
  
}  
  
public class Main {  
  
    public static void main(String[] args) {  
  
        Student s1 = new Student();  
  
        s1.setName("Alice");  
  
        s1.printName(); // Output: Student Name: Alice  
  
        Student s2 = new Student();  
  
        s2.setName("Bob");  
  
        s2.printName(); // Output: Student Name: Bob  
  
    }  
  
}
```

3. STATIC VARIABLE (CLASS SCOPE)

A static variable is also declared inside the class but with the keyword static. It is shared among all objects of the class. So, if one object changes its value, all other objects see the new value.

```
public class School {
```




```
static String schoolName = "Greenwood High"; // Static variable

public void showSchool() {

    System.out.println("School: " + schoolName);

}

}

public class Main {

    public static void main(String[] args) {

        School s1 = new School();

        s1.showSchool(); // Output: School: Greenwood High

        School.schoolName = "Sunrise School"; // Changing the value

        School s2 = new School();

        s2.showSchool(); // Output: School: Sunrise School

    }

}
```

IDENTIFIERS

In Java, identifiers are the names used to identify variables, methods, classes, objects, packages, arrays, and more. They are simply the names you give to different parts of your Java program. Imagine you are labeling different jars in your kitchen — sugar, salt, rice — so you can identify them easily. In the same way, Java uses identifiers to label different elements in your code. An identifier is the name you give to any Java element. This name helps the program recognize and work with the value or logic that it represents.

```
int age = 25;
```



```
String studentName = "Alice";
```

Java Identifier Rules

Java has a few strict rules for writing valid identifiers:

- ❖ Can use letters (A–Z or a–z), digits (0–9), underscore (_), and dollar sign (\$)
- ❖ Must begin with a letter, underscore (_), or dollar sign (\$)
- ❖ Cannot start with a digit
- ❖ Cannot use Java keywords as identifiers (e.g., int, class, if, while)
- ❖ Identifiers are case-sensitive

Score, score, and SCORE are three different identifiers.

```
int marks2;
```

```
float height;
```

```
String $name;
```

CONSTANT

In Java, a constant is a value that never changes while the program runs. Once you assign a value to a constant, you cannot change it again. Constants are useful when you want to protect important values from being changed by mistake. For example, if you are writing a program that calculates the area of a circle, the value of π (pi) is always 3.14159. You do not want someone accidentally change that value later in the code. Therefore, you make it a constant.

How to Create a Constant in Java

To declare a constant, Java uses the final keyword. This keyword tells the compiler, “This variable will never change.”

```
final dataType CONSTANT_NAME = value;
```

```
final double PI = 3.14159;
```



```
PI = 2.59; // Error: cannot assign a value to final variable
```

COMMENTS IN JAVA

In Java, comments are special lines in the code that are ignored by the compiler. They are written only for people reading the code, like programmers or students. Comments help explain what the code does, why something was written a certain way, or to temporarily disable a line of code during testing. Think of comments like notes in the margin of a notebook. They don't affect the actual program, but they help others (or your future self) understand your logic.

Types of Comments in Java

Java supports three types of comments:

1. Single-line Comment (//)

Used for short notes or explanations on one line.

```
int age = 18; // stores the user's age.
```

2. Multi-line Comment (/* ... */)

Used for writing longer notes or when you need multiple lines for explanation.

```
/*
```

This is a multi-line comment.

It explains several lines of code.

Useful for detailed instructions.

```
*/
```

```
int x = 10;
```

```
int y = 20;
```



```
int sum = x + y;
```

3. Documentation Comment (/** ... */)

This type is used when you want to generate official documentation for your code using Javadoc.

These comments are written above classes, methods, or variables.

```
/**
```

```
 * This class represents a simple calculator.
```

```
 * It can perform basic arithmetic operations.
```

```
 */
```

```
public class Calculator {
```

```
    /**
```

```
     * Adds two numbers.
```

```
     * a First number
```

```
     * b Second number
```

```
     * @return Sum of a and b
```

```
    */
```

```
    public int add(int a, int b) {
```

```
        return a + b;
```

```
    }
```

```
}
```

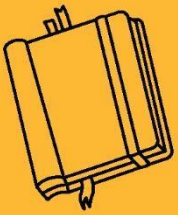
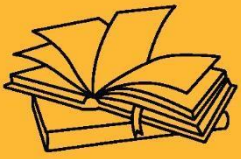


CONCLUSION:

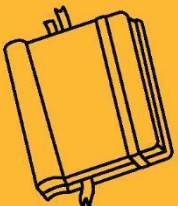
- Variables, scope, identifiers, constants, and comments are the basic parts of Java programming.
- They help in writing clear, correct, and organized code.
- Using them properly makes it easier to understand and change the code later.
- These basics are important for every beginner to learn first.
- Knowing these well will help you move to more advanced topics in Java easily.

KEY TAKEAWAYS

- Variables are used to store different types of values like numbers, text, etc.
- Scope tells where a variable can be used — inside a method, inside a class, or by the whole program.
- Identifiers are the names we give to things like variables and classes, and they should follow certain rules.
- Constants are values that do not change, and we use the final keyword to create them.
- Comments are notes in the code to help people understand it better, but they are not run by the computer.
- Bytecode makes Java platform-independent, running on any JVM-supported OS.
- A simple `System.out.println()` is used to display output on the console.



OVERVIEW TO JAVA PROGRAMMING





SUB LESSON 1.7

WRAPPER CLASS

In Java, every primitive data type (like `int`, `char`, `Boolean`) has a corresponding class known as a Wrapper Class. These classes "wrap" the primitive value into an object. In simple words, a wrapper class converts a primitive value into an object. Java is an object-oriented language, but primitive types are not objects. Sometimes, especially when working with collections like `ArrayList`, Java needs to use objects instead of simple primitive values. That is where wrapper classes come in handy.

LIST OF WRAPPER CLASSES

In Java, a Wrapper Class is used to convert primitive data types into objects. This is helpful because many data structures in Java, like collections (e.g., `ArrayList`), work only with objects. Each primitive type has a corresponding wrapper class—for example, `int` has `Integer`, `double` has `Double`, and so on. Wrapper classes also provide useful methods for converting between types and manipulating data.

Primitive Type	Wrapper Class
<code>byte</code>	<code>Byte</code>
<code>short</code>	<code>Short</code>
<code>int</code>	<code>Integer</code>
<code>long</code>	<code>Long</code>
<code>float</code>	<code>Float</code>
<code>double</code>	<code>Double</code>



char	Character
boolean	Boolean

WHY USE WRAPPER CLASSES?

1. To store primitive values in collection objects like ArrayList, which only store objects.
2. To use object methods for operations like parsing, comparing, converting, etc.
3. To take advantage of Java's object features such as polymorphism and generics.
4. To convert strings into numbers, using methods like Integer.parseInt ("123").
5. To make primitive data types usable in frameworks, which mostly work with objects.

Example 1: Wrapping a Primitive Value

```
int num = 100; // primitive
```

```
Integer obj = Integer.valueOf(num); // wrapping into object
```

```
System.out.println("Object value: " + obj);
```

Autoboxing and Unboxing in Java

Java 5 introduced Autoboxing and Unboxing to make code simpler.

```
int x = 50;
```

```
Integer y = x; // autoboxing
```

Unboxing:

Automatically converting a wrapper object back to a primitive.



```
Integer a = 100;
```

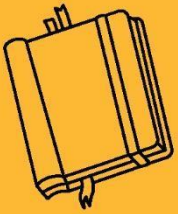
```
int b = a; // unboxing
```

CONCLUSION:

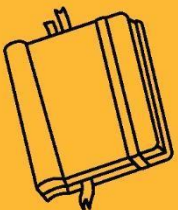
- Wrapper classes allow primitive data types to be used as objects in Java.
- They are especially useful in collections, frameworks, and object-oriented operations.
- With the help of autoboxing and unboxing, Java simplifies the conversion between primitives and objects.
- Wrapper classes make it easier to use methods and utilities that aren't available for primitive types.

KEY TAKEAWAYS

- Java provides a wrapper class for each primitive type (e.g., `int` → `Integer`, `char` → `Character`).
- Wrapper classes are used to convert primitive values into objects and vice versa.
- Autoboxing automatically converts a primitive to a wrapper object; unboxing does the reverse.
- Wrapper classes allow primitives to be stored in collections like `ArrayList`, which only accept objects.



OVERVIEW TO JAVA PROGRAMMING





SUB LESSON 1.8

STRING IN JAVA

In Java, a String is a sequence of characters. It is used to store and manipulate text, such as words, sentences, or any group of characters. A string is not a primitive data type, but Java makes it very easy to work with strings because it provides a built-in String class in the java.lang package. Strings in Java are immutable, which means once a string object is created, it cannot be changed. If you try to modify a string, Java actually creates a new string object in memory.

CREATING A STRING IN JAVA

There are two common ways to create strings in Java:

1. Using String Literal

```
String name = "John";
```

- In this method, the string is stored in the String Pool (a special memory area).
- If the same string is used again, Java will reuse it instead of creating a new object.

2. Using new Keyword

```
String name = new String("John");
```

- This always creates a new string object in memory, even if the value is the same.

String Immutability Example

```
String str1 = "Hello";
```

```
str1.concat(" World");
```

```
System.out.println(str1); // Output: Hello
```



Even though `concat()` was used, `str1` is still "Hello". Why? Because Strings are immutable. The result of `concat()` is a new string, but we didn't store it in a new variable.

To get the changed string:

```
String str2 = str1.concat(" World");
```

```
System.out.println(str2); // Output: Hello World
```

COMMON STRING METHODS

Java's `String` class provides many built-in methods to perform operations like comparing, joining, finding, and changing strings.

Method	Description	Example
<code>length()</code>	Returns the number of characters	<code>"Hello".length()</code> → 5
<code>charAt(index)</code>	Returns the character at a specific index	<code>"Java".charAt(2)</code> → 'v'
<code>concat(str)</code>	Joins two strings	<code>"Good".concat(" Morning")</code> → "Good Morning"
<code>equals(str)</code>	Checks if two strings are equal	<code>"a".equals("a")</code> → true
<code>equalsIgnoreCase(str)</code>	Case-insensitive comparison	<code>"java".equalsIgnoreCase("JAVA")</code> → true
<code>substring(start, end)</code>	Returns part of the string	<code>"Hello".substring(1, 4)</code> → "ello"



<code>toLowerCase()</code> <code>toUpperCase()</code>	/ Converts case	<code>"Java".toLowerCase() → "java"</code>
<code>trim()</code>	Removes spaces at the start and end	<code>" Hello ".trim() → "Hello"</code>

STRING COMPARISON

Using `==` Operator

```
String a = "Java";
```

```
String b = "Java";
```

```
System.out.println(a == b); // true (same object in memory)
```

Using `.equals()`

```
String a = new String("Java");
```

```
String b = new String("Java");
```

```
System.out.println(a == b); // false (different objects)
```

```
System.out.println(a.equals(b)); // true (same value)
```

Always use `.equals()` to compare string values.

STRING CONCATENATION

You can join strings using:

1. `+` Operator:



```
String first = "Good";
```

```
String second = "Morning";
```

```
String result = first + " " + second;
```

```
System.out.println(result); // Output: Good Morning
```

2. `concat()` Method:

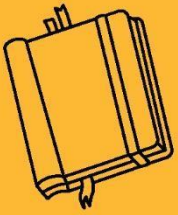
```
String result = first.concat(" ").concat(second);
```

CONCLUSION:

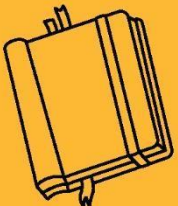
- Strings in Java are objects used to store and manipulate text.
- They are immutable, so every change creates a new string.
- Java provides many built-in methods for common string operations.

KEY TAKEAWAYS

- Use double quotes to create a string like "Hello".
- String values should be compared using `.equals()` method.
- Common methods like `substring()`, `length()`, and `toUpperCase()` make string tasks easier.



OVERVIEW TO JAVA PROGRAMMING





SUB LESSON 1.9

GARBAGE COLLECTION

GARBAGE COLLECTION

In Java, garbage collection is the process by which the Java Virtual Machine (JVM) automatically removes unused objects from memory to free up space. This helps in managing memory efficiently and prevents memory leaks. When an object is no longer reachable (meaning no part of the code can access it), Java considers it as garbage and automatically deletes it using the Garbage Collector (GC). Unlike languages like C or C++, in Java, programmers do not need to manually deallocate memory. The JVM handles it in the background.

WHY IS GARBAGE COLLECTION IMPORTANT?

- It prevents memory overflow by cleaning up unused objects.
- It simplifies programming by automatically managing memory.
- It ensures efficient use of system resources, especially in large applications.

HOW DOES GARBAGE COLLECTION WORK?

Java uses a concept called reference counting and reachability to determine whether an object is still in use.

```
public class Example {  
  
    public static void main(String[] args) {  
  
        String name = new String("John");  
  
        name = null; // The object "John" is now eligible for garbage collection  
  
    }  
  
}
```




In the code above:

- The object "John" is created using the new keyword.
- When we assign name = null;, the original object is no longer referenced.
- The garbage collector will now identify it as unused and remove it from memory when it runs.

WHEN DOES GARBAGE COLLECTION HAPPEN?

Garbage collection doesn't happen at a fixed time. It is automatically triggered by the JVM when:

- Memory is low
- The system is idle
- The application has created many objects

You can request garbage collection manually using:

```
System.gc();
```

EXAMPLE

```
public class GarbageTest {  
  
    public static void main(String[] args) {  
  
        GarbageTest obj1 = new GarbageTest();  
  
        GarbageTest obj2 = new GarbageTest();  
  
        obj1 = null;    // eligible for GC  
  
        obj2 = null;    // eligible for GC  
  
        System.gc();    // Request garbage collection  
  
    }  
}
```



```
protected void finalize() throws Throwable {  
  
    System.out.println("Garbage collected!");  
  
}  
  
}
```

Output:

Garbage collected!

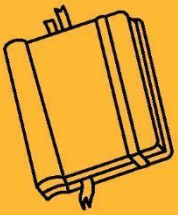
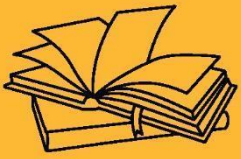
Garbage collected!

CONCLUSION:

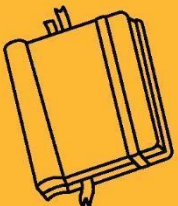
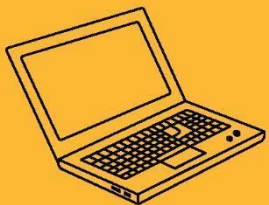
- Java's garbage collection automatically clears unused memory, making memory management easier.
- It ensures that objects that are no longer in use are cleaned up to keep the program efficient.
- The garbage collector runs in the background, and developers don't need to worry about freeing memory.
- Although developers can request GC, it is best to let JVM decide when to collect garbage.

KEY TAKEAWAYS

- Garbage Collection in Java helps in automatic memory management.
- Objects with no reference become eligible for collection.
- Java offers different types of garbage collectors for different needs.
- Using `System.gc()` only suggests collection, but doesn't guarantee it.



OVERVIEW TO JAVA PROGRAMMING





SUB LESSON 1.10

COMMAND LINE ARGUMENTS

COMMAND LINE ARGUMENTS

Command line arguments are inputs provided to a Java program at the time of running it, not during coding or inside the program. These arguments are passed when we execute the program from the command prompt (or terminal). They allow users to pass information into the program dynamically.

In Java, the `main()` method receives these arguments through an array called:

```
public static void main(String[] args);
```

EXAMPLE PROGRAM: USING COMMAND LINE ARGUMENTS

```
public class CommandLineExample {  
  
    public static void main(String[] args) {  
  
        System.out.println("You entered " + args.length + " arguments:");  
  
        for (int i = 0; i < args.length; i++) {  
  
            System.out.println("Argument " + (i+1) + ": " + args[i]);  
  
        }  
  
    }  
  
}
```

Save the file as: `CommandLineExample.java`

Compile the code:



```
javac CommandLineExample.java
```

Run with command line arguments:

```
java CommandLineExample Hello World 123
```

Output:

You entered 3 arguments:

Argument 1: Hello

Argument 2: World

Argument 3: 123

All command line arguments are received as strings. If you need to use them as numbers, you must convert them.

EXAMPLE: CONVERTING TO INTEGER

```
public class SumArguments {  
  
    public static void main(String[] args) {  
  
        if (args.length < 2) {  
  
            System.out.println("Please provide two numbers.");  
  
            return;  
  
        }  
  
        int num1 = Integer.parseInt(args[0]);  
  
        int num2 = Integer.parseInt(args[1]);  
  
        int sum = num1 + num2;
```



```
        System.out.println("Sum is: " + sum);  
    }  
}
```

Run:

```
java SumArguments 10 20
```

Output:

```
Sum is: 30
```

CONCLUSION:

- Command line arguments let users provide input when running the program.
- They are stored as `String[]` args in the `main()` method.
- You can loop through the args array to access each argument.

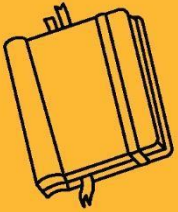
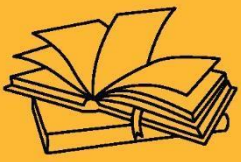
KEY TAKEAWAYS

- Java command line arguments are passed through the `main(String[] args)` method.
- They are always in string format and may need conversion to other types.
- They allow dynamic input during program execution.



OUTCOMES OF CHAPTER 1

- After completing this unit, students will be able to:
- Understand the basic concepts of Java, how it differs from procedural languages like C and C++, and its object-oriented nature.
- Explain and apply OOP principles such as encapsulation, inheritance, and polymorphism in Java programs.
- Identify and use Java features like platform independence, security, and robustness, and understand how JVM and bytecode work.
- Work with data types, variables, type casting, and understand concepts like scope, constants, comments, and wrapper classes.
- Write, compile, and run Java programs using proper syntax and structure, including command line arguments, strings, and garbage collection mechanisms.



OVERVIEW TO JAVA PROGRAMMING





SUB LESSON 2.1

DIFFERENT OPERATORS: ARITHMETIC, BITWISE, RATIONAL, LOGICAL, ASSIGNMENT, CONDITIONAL, TERNARY, INCREMENT AND DECREMENT, MATHEMATICAL FUNCTIONS

OBJECTIVES OF CHAPTER 2

2.1 Understanding of Operators and Mathematical Functions

- Understand the purpose and usage of various operators in programming, including arithmetic, bitwise, relational, logical, assignment, conditional, ternary, increment, and decrement operators.
- Apply mathematical functions effectively to perform complex calculations within a program.
- Demonstrate the ability to utilize operators for manipulating data and controlling program flow efficiently.

2.2 Exploring Arrays for Data Management

- Understand the concept of arrays and their significance in storing multiple values of the same data type.
- Implement arrays in a programming language to store, access, and manipulate data efficiently.
- Apply array operations, including traversal, insertion, deletion, and searching, in problem-solving scenarios.

2.3 Implementing Flow Control Statements in Programming

- Understand the role of control statements in managing the flow of a program, including decision-making and looping structures.
- Implement various control statements, such as if-else, switch, for, while, and do-while, to solve real-world programming problems.



Operators are symbols or keywords used to perform operations on variables and values in programming. They include arithmetic, bitwise, relational, logical, assignment, conditional, ternary, increment, and decrement operators, as well as mathematical functions.

OPERATORS

- Operators operate on operands and cause changes in the operand value or give a new value.
 - Operators are special symbols that perform specific operations on one, two, or three *operands*, and then return a result.
 - There are various types of operators and operations performed by java. The different types of operators are:
1. Arithmetic Operator
 2. Relational operator
 3. Bitwise operator
 4. Logical operator
 5. Assignment operator
 6. Conditional operator
 7. Increment and decrement operator

ARITHMETIC OPERATORS:

- This operator is used to perform various mathematical operations in java. The following is the list of operations performed by arithmetic operator.
- Let's Take A= 10 , B=5

Operator	Name	Explanation
----------	------	-------------



+	Addition	A+B=15
-	Subtraction	A-B=5
*	Multiplication	A*B=50
/	Division	A/B=2
%	Modulo	A%B = 0

Example 1

```
public class ArithmeticOperators
```

```
{
```

```
    public static void main(String[] args)
```

```
    {
```

```
        int num1 = 20, num2 = 10;
```

```
        // Addition
```

```
        int sum = num1 + num2;
```

```
        System.out.println("Addition: " + num1 + " + " + num2 + " = " + sum);
```

```
        // Subtraction
```

```
        int difference = num1 - num2;
```

```
        System.out.println("Subtraction: " + num1 + " - " + num2 + " = " + difference);
```

```
        // Multiplication
```



```
int product = num1 * num2;

System.out.println("Multiplication: " + num1 + " * " + num2 + " = " + product);

// Division

int quotient = num1 / num2;

System.out.println("Division: " + num1 + " / " + num2 + " = " + quotient);

// Modulus

int remainder = num1 % num2;

System.out.println("Modulus: " + num1 + " % " + num2 + " = " + remainder);

}

}
```

Output

Addition: 20 + 10 = 30

Subtraction: 20 - 10 = 10

Multiplication: 20 * 10 = 200

Division: 20 / 10 = 2

Modulus: 20 % 10 = 0

RELATIONAL OPERATOR:

Relational operators in Java are used to compare two values and return a boolean result (true or false). These operators include ==, !=, >, <, >=, and <=, and are commonly used in conditional statements. They help in evaluating expressions and controlling program flow based on comparisons. This operator is used to find the relation between the two operands.



- Following are the relational operators.

Operator	Name	Explanation
==	Equal to	A==B? False
!=	Not Equal to	A!=B? True
>	Greater than	A>B? False
<	Less than	A<B? True
>=	Greater than or equal to	A>=B? False
<=	Less than or equal to	A<=B? True

Example 2

```
public class RelationalOperators
{
    public static void main(String[] args)
    {
        int a = 10, b = 20;

        System.out.println("a == b: " + (a == b)); // Equal to

        System.out.println("a != b: " + (a != b)); // Not equal to

        System.out.println("a > b: " + (a > b)); // Greater than

        System.out.println("a < b: " + (a < b)); // Less than
```



```
System.out.println("a >= b: " + (a >= b)); // Greater than or equal to
```

```
System.out.println("a <= b: " + (a <= b)); // Less than or equal to
```

```
}
```

```
}
```

Output

a == b: false

a != b: true

a > b: false

a < b: true

a >= b: false

a <= b: true

BITWISE OPERATORS:

Bitwise operators in Java perform operations on individual bits of integer values. These operators include & (AND), | (OR), ^ (XOR), ~ (NOT), << (left shift), >> (right shift), and >>> (unsigned right shift). They are useful for tasks like bit manipulation, setting or clearing specific bits, and performing efficient arithmetic operations. Bitwise operators are particularly valuable in low-level programming and performance-critical applications. They work directly on binary representations of data.

Operator	Description
&	It is binary and operation



	It is binary OR operation
^	It is binary XOR operation
~	It is binary ones complement operation
<<	It is left shift operation
>>	It is right shift operation

This operator works on bits to perform bit by bit operations. Suppose that if $a=8$ and $b=4$ then the binary format will be as:

- $a=1000$ and
- $b=0100$ then operations like
- $a \& b=0000$
- $a | b=1100$
- $a \wedge b=1100$
- $\sim a=0111$

LOGICAL OPERATOR:

Logical operators in Java are used to combine multiple boolean expressions and return a boolean result (true or false). The primary logical operators are $\&\&$ (AND), $||$ (OR), and $!$ (NOT). These operators are typically used in conditional statements to evaluate multiple conditions at once. They help in controlling the flow of a program based on complex logical conditions.



A	B	A B	A&B	A^B	!A
0	0	0	0	0	1
0	1	1	0	1	1
1	0	1	0	1	0
1	1	1	1	0	0

Example

```
public class LogicalOperators
{
    public static void main(String[] args)
    {
        boolean x = true;

        boolean y = false;

        // AND operator (&&)

        System.out.println("x && y: " + (x && y)); // false

        // OR operator (||)

        System.out.println("x || y: " + (x || y)); // true
    }
}
```




```
// NOT operator (!)

System.out.println("!x: " + (!x));    // false

System.out.println("!y: " + (!y));    // true

// Combining logical operators

System.out.println("(x && !y) || (!x && y): " + ((x && !y) || (!x && y))); // true

}

}
```

Output

x && y: false

x || y: true

!x: false

!y: true

(x && !y) || (!x && y): true

ASSIGNMENT OPERATORS:

Assignment operators in Java are used to assign values to variables. The basic assignment operator is =, which assigns the right-hand value to the left-hand variable. Other compound assignment operators include +=, -=, *=, /=, and %=, which combine arithmetic operations with assignment. These operators make code more concise and efficient.

Here is a list of assignment operators in Java:

1. = → Assigns a value to a variable.



- Example: `a = 10;`
- 2. `+=` → Adds and assigns the result.**
 - Example: `a += 5;` (equivalent to `a = a + 5;`)
- 3. `-=` → Subtracts and assigns the result.**
 - Example: `a -= 3;` (equivalent to `a = a - 3;`)
- 4. `*=` → Multiplies and assigns the result.**
 - Example: `a *= 2;` (equivalent to `a = a * 2;`)
- 5. `/=` → Divides and assigns the result.**
 - Example: `a /= 4;` (equivalent to `a = a / 4;`)
- 6. `%=` → Finds the remainder and assigns the result.**
 - Example: `a %= 3;` (equivalent to `a = a % 3;`)
- 7. `&=` → Bitwise AND and assignment.**
 - Example: `a &= 2;` (equivalent to `a = a & 2;`)
- 8. `|=` → Bitwise OR and assignment.**
 - Example: `a |= 1;` (equivalent to `a = a | 1;`)
- 9. `^=` → Bitwise XOR and assignment.**
 - Example: `a ^= 3;` (equivalent to `a = a ^ 3;`)
- 10. `<<=` → Left shift and assignment.**
 - Example: `a <<= 2;` (equivalent to `a = a << 2;`)
- 11. `>>=` → Right shift and assignment.**
 - Example: `a >>= 1;` (equivalent to `a = a >> 1;`)

CONDITIONAL OPERATOR

The conditional operator in Java, also known as the ternary operator, is a shorthand way of writing an if-else statement.

Syntax:

```
variable = (condition) ? value_if_true : value_if_false;
```



Example

```
public class ConditionalOperator {  
  
    public static void main(String[] args) {  
  
        int a = 10;  
  
        int b = 20;  
  
  
        // Using the conditional (ternary) operator  
  
        String result = (a > b) ? "a is greater" : "b is greater or equal";  
  
  
        System.out.println("Comparison Result: " + result);  
  
  
        // Another example: checking if a number is even or odd  
  
        int num = 15;  
  
        String parity = (num % 2 == 0) ? "Even" : "Odd";  
  
  
        System.out.println("Number " + num + " is: " + parity);  
  
    }  
  
}
```

Output

Comparison Result: b is greater or equal



Number 15 is: Odd

INCREMENT /DECREMENT OPERATOR

Increment and decrement operators in Java are used to increase or decrease the value of a variable by 1.

1. Increment Operator (++):

- Pre-increment (++a): Increments the value first, then uses it.
- Post-increment (a++): Uses the value first, then increments it.

2. Decrement Operator (--):

- Pre-decrement (--a): Decrements the value first, then uses it.
- Post-decrement (a--): Uses the value first, then decrements it.

Example

```
public class IncrementDecrementOperators {  
  
    public static void main(String[] args) {  
  
        int a = 5;  
  
        int b = 5;  
  
        // Pre-increment  
  
        System.out.println("Pre-increment: ++a = " + (++a)); // Increments first, then prints  
  
        // Post-increment
```



```
System.out.println("Post-increment: b++ = " + (b++)); // Prints first, then increments

System.out.println("Value of b after post-increment: " + b); // Checking updated value

// Pre-decrement

int x = 10;

System.out.println("Pre-decrement: --x = " + (--x)); // Decrements first, then prints

// Post-decrement

int y = 10;

System.out.println("Post-decrement: y-- = " + (y--)); // Prints first, then decrements

System.out.println("Value of y after post-decrement: " + y); // Checking updated value

}

}
```

Output

Pre-increment: ++a = 6

Post-increment: b++ = 5

Value of b after post-increment: 6

Pre-decrement: --x = 9

Post-decrement: y-- = 10

Value of y after post-decrement: 9



MATHEMATICAL FUNCTION

The Math class in Java, part of the java.lang package, provides various methods to perform mathematical operations. Since java.lang is automatically imported, you can use the Math class without any additional import statement. The methods in the Math class are static, so they can be called directly using the class name.

Function	Syntax	Description	Example
Absolute Value	Math.abs(x)	Returns the absolute value of x.	Math.abs(-5) → 5
Power	Math.pow(x, y)	Returns x raised to the power of y.	Math.pow(2, 3) → 8
Square Root	Math.sqrt(x)	Returns the square root of x.	Math.sqrt(16) → 4
Maximum	Math.max(x, y)	Returns the greater of x and y.	Math.max(5, 10) → 10
Minimum	Math.min(x, y)	Returns the lesser of x and y.	Math.min(5, 10) → 5
Rounding	Math.round(x)	Rounds x to the nearest integer.	Math.round(3.6) → 4
Ceiling	Math.ceil(x)	Rounds x upward to the nearest integer.	Math.ceil(4.3) → 5
Floor	Math.floor(x)	Rounds x downward to the nearest integer.	Math.floor(4.8) → 4

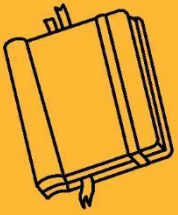
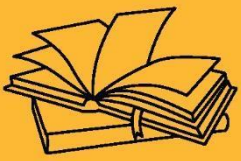
CONCLUSION:



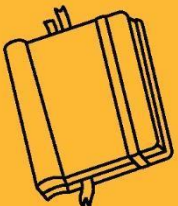
- Operators are essential in programming for performing a wide range of operations, from arithmetic to logical comparisons. They make data manipulation and control flow more efficient.
- Using operators effectively reduces code complexity and enhances readability, especially when combining conditions and performing calculations.
- Mathematical functions in Java simplify complex calculations, including power, square root, and trigonometric operations, through easy-to-use methods.

KEY TAKEAWAYS

- Operators are used to perform various operations such as arithmetic, comparison, assignment, and logic.
- Mathematical functions provide predefined formulas to perform complex calculations easily.



OVERVIEW TO JAVA PROGRAMMING





SUB LESSON 2.2

ARRAYS, TYPES OF ARRAY

ARRAYS

An array in Java is a data structure that stores a collection of similar type values. Instead of creating multiple variables to store multiple values, an array lets you store them in a single variable.

Example:

If you want to store the marks of five students, you can do this:

```
int[] marks = {85, 90, 78, 88, 92};
```

- To store multiple values in a single variable.
- To loop through a set of values easily.
- To organize data of the same type in a structured format.
- Arrays are fixed in size once declared.
- All elements are stored in contiguous memory locations.
- Each element can be accessed using an index, starting from 0.
- Arrays can be of primitive types or objects.

SYNTAX

```
dataType[] arrayName;    // Declaration
```

```
arrayName = new dataType[size]; // Initialization
```

DECLARE AND INITIALIZE ARRAYS IN JAVA

Declare the array

```
int[] numbers;
```



```
numbers = new int[5]; // An array of 5 integers
```

Initialize values

```
numbers[0] = 10;
```

```
numbers[1] = 20;
```

```
numbers[2] = 30;
```

```
numbers[3] = 40;
```

```
numbers[4] = 50;
```

Or

```
int[] numbers = {10, 20, 30, 40, 50};
```

Accessing Array Elements

```
System.out.println(numbers[2]); // Output: 30
```

```
System.out.println(numbers[4]); // Output: 50
```

TYPES OF ARRAYS IN JAVA

Java supports two main types of arrays:

2.1 Single-Dimensional Array: This is the most basic type. It stores values in a single row or line.

Example

```
import java.util.Scanner;
```

```
public class ArrayExample {
```

```
    public static void main(String[] args) {
```

```
        int[] numbers = new int[10]; // Array to store 10 elements
```



```
Scanner sc = new Scanner(System.in);

// Input 10 elements

System.out.println("Enter 10 integer elements:");

for (int i = 0; i < 10; i++) {

    System.out.print("Element " + (i + 1) + ": ");

    numbers[i] = sc.nextInt();

}

// Output 10 elements

System.out.println("\nThe elements in the array are:");

for (int i = 0; i < 10; i++) {

    System.out.println("Element " + (i + 1) + ": " + numbers[i]);

}

sc.close();

}
```

Output

Enter 10 integer elements:

Element 1: 10

Element 2: 20

Element 3: 30



Element 4: 40

Element 5: 50

Element 6: 60

Element 7: 70

Element 8: 80

Element 9: 90

Element 10: 100

The elements in the array are:

Element 1: 10

Element 2: 20

Element 3: 30

Element 4: 40

Element 5: 50

Element 6: 60

Element 7: 70

Element 8: 80

Element 9: 90

Element 10: 100

2.2 Multi-Dimensional Array:A multi-dimensional array stores data in a table-like structure (rows and columns). The most common type is a two-dimensional array.

Example



```
public class MatrixAddition

{

    public static void main(String[] args)

    {

        int[][] matrix1 = { {1, 2}, {3, 4} };


        int[][] matrix2 = { {5, 6}, {7, 8} };


        // Create a matrix to store the result

        int[][] sum = new int[2][2];


        // Perform addition

        for (int i = 0; i < 2; i++) {

            for (int j = 0; j < 2; j++) {

                sum[i][j] = matrix1[i][j] + matrix2[i][j];

            }

        }


        // Display the result

        System.out.println("Sum of the two 2x2 matrices:");


        for (int i = 0; i < 2; i++) {
```



```
        for (int j = 0; j < 2; j++) {  
            System.out.print(sum[i][j] + " ");  
        }  
        System.out.println();  
    }  
}
```

Output

Sum of the two 2x2 matrices:

6 8

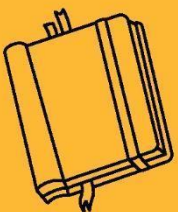
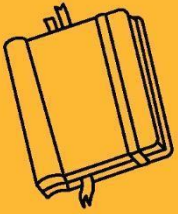
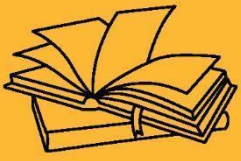
10 12

CONCLUSION:

- Arrays in Java help store multiple values of the same type in one variable.
- They make code cleaner, faster, and easier to manage when working with a list of similar items.
- You can use single-dimensional or multi-dimensional arrays depending on your needs.

KEY TAKEAWAYS

- Arrays are used to store multiple values of the same type.
- In Java, arrays are zero-indexed and fixed in size.
- Java supports single-dimensional and multi-dimensional arrays.
- Arrays make it easy to loop through and manage related data.



OVERVIEW TO JAVA PROGRAMMING





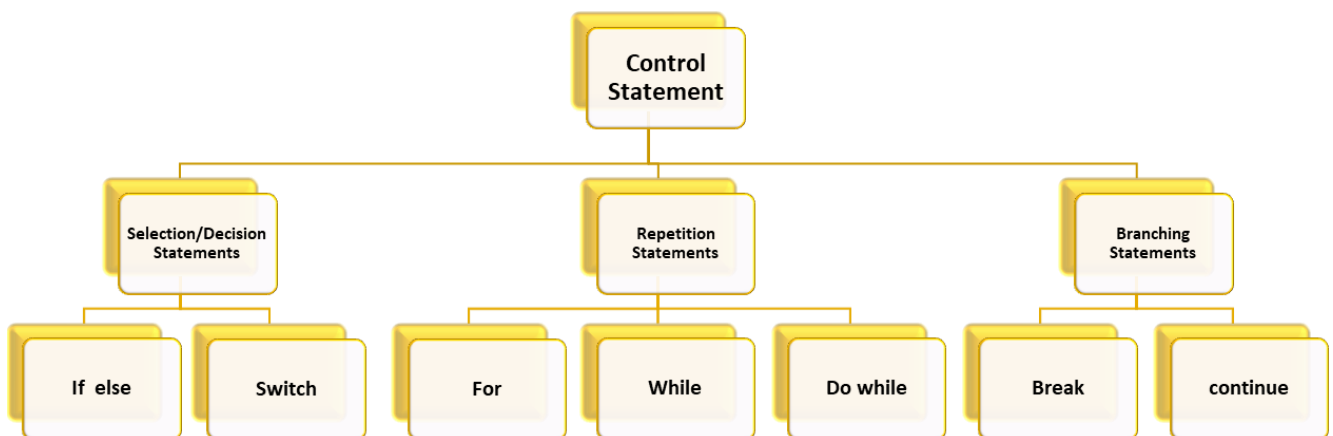
SUB LESSON 2.3

CONTROL STATEMENTS

Control statements help to control the flow of a program. They decide which part of the code should run and when. These are used to make decisions, repeat actions, or skip parts of the code. There are three main types: conditional, looping, and jump statements. Using control statements makes the program smart and flexible.

CONTROL STATEMENTS

- Control statements guide the program on what to do next based on conditions.
- They help in making decisions, repeating actions, or jumping to specific code parts.
- Common types include conditional (if, switch), looping (for, while, do while), and jump (break, continue) statements.
- These statements make programs more logical, organized, and efficient.



CONDITIONAL/SELECTION/DECISION STATEMENTS:



- Conditional statements allow the program to make decisions. They run different code based on whether a condition is true or false.
- Common types include if, if-else, if-else-if, and switch. Each helps control the program flow depending on different conditions.
- The program checks a condition to decide which code block to execute. This helps in handling multiple possibilities during execution.
- These statements enable the program to choose between different paths. They make the program flexible and responsive to various inputs or situations.
- Following are the conditional statements:
 - If else statements
 - Simple if statement
 - If else statement
 - Ladder if else statement
 - Nested if else statement
 - Switch statements

1.1 SIMPLE IF STATEMENT

- Simple If Statement: if statement execute a block of code only if the given condition is true.
- Syntax:

```
If(condition)
{
    Statement;
}
```

Example

```
public class SimpleIfExample
```



```
{  
  
    public static void main(String[] args) {  
  
        int number = 10;  
  
        if (number > 5) {  
  
            System.out.println("Number is greater than 5");  
  
        }  
  
    }  
  
}
```

Output

Number is greater than 5

1.2 IF ELSE STATEMENT

An if-else statement is used in programming to make decisions based on conditions. It evaluates a condition, and if the condition is true, it executes a specific block of code. If the condition is false, it runs a different block of code. This allows the program to choose between two possible actions, making the code more flexible and responsive to different situations.

Syntax

```
    if(condition)  
  
    {  
  
        Statement1;  
  
    }  
  
    else
```



```
{  
  
    Statement2;  
  
}
```

Example

```
class Check_Demo  
  
{  
  
    public static void main(String args[])  
  
    {  
  
        int a=5;  
  
        if(a%2==0)  
  
        {   System.out.println("No is Even");  
  
        }  
  
        else  
  
        {   System.out.println("No is ODD");  
  
        }  
  
    }  
  
}
```

Output

No is ODD

1.3 LADDER IF ELSE STATEMENT



Ladder if-else is a type of conditional control structure used in programming to check multiple conditions one after another. It is useful when you want to test a series of expressions and execute different blocks of code depending on which condition is true. The structure starts with a simple if condition, followed by one or more else if conditions, and ends with an optional else block. The program checks the first condition, and if it is true, the corresponding block is executed and the rest are skipped. If the first condition is false, it checks the next else if, and so on. If none of the conditions are true, the final else block is executed. This approach is helpful when you need to handle multiple possible outcomes or ranges, such as assigning grades based on marks, determining age groups, or selecting menu options.

Syntax

```
if(condition1)
{
    Statement1;
}

else if(condition2)
{
    Statement2;
}

else
{
    Statement3;
}
```



Example

```
public class NumberCheck

{

    public static void main(String[] args)

    {

        int number = 0; // You can change this value to test different inputs

        if (number > 0)

            System.out.println("The number is positive.");

        else if (number < 0)

            System.out.println("The number is negative.");

        else

            System.out.println("The number is zero.");

    }

}
```

Output

The number is zero

1.4 NESTED IF ELSE STATEMENT

Nested if-else is a control structure where an if or else statement is placed inside another if or else block. This allows a program to perform further checks only if a certain condition is met. It is



useful when decisions need to be made based on multiple related conditions. For example, if we want to check whether a number is positive, and if it is, further check whether it is even or odd, we can use a nested if-else. The outer if checks the first condition (e.g., whether the number is positive), and if it is true, the inner if-else checks the second condition (e.g., even or odd). This structure helps in organizing complex decision-making processes in a clear and logical way.

Syntax

```
if (condition1)
{
    if (condition2)
    {
        Statement1;
    }
    else
    {
        Statement2;
    }
}
else
{
    Statement3;
}
```

Example



```
public class MaxOfThree

{

    public static void main(String[] args)

    {

        int a = 25, b = 40, c = 15;


        if (a > b) {

            if (a > c)

                System.out.println("Maximum number is: " + a);

            else

                System.out.println("Maximum number is: " + c);

        }

        else {

            if (b > c)

                System.out.println("Maximum number is: " + b);

            else

                System.out.println("Maximum number is: " + c);

        }

    }

}
```



Output

Maximum number is: 40

1.5 SWITCH STATEMENT

The switch statement in Java is a control structure that allows you to test a variable against a list of values, called cases. It evaluates the variable once and executes the code block corresponding to the matching case. If a case matches, the code runs until a break statement is encountered, which prevents the program from executing the remaining cases. If none of the cases match, the default block executes. This structure makes the code cleaner and easier to read compared to multiple if-else conditions when checking a variable against several fixed values.

Syntax

```
switch (expression) {  
    case value1:  
        // Code block  
        break;  
    case value2:  
        // Code block  
        break;  
    default:  
        // Default code block  
}
```

Example

```
public class DayOfWeek
```




```
{  
  
    public static void main(String[] args)  
  
    {  
  
        int day = 3;  
  
  
        switch (day) {  
  
            case 1:  
  
                System.out.println("Monday");  
  
                break;  
  
            case 2:  
  
                System.out.println("Tuesday");  
  
                break;  
  
            case 3:  
  
                System.out.println("Wednesday");  
  
                break;  
  
            case 4:  
  
                System.out.println("Thursday");  
  
                break;  
  
            case 5:  
  
                System.out.println("Friday");  
  
        }  
    }  
}
```



```
        break;

    default:

        System.out.println("Weekend");

    }

}

}
```

Output

Wednesday

REPETITION/LOOP STATEMENT:

Loops are used in programming to repeat actions multiple times. They help save time by avoiding writing the same code again and again. Common types of loops are for, while, and do-while, which run as long as a condition is true. Using loops makes programs simpler and easier to understand.

FOR LOOP

A for loop is used to repeat a block of code a specific number of times. It has three parts: initialization, condition, and update, all in one line. The loop runs as long as the condition is true, executing the code inside its block each time. This makes for loops ideal for counting or iterating through items in a list.

Syntax

```
for (initialization; condition; update)

{

    // Code to be executed repeatedly

}
```



```
}
```

- Initialization: Runs once at the start to set up a counter variable.
- Condition: Checked before each loop iteration; if true, the loop continues.
- Update: Runs after each iteration to change the counter variable.

Example

```
public class Factorial

{

    public static void main(String[] args)

    {

        int number = 5; // Number to find factorial of

        int factorial = 1;

        for (int i = 1; i <= number; i++) {

            factorial = factorial * i;

        }

        System.out.println("Factorial of " + number + " is " + factorial);

    } }
```

Output

Factorial of 5 is 120

WHILE LOOP



A while loop is a control structure in programming that repeatedly executes a block of code as long as a specified condition is true. Before each iteration, the condition is checked, and if it evaluates to true, the code inside the loop runs. This process continues until the condition becomes false, at which point the loop stops. The while loop is useful when the number of iterations is not known in advance and depends on dynamic conditions, such as reading input until a certain value is entered. It provides a simple way to repeat actions based on changing conditions.

Syntax:

```
while (condition) {  
  
    // Code to be executed repeatedly  
  
}
```

- The condition is checked before each iteration.
- If the condition is true, the loop body runs.
- The loop continues until the condition becomes false.

Example

```
public class WhileExample  
{  
  
    public static void main(String[] args)  
    {  
  
        int i = 1;  
  
        while (i <= 5)  
        {
```



```
        System.out.println(i);

        i++; // Increment i by 1

    }

}
```

Output

```
1
2
3
4
5
```

DO WHILE LOOP

A do-while loop is similar to a while loop, but it guarantees that the code block inside the loop runs at least once. This is because the condition is checked after executing the loop body, not before. The loop continues to repeat as long as the condition remains true. This type of loop is useful when you want to ensure that the code runs once before any condition is tested, such as when displaying a menu or asking for user input at least once.

Syntax:

```
do {

    // Code to be executed

} while (condition);
```

- The code inside the do block runs at least once.



- After executing the block, the condition is checked.
- If the condition is true, the loop repeats; otherwise, it stops.

Example

```
public class DoWhileExample  
{  
    public static void main(String[] args)  
    {  
        int i = 1;  
  
        do {  
            System.out.println(i);  
            i++; // Increment i by 1  
        } while (i <= 5);  
    }  
}
```

Output

1
2
3
4
5



JUMP STATEMENT:

Jump statements in Java are used to transfer control to another part of the program. The main jump statements are break, continue, and return. break exits a loop or switch, continue skips the current loop iteration, and return exits from a method. These statements help manage the flow of execution in loops and methods.

BREAK STATEMENT

The break statement in Java is used to immediately exit a loop or switch statement. When break is executed, the control jumps out of the loop or switch, skipping any remaining code inside it. It is commonly used to stop a loop when a certain condition is met.

Syntax:

```
break;
```

Example:

```
public class BreakExample
{
    public static void main(String[] args)
    {
        for (int i = 1; i <= 10; i++) {
            if (i == 6) {
                break; // Exit the loop when i is 6
            }
        }
    }
}
```



```
        System.out.println(i);
    }
}
}
```

Output

```
1
2
3
4
5
```

CONTINUE STATEMENT

The continue statement in Java is used to skip the current iteration of a loop and jump to the next one. When continue is encountered, the remaining code inside the loop for that iteration is skipped. It is useful when you want to ignore certain values or conditions during looping without exiting the loop entirely.

Syntax:

```
continue;
```

Example

```
public class ContinueExample
{
```




```
public static void main(String[] args)
{
    for (int i = 1; i <= 5; i++) {
        if (i == 3) {
            continue; // Skip printing when i is 3
        }
        System.out.println(i);
    }
}
```

Output

1
2
4
5

CONCLUSION:

- Java control statements like if, switch, loops, and jump statements help manage the flow of execution in programs.
- for, while, and do-while loops allow repeating tasks efficiently, saving time and reducing code duplication.
- break and continue let you control how loops run, making your code more flexible and readable.



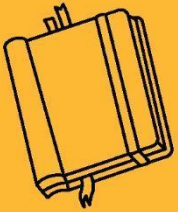
KEY TAKEAWAYS

- Control statements manage the decision-making and flow of a program.
- Loops are used to repeat code until a condition is false.
- Jump statements (break, continue, return) alter normal loop or method flow.

OUTCOMES OF CHAPTER 2

After completing this unit, students will be able to:

- Understand and effectively use different types of operators in Java such as arithmetic, logical, relational, bitwise, and conditional operators for building expressions and performing calculations.
- Declare, initialize, and manipulate single-dimensional and multi-dimensional arrays, and perform array-based computations.
- Apply control statements like if-else, switch, loops (for, while, do-while) for decision-making and repetitive tasks in Java programs.



OBJECT ORIENTED PROGRAMMING CONCEPTS





SUB LESSON 3.1

CLASS, OBJECT AND METHOD

OBJECTIVES OF CHAPTER 3

3.1 Defining Class, Object, Methods

- Understand how to define a class and create objects in Java.
- Learn how to declare and invoke methods to define object behavior.

3.2 Method Overloading

- Explore how multiple methods can have the same name but different parameters.
- Understand the advantages of method overloading for improving code readability and reusability.

3.3 Static Keyword

- Learn the purpose of the static keyword in variables, methods, blocks, and classes.
- Understand how static members belong to the class rather than to instances.

3.4 This Keyword

- Understand the use of the this keyword to refer to the current object.
- Learn how this resolves naming conflicts and is used in constructors and methods.

3.5 Constructor & Types of Constructor, Constructor Overloading, Copy Constructor

- Understand what constructors are and how they initialize objects.
- Learn different types of constructors and how constructor overloading works.
- Explore the concept and implementation of copy constructors in Java.



In object-oriented programming, the core concepts of classes, objects, and methods form the foundation for writing organized and reusable code. Understanding these building blocks is essential before diving deeper into Java programming.

CLASS, OBJECT AND METHOD

Java is an object-oriented programming language that uses classes and objects to model real-world entities. Learning about classes, objects, and methods helps in structuring programs efficiently and promoting code reuse.

CLASS

- A class is a blueprint or prototype from which objects are created.
- Classes are the basic blocks for object-oriented programming in Java.
- Class is a use defined data type; all data and method are encapsulated within the class.
- The data and function written within the class are called as a member of a class.

DEFINING A CLASS

A class defines the data types used and methods that manipulate them. The variables are called instance variables and methods are called instance methods. A class contains any number of methods. The general form of declaring class is

```
class class_name  
{  
  
    type instance_variable_1;  
  
    type instance_variable_2;  
  
    .  
  
    .  
}
```



```
    type instance_variable_n;

    return type method_name_1(parameter list);

    {

        // body of the method_1;

    }

    return type method_name_2(parameter list);

    {

        // body of the method_2;

    }.

    return type method_name_n(parameter list);

    {

        // body of the method_n;

    }

}
```

Class name is user defined class name. In general, class declarations can include these components, in order:

- Modifiers such as public, private, and a number of others that you will encounter later.
- The class name, with the initial letter capitalized by convention.
- The name of the class's parent (superclass), if any, preceded by the keyword `extends`. A class can only extend (subclass) one parent.
- A comma-separated list of interfaces implemented by the class, if any, preceded by the keyword `implements`. A class can implement more than one interface.



- The class body, surrounded by braces, {}.

Declaring Member Variables. There are several kinds of variables:

- Member variables in a class—these are called fields.
- Variables in a method or block of code—these are called local variables.
- Variables in method declarations—these are called parameters.

Example.

```
class rectangle  
  
{  
  
    int length;  
  
    int breath  
  
    void area();  
  
}
```

OBJECT

A Java object is one instance of class. An object is a block of memory that contains space to store all the instance variables. Creating an object is also referred as instantiating an object. Objects are created by using new operator. The new operator creates an object of the specified class and returns a reference to the object created.

Example

```
rectangle r1=new rectangle();
```

A variables that store object references are called object reference variable. The variable r1 is the object reference variable whose type is Rectangle. When you copy one object reference variable to another object reference variable, no object is created but another copy of the reference variable created, which points to same object.



```
rectangle r2=r1;
```

The dot(.) operator

The dot operator (.) is used to access the instance variables and methods defined in that object.

Example

```
class test
{
    int a, b;
}

class testmain
{
    public static void main(String arg[])
    {
        test t1=new test();

        t1.a=5;

        t1.b=10;

        System.out.println(" value of a is =" + t1.a);

        System.out.println(" value of b is =" + t1.b);

    }
}
```




METHOD

A method in Java is a block of code that performs a specific task. It is used to define the behavior of objects and promote code reusability by allowing a piece of logic to be executed multiple times.

- Improves modularity by dividing the program into smaller parts.
- Avoids repetition of code.
- Makes code easy to understand, test, and maintain.
- Can return values or perform actions.
- Can take input parameters to operate dynamically.

Syntax:

```
returnType methodName(parameters)

{

    // Method body

}
```

Example

```
class Calculator

{

    int add(int x, int y)

    {

        return x + y;

    }

}
```



```
public class MainClass

{

    public static void main(String[] args)

    {

        Calculator calc = new Calculator();

        int result = calc.add(10, 20);

        System.out.println("Addition: " + result);

    }

}
```

Output

Addition: 30

Example

```
import java.util.Scanner;

class Student

{

    int rollNumber;

    String name;

    int attendance;

    // Method to input student data

    void getData()
```



```
{

    Scanner sc = new Scanner(System.in);

    System.out.print("Enter Roll Number: ");

    rollNumber = sc.nextInt();

    sc.nextLine(); // consume leftover newline

    System.out.print("Enter Name: ");

    name = sc.nextLine();

    System.out.print("Enter Attendance (%): ");

    attendance = sc.nextInt();

}

// Method to display student data

void putData()

{

    System.out.println("\n--- Student Details ---");

    System.out.println("Roll Number: " + rollNumber);

    System.out.println("Name: " + name);

    System.out.println("Attendance: " + attendance + "%");

}

}

public class MainClass
```



```
{  
  
    public static void main(String[] args)  
  
    {  
  
        Student stu = new Student(); // Create object  
  
        stu.getData();           // Call method to enter data  
  
        stu.putData();           // Call method to display data  
  
    }  
  
}
```

Output:

Enter Roll Number: 101

Enter Name: Hetal

Enter Attendance (%): 90

--- Student Details ---

Roll Number: 101

Name: Hetal

Attendance: 90%

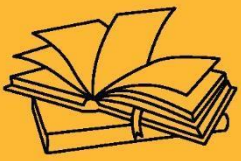
CONCLUSION:

- Classes, objects, and methods are the building blocks of Java's object-oriented programming.
- These concepts help in creating organized, reusable, and real-world oriented programs by combining data and behavior in a single structure.

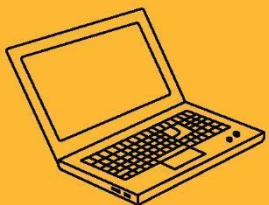


KEY TAKEAWAYS

- A class is a blueprint for creating objects that hold data and behavior.
- An object is an instance of a class used to access its members.
- A method defines an action or behavior that an object can perform.



OBJECT ORIENTED PROGRAMMING CONCEPTS





SUB LESSON 3.2

METHOD OVERLOADING

Method overloading is a basic concept in Java where you can use the same method name with different parameter lists in the same class. It helps perform similar tasks in a flexible way.

METHOD OVERLOADING

- Method overloading allows creating multiple methods with the same name but different input parameters in a class.
- It helps perform similar operations using a single method name, improving code clarity and organization.
- Overloading is an example of compile-time (static) polymorphism in object-oriented programming.
- Overloading is based on differences in parameter number, type, or order—not return types alone.
- Method overloading occurs within the same class, not between parent and child classes.

BENEFITS OF USING METHOD OVERLOADING:

- **Readability:**
Method overloading makes programs easier to read by using the same method name for different tasks with different inputs.
- **Flexible:**
You can do similar work using one method name, which means you don't need to create many different names.
- **Less Repetition:**
It helps reduce repeated code by combining similar actions under one method name.
- **Ease to Use:**



Programmers don't have to remember many names — they can just use the same name with different values.

- Better and Faster:

It makes the code faster and more organized by avoiding too many separate methods.

Example

```
class Calculator

{

    void add(int a, int b)

    {

        System.out.println("Sum of integers: " + (a + b));    }

    void add(double a, double b)

    {

        System.out.println("Sum of doubles: " + (a + b));    }

}

class Demo_Methodoverloading

{

    public static void main(String[] args)

    {

        Calculator calc = new Calculator();

        calc.add(10, 20);    // Calls method with int parameters

        calc.add(5.5, 4.5);    // Calls method with double parameters

    }

}
```




```
}
```

```
}
```

Output

Sum of integers: 30

Sum of doubles: 10.0

RULES OF METHOD OVERLOADING:

- Method overloading in Java allows you to define multiple methods with the same name but different parameters within the same class.
- There are several ways to achieve method overloading:
 - Changing the number of Parameter,
 - Changing data types of the Arguments,
 - Changing the order of the parameters of Methods

CHANGING THE NUMBER OF PARAMETER

Method performs different tasks depending on how many arguments are passed. This allows the same method name to handle varying input counts.

Example

```
class Display
```

```
{
```

```
    void show(String message)
```

```
    {
```

```
        System.out.println("Message: " + message);
```

```
    }
```



```
void show(String message, int count)

{

    System.out.println("Message: " + message + ", Count: " + count);

}

}

class Demo

{

    public static void main(String[] args)

    {

        Display obj = new Display();

        obj.show("Hello");    // Calls method with int parameters

        obj.show("Hi", 2);    // Calls method with double parameters

    }

}
```

Output

Message: Hello

Message: Hi , Count: 2

CHANGING THE DATATYPE OF ARGUMENT

Same method name can accept arguments of different data types. It helps handle operations like addition or display for various input formats.

```
class Calculator
```



```
{  
  
    void add(int a, int b)  
  
    {  
  
        System.out.println("Sum of integers: " + (a + b));    }  
  
    void add(double a, double b)  
  
    {  
  
        System.out.println("Sum of doubles: " + (a + b));    }  
  
}  
  
class Demo_Methodoverloading  
  
{  
  
    public static void main(String[] args)  
  
    {  
  
        Calculator calc = new Calculator();  
  
        calc.add(30, 20);    // Calls method with int parameters  
  
        calc.add(7.5, 7.5);    // Calls method with double parameters  
  
    }  
  
}
```

Output

Sum of integers: 50

Sum of doubles: 15.0



CHANGING THE ORDER OF THE PARAMETER IN METHOD

Method is defined with the same name and parameter types but in a different sequence. This allows correct method selection based on the input order.

Example

```
class Person

{

    void info(String name, int age)

    {

        System.out.println("Name: " + name + ", Age: " + age);

    }

    void info(int age, String name)

    {

        System.out.println("Age: " + age + ", Name: " + name);

    }

}

public class Demo

{

    public static void main(String[] args)

    {

        Person p = new Person();
```



```
p.info("Hetal", 25);    // Calls method with name first

p.info(30, "Priya");    // Calls method with age first

}

}
```

Output

Name: Hetal, Age: 25

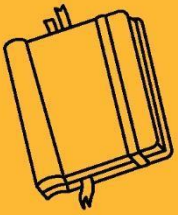
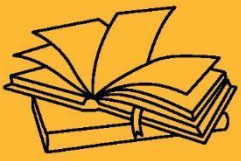
Age: 30, Name: Priya

CONCLUSION

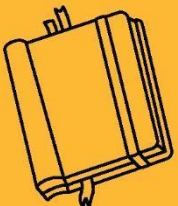
- Method overloading improves code clarity and flexibility by allowing multiple methods with the same name but different parameters.
- It supports compile-time polymorphism, making programs easier to read and maintain.

KEY TAKEAWAYS

- Methods with the same name must have different parameter lists (type, number, or order).
- Overloading increases code readability by using a single method name for related tasks.
- Return type alone cannot be used to overload a method.



OBJECT ORIENTED PROGRAMMING CONCEPTS





SUB LESSON 3.3

STATIC KEYWORD

The static keyword in Java is used to create class-level members that belong to the class rather than any specific instance (object).

STATIC KEYWORD

The static keyword in Java is used to define class-level members that can be accessed without creating an object. It applies to variables, methods, blocks, and nested classes. Static members are shared among all objects of the class, meaning only one copy exists. This feature helps save memory and is commonly used for constants and utility method.

The static can be used with:

1. variable (also known as class variable)
2. method (also known as class method)
3. block
4. nested class

STATIC VARIABLE:

- A static variable is shared among all instances of a class; only one copy exists in memory.
- It is declared using the static keyword inside the class but outside any method or constructor.
- Static variables are initialized only once when the class is loaded into memory.
- They can be accessed directly using the class name without creating an object.
- Static variables are useful for common properties like a counter or configuration values.

Syntax

```
static dataType variableName;
```



Example

```
class Demo

{

    static int a = 0; // Static variable

    int b = 0;      // Non-static variable

    Demo()

    {

        a++; // Increments shared static variable

        b++; // Increments non-static variable for each object

        System.out.println("a (static): " + a + ", b (non-static): " + b);

    }

}

public class MainClass

{

    public static void main(String[] args)

    {

        Demo obj1 = new Demo(); // Output: a = 1, b = 1

        Demo obj2 = new Demo(); // Output: a = 2, b = 1

        Demo obj3 = new Demo(); // Output: a = 3, b = 1

    }

}
```




```
}
```

OUTPUT

a (static): 1, b (non-static): 1

a (static): 2, b (non-static): 1

a (static): 3, b (non-static): 1

STATIC METHOD

A static method in Java belongs to the class rather than any specific object. It can be called without creating an object of the class. Static methods can access only static variables and call other static methods. They are often used for utility or helper methods that don't require any object state.

Syntax

```
static returnType methodName(parameters)

{

    // method body

}
```

Example

```
class Demo

{

    static void displayMessage()

    {

        System.out.println("This is a static method.");

    }

}
```



```
    }  
}  
  
public class MainClass  
{  
  
    public static void main(String[] args)  
  
    {  
  
        Demo.displayMessage();  
  
    }  
}
```

Output

This is a static method.

STATIC BLOCK

A static block in Java is a block of code that runs only once when the class is loaded into memory. It is mainly used to initialize static variables or perform setup operations before the main method or any object creation. Static blocks execute automatically and before any constructor or method. This can only be used inside instance methods or constructors.

Syntax

```
static  
  
{  
  
    // Initialization code  
  
}
```



Example

```
class Demo
```

```
{
```

```
    static int value;
```

```
    static {
```

```
        value = 100;
```

```
        System.out.println("Static block executed.");
```

```
    }
```

```
    static void showValue() {
```

```
        System.out.println("Value = " + value);
```

```
    }
```

```
}
```

```
public class MainClass
```

```
{
```

```
    public static void main(String[] args)
```

```
    {
```

```
        Demo.showValue(); // Static block runs before this call
```

```
    }
```

```
}
```

Output:



Static block executed.

Value = 100

STATIC CLASS

In Java, a static class refers to a static nested class, which is a class defined inside another class with the static keyword. Unlike inner (non-static) classes, a static nested class can be accessed without creating an object of the outer class. It cannot access non-static members of the outer class directly and is mostly used to logically group classes that are only used by their outer class.

Syntax:

```
static class ClassName

{

    // static nested class content

}
```

Example

```
class Outer

{

    static class Nested

    {

        void display()

        {

            System.out.println("Static nested class method called.");

        }

    }

}
```



```
}  
}
```

```
public class MainClass
```

```
{
```

```
    public static void main(String[] args) {
```

```
        Outer.Nested obj = new Outer.Nested(); // No need for Outer object
```

```
        obj.display();
```

```
    }
```

```
}
```

Output

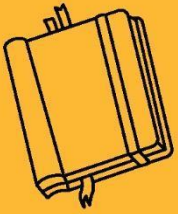
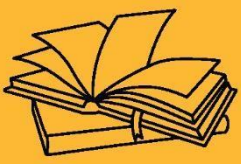
Static nested class method called.

CONCLUSION:

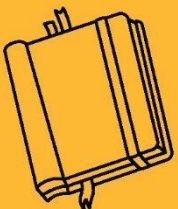
- The static keyword allows members to belong to the class itself rather than instances, helping in efficient memory usage.
- It simplifies code by allowing access to methods and variables without needing to create objects.

KEY TAKEAWAYS

- Static members are shared across all objects of the class.
- Static blocks are used for initializing static data before the program starts.
- Static nested classes can be used without creating an object of the outer class.



OBJECT ORIENTED PROGRAMMING CONCEPTS





SUB LESSON 3.4

THIS KEYWORD

In Java, this keyword is a reference variable that refers to the current object — the instance of the class where the code is executing. It is used inside an instance method or constructor to refer to the current object's fields, methods, or constructors.

WHY USE THE “THIS” KEYWORD?

- It helps distinguish between instance variables and parameters or local variables with the same name.
- It allows invoking other constructors in the same class (constructor chaining).
- It can be used to pass the current object as an argument to another method or constructor.
- It improves code clarity by explicitly showing you are referring to the current object.

WHY USE THE “THIS” KEYWORD?

ACCESSING INSTANCE VARIABLE:

- When method parameters or constructor parameters have the same name as instance variables, this is used to refer to the instance variables.

Example

```
class Display
```

```
{
```

```
    int b =5;
```



```
void display(int a)

{

    System.out.println("a = "+a);

    System.out.println("b = "+b);

}

}

class Demo_this

{

    Public static void main(String args[])

    {

        Display obj = new Display();

        Obj.display(7);

    }

}
```

Output

a=7

b=5

Here, in above example a is the local variable and b is the class variable.

Example

```
class Display
```




```
{  
  
    int a = 5;  
  
    void display(int a)  
  
    {  
  
        System.out.println("a = " + a);  
  
        System.out.println("a = " + a);  
  
    }  
  
}  
  
class Demo_this  
  
{  
  
    Public static void main(String args[])  
  
    {  
  
        Display obj = new Display();  
  
        Obj.display(7);  
  
    }  
  
}
```

Output

a=7

a=7



Here, In above program both local and instance variable name are same. So, priority is given to local variable. When instance variable and local variable both name are same then , compiler give first priority to functional variable.

When local variable and instance variable names are same then using this keyword, we can access instance variable.

Example

```
class Display

{

    int a =5;

    void display(int a)

    {

        System.out.println("Functional Variable a = :"+a);

        System.out.println("Instance variable a = :"+this.a);

    }

}

class Demo_this

{

    Public static void main(String args[])

    {

        Display obj = new Display();

        Obj.display(7);

    }

}
```



```
    }
```

```
}
```

Output

a=7

b=5

CALLING ONE CONSTRUCTOR FROM ANOTHER

Using this() inside a constructor allows calling another constructor of the same class.

Example

```
class Student
```

```
{
```

```
    int id;
```

```
    String name;
```

```
    // Default constructor using this() to call parameterized constructor
```

```
    Student()
```

```
    {
```

```
        this(0, "Unknown");
```

```
    }
```

```
    // Parameterized constructor
```

```
    Student(int id, String name)
```

```
    {
```



```
this.id = id;

this.name = name;

}

// Method to display student details

void display()

{

    System.out.println("ID: " + id + ", Name: " + name);

}

}

public class Main {

    public static void main(String[] args)

    {

        // Creating objects

        Student s1 = new Student();           // Calls default constructor

        Student s2 = new Student(100, "Hetal"); // Calls parameterized constructor

        // Displaying object data

        s1.display();

        s2.display();

    }

}
```



Output

ID: 0, Name: Unknown

ID: 100, Name: Hetal

IMPORTANT OF THIS KEYWORD

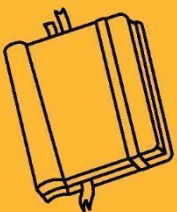
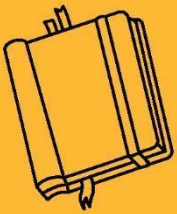
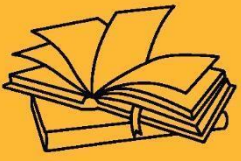
- this can only be used inside instance methods or constructors.
- It cannot be used inside static methods because static methods belong to the class, not instances.
- Using this helps avoid confusion between local variables and instance variables.
- It improves readability and maintainability of code.

CONCLUSION:

- The this keyword in Java is used to refer to the current object within a class. It helps distinguish between local and instance variables when they share the same name.
- It enables constructor chaining and allows passing the current object to methods or constructors. This improves code clarity and promotes reusability.

KEY TAKEAWAYS

- This this keyword is a reference to the current object.
- It helps differentiate between instance variables and parameters with the same name.
- It is used for constructor chaining and passing the current object as an argument.
- This cannot be used in static contexts.



OBJECT ORIENTED PROGRAMMING CONCEPTS



SUB LESSON 3.5

CONSTRUCTOR & TYPES OF CONSTRUCTOR, CONSTRUCTOR OVERLOADING, COPY CONSTRUCTOR, CONSTRUCTOR OVERLOADING

A constructor in Java is a special method used to initialize objects when they are created. It automatically sets initial values and has the same name as the class, with no return type.

CONSTRUCTOR

- Constructor is a member function which name is same as class name.
- Constructor has no return type.
- Constructor has no return statement.
- A constructor is automatically called when an object of a class is created using the new keyword

TYPES OF CONSTRUCTOR

In Java, constructors are used to initialize objects when they are created. Based on how they are defined and used, constructors can be classified into different types.

DEFAULT CONSTRUCTOR:

A default constructor is a constructor that takes no arguments. If no constructor is explicitly defined, Java provides a default one automatically. It also known as without parameter constructor.

Example

class Demo

{

// Default constructor



```
Demo()

{

    System.out.println("This is a default constructor.");

}

}

public class MainClass

{

    public static void main(String[] args)

    {

        Demo obj = new Demo(); // Calls default constructor

    }

}
```

Output

This is a default constructor.

PARAMETERIZED CONSTRUCTOR:

A parameterized constructor is a constructor that takes arguments to initialize an object with custom values at the time of its creation.

Example

```
class Addition
```

```
{

    int num1, num2;
```




```
// Parameterized constructor

Addition(int a, int b)

{

    num1 = a;

    num2 = b;

}

void sum()

{

    int result = num1 + num2;

    System.out.println("Sum = " + result);

}

}

// Main class

public class MainClass

{

    public static void main(String[] args)

    {

        Addition obj = new Addition(10, 20); // Passing two numbers

        obj.sum(); // Display the sum

    }

}
```



```
}  
  
}
```

Output

Sum=30

COPY CONSTRUCTOR

A copy constructor is used to create a new object by copying the values from another object of the same class. Java does not provide a built-in copy constructor, but you can define it manually.

Example

```
class Student {  
  
    String name;  
  
    int age;  
  
    // Parameterized constructor  
  
    Student(String n, int a) {  
  
        name = n;  
  
        age = a;  
  
    }  
  
    // Copy constructor  
  
    Student(Student s) {  
  
        this.name = s.name;  
  
        this.age = s.age;  
  
    }  
}
```



```
void display() {  
  
    System.out.println("Name: " + name + ", Age: " + age);  
  
}  
  
}  
  
public class MainClass  
  
{  
  
    public static void main(String[] args)  
  
{  
  
        Student s1 = new Student("Hetal", 30); // Original object  
  
        Student s2 = new Student(s1);        // Copy object using copy constructor  
  
        s1.display();  
  
        s2.display();  
  
    }  
  
}
```

Output

Name: Hetal, Age: 30

Name: Hetal, Age: 30



CONSTRUCTOR OVERLOADING:

Constructor overloading means having more than one constructor in a class, but each constructor has different types or numbers of parameters. This allows creating objects in different ways, depending on the information available. It helps to initialize objects flexibly by choosing the right constructor based on the arguments passed.

Example

```
class Addition {  
  
    int sum;  
  
    // Constructor for adding 2 numbers  
  
    Addition(int a, int b) {  
  
        sum = a + b;  
  
    }  
  
    // Constructor for adding 3 numbers  
  
    Addition(int a, int b, int c) {  
  
        sum = a + b + c;  
  
    }  
  
    void display() {  
  
        System.out.println("Sum = " + sum);  
  
    }  
  
}  
  
public class MainClass {
```



```
public static void main(String[] args) {  
  
    Addition addTwo = new Addition(10, 20);    // Adds two numbers  
  
    Addition addThree = new Addition(5, 15, 25); // Adds three numbers  
  
    addTwo.display();  
  
    addThree.display();  
  
}  
}
```

Output

Sum = 30

Sum = 45

CONCLUSION:

- Constructor overloading allows creating multiple constructors with different parameters in the same class, increasing flexibility.
- It helps initialize objects in various ways depending on the data available at the time of object creation.
- Using constructor overloading improves code readability and reusability by avoiding multiple method names for similar tasks.

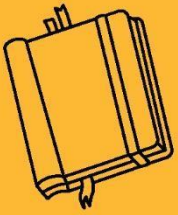
KEY TAKEAWAYS

- Constructors have the same name as the class and no return type.
- Overloaded constructors differ in the number or type of parameters.
- Constructor overloading enables different ways to initialize objects efficiently.

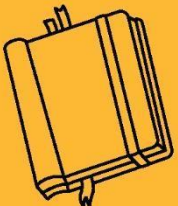


OUTCOME OF CHAPTER 3

- Understand how to define classes and create objects, and invoke methods in Java for structured and modular programming.
- Implement method overloading to allow multiple methods with the same name but different parameters.
- Use the static keyword effectively for variables, methods, and blocks that belong to the class rather than to instances.
- Apply the this keyword to refer to the current object and resolve variable shadowing and method calls.
- Understand the concept of constructors, including default, parameterized, copy constructors, and apply constructor overloading in Java programs.



INHERITANCE & INTERFACE





SUB LESSON 4.1

INHERITANCE & TYPES OF INHERITANCE

OBJECTIVES OF CHAPTER 4

4.1 Inheritance & Types of Inheritance

- Understand the concept of inheritance and how it allows one class to acquire properties and behaviors of another.
- Learn different types of inheritance supported in Java such as single, multilevel, and hierarchical.

4.2 Method Overriding

- Explore how a subclass can provide a specific implementation of a method already defined in its superclass.
- Understand rules and benefits of overriding for runtime polymorphism.

4.3 Dynamic Method Dispatch

- Learn how Java decides at runtime which method to execute when a superclass reference refers to a subclass object.
- Understand the role of dynamic method dispatch in achieving polymorphism.

4.4 Super Keyword

- Understand the use of the super keyword to refer to parent class members.
- Learn how to call superclass constructors and methods using super.

4.5 Final Keyword

- Explore how the final keyword restricts modification of classes, methods, and variables.
- Understand its use in securing program design and preventing inheritance or overriding.



4.6 Abstract Class

- Understand what an abstract class is and how it can define common behavior while leaving some methods abstract.
- Learn how abstraction helps in defining generic templates for derived classes.

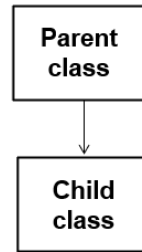
Before learning inheritance, it's important to understand the concept of classes and objects, as they form the foundation of object-oriented programming. Inheritance builds upon these concepts by allowing one class to reuse and extend the features of another.

INHERITANCE

Inheritance is a foundational concept of Object-Oriented Programming (OOP), and it plays a critical role in promoting code reusability and hierarchical class structure. In Java, inheritance allows a class (called the subclass or child class) to inherit properties and behaviors (fields and methods) from another class (called the superclass or parent class). This means that the subclass automatically gains access to all the non-private members of the parent class, which significantly reduces code duplication.

Java supports single, multilevel, and hierarchical inheritance, and multiple inheritance through the use of interfaces. Inheritance also makes it possible to implement polymorphism, especially through method overriding and dynamic method dispatch, which allow for flexible and dynamic program execution.

Inheritance of the mechanism in which one class object gets all the properties of another class. To establish this relation Java uses 'extends' keyword.



Syntax:

```
class Parent

{

    //code

}

class Child extends Parent

{

    //code

}
```

ADVANTAGE OF INHERITANCE

- Helps to reuse code from one class in another.
- Makes it easier to manage and update common features.
- Allows a child class to change or improve the method from the parent class.
- Saves time by avoiding repeated code in multiple classes.
- Makes the program more organized and clear.
- Helps in adding new features without changing old code.

Types of Inheritance

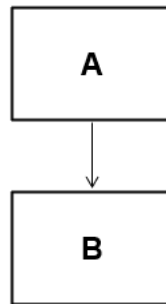


In Java, inheritance allows a class to acquire the properties and behaviors of another class. Based on how classes are related, there are several types of inheritance:

1. Single Inheritance
2. Multilevel Inheritance
3. Hierarchical Inheritance

SINGLE INHERITANCE

Single inheritance refers to a scenario where a child class (subclass) inherits from only one parent class (superclass). This is the most basic and commonly used form of inheritance in Java. Only two classes are involved: one base class and one derived class. The subclass extends the superclass using the extends keyword.



Syntax:

```
class A
{
    //parent class
}

class B extends A
{
```



```
//child class
```

```
}
```

Example

```
import java.io.*;
```

```
class A
```

```
{
```

```
    void disp_A()
```

```
{
```

```
    System.out.println("Class A");
```

```
}
```

```
}
```

```
class B extends A
```

```
{
```

```
    void disp_B()
```

```
{
```

```
    System.out.println("Class B");
```

```
}
```

```
}
```

```
class Demo
```

```
{
```



```
public static void main(String args[])
```

```
{
```

```
    B obj = new B();
```

```
    Obj.disp_A();
```

```
    Obj.disp_B();
```

```
}
```

```
}
```

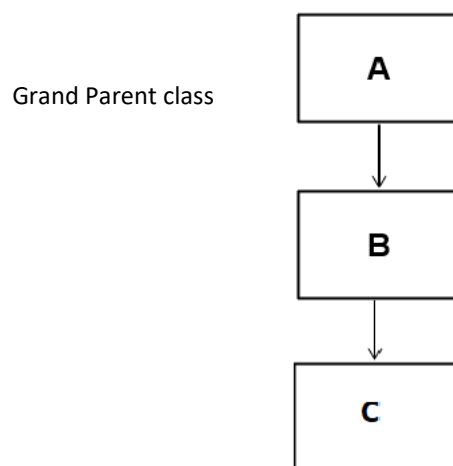
Output

Class A

Class B

MULTILEVEL INHERITANCE

Multilevel inheritance occurs when a class is derived from a subclass, which is already derived from another superclass, forming a chain of inheritance.





At least three classes are involved — a base class, a derived class, and a sub-derived class (child of a child). Each level of class inherits fields and methods from its parent, so the bottom-most class inherits from all its ancestors, promoting high reusability.

Syntax:

```
class A
{
    //parent class
}

class B extends A
{
    //intermediate class
}

class C extends B
{
    //child class
}
```

Example

```
import java.io.*;

class A
{
    void disp_A()
```



```
        {  
            System.out.println("Class A");  
        }  
    }  
  
class B extends A  
{  
    void disp_B()  
    {  
        System.out.println("Class B");  
    }  
}  
  
class C extends B  
{  
    void disp_C()  
    {  
        System.out.println("Class C");  
    }  
}  
  
class Demo  
{
```



```
public static void main(String args[])

{

    C Obj = new C();

    Obj.disp_A();

    Obj.disp_B();

    Obj.disp_C();

}

}
```

Output

Class A

Class B

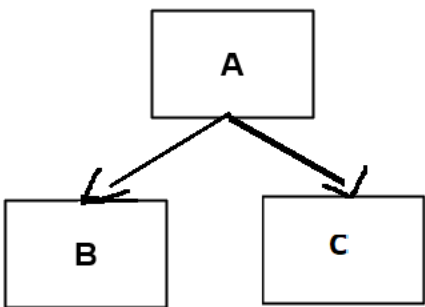
Class C

HIERARCHICAL INHERITANCE

Hierarchical inheritance occurs when multiple subclasses inherit from a single superclass. It models

situations where different types share common behavior but also have their own unique features.

There is one parent class and two or more child classes. Each child class gets access to the members of the same parent class. All child classes share the methods and variables of the parent class, enabling consistent behavior and avoiding code repetition.



Syntax:

```
class A
{
    //parent class
}

class B extends A
{
    //intermediate class
}

class C extends A
{
    //child class
}
```

Example

```
import java.io.*;
```



```
class A
```

```
{
```

```
    void disp_A()
```

```
    {
```

```
        System.out.println("Class A");
```

```
    }
```

```
}
```

```
Class B extends class A
```

```
{
```

```
    void disp_B()
```

```
    {
```

```
        System.out.println("Class B");
```

```
    }
```

```
}
```

```
Class C extends class A
```

```
{
```

```
    void disp_C()
```

```
    {
```

```
        System.out.println("Class C");
```

```
    }
```



```
}  
  
class Demo  
  
{  
  
    public static void main(String args[])  
  
    {  
  
        B Obj1 = new B();  
  
        Obj1.disp_A();  
  
        Obj1.disp_B();  
  
        C Obj = new C();  
  
        Obj.disp_A();  
  
        Obj.disp_C();  
  
    }  
  
}
```

Output

Class A

Class B

Class A

Class C

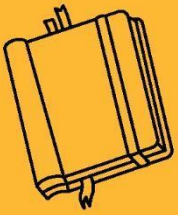
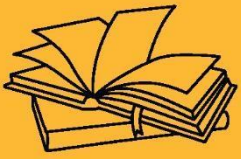
CONCLUSION:

- Inheritance is a powerful feature in Java that allows one class to inherit the properties and behaviors of another, promoting code reusability.

- 
- It simplifies program structure, makes code easy to manage, and supports key OOP concepts like polymorphism and method overriding.

KEY TAKEAWAYS

- Inheritance allows code sharing between classes.
- Java supports single, multilevel, and hierarchical inheritance.
- Subclasses can override methods of the parent class.
- Inheritance improves code readability and reduces duplication.
- `extends` keyword is used to implement inheritance in Java.



INHERITANCE & INTERFACE



SUB LESSON 4.2

METHOD OVERRIDING

Method overriding is an important feature of object-oriented programming in Java.

METHOD OVERRIDING

It allows a subclass to provide a specific implementation of a method that is already defined in its superclass. This is useful when the same behavior cannot be shared by all subclasses, and each needs to define its own version of the method.

Overriding ensures that the correct version of a method is called at runtime, even when using a reference of the parent class. This is a key concept behind runtime polymorphism, making Java programs more flexible and dynamic.

- Method overriding occurs when a subclass defines a method that already exists in its superclass with the same name, return type, and parameters.
- It allows a subclass to provide its own version of a method for specific behavior.
- Overriding supports runtime polymorphism by determining the method to call at runtime.
- Only inherited methods can be overridden (private, static, and final methods cannot be overridden).
- It improves flexibility and reusability in object-oriented design.
- Java decides which method to call based on the object, not the reference type.
- Use the `super` keyword to call the parent class method inside the overridden method if needed.

Example

```
class A
```

```
{
```



```
public void disp()

{
    System.out.println("Class A");
}

class B extends A

{

    public void disp() {

        System.out.println("class B");
    }

    public class Demo

    {

        public static void main(String args[])

        {
            B b = new B();

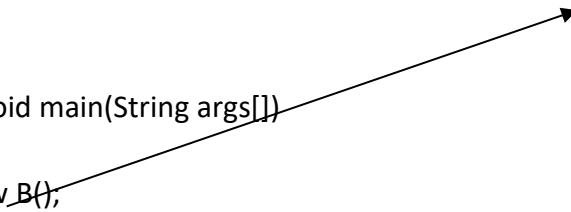
            b.disp();

        }

    }

}
```

Only access child class property



Output

Class B

Example

class A



```
{  
  
public void disp()  
  
{    System.out.println("Class A"); }  
  
}
```

class B extends A

```
{  
  
public void disp()  
  
{  
    Super.disp();  
    System.out.println("class B"); }  
  
}
```

Parent class method access through
super keyword

```
public class Demo  
  
{  
  
    public static void main(String args[])  
  
    {    B b = new B();  
        b.disp();  
  
    }  
  
}
```

Output

Class A



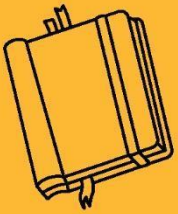
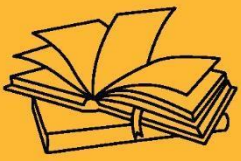
Class B

CONCLUSION:

- Method overriding allows a subclass to change the behavior of a method inherited from its superclass.
- It is a key concept in runtime polymorphism, helping Java programs become more flexible and dynamic.

KEY TAKEAWAYS

- Overriding occurs when a method in the child class has the same signature as in the parent class.
- It improves code customization and object-oriented design.
- Use `@Override` for better readability and error checking



INHERITANCE & INTERFACE



SUB LESSON 4.3

DYNAMIC METHOD DISPATCH

Dynamic Method Dispatch is a process in Java where a call to an overridden method is resolved at runtime rather than compile time. It is a fundamental feature that enables runtime polymorphism.

DYNAMIC METHOD DISPATCH

Dynamic Method Dispatch allows Java to determine which version of an overridden method to call based on the object type that is referred to at runtime. When a parent class reference refers to a child class object, Java uses dynamic method dispatch to call the overridden method in the child class. This feature enables more flexible and extensible code, where decisions are deferred until runtime. It is commonly used when methods are overridden and objects are accessed through parent class references.

- Used to achieve runtime polymorphism in Java.
- Involves method overriding and superclass reference pointing to a subclass object.
- Only overridden methods are resolved dynamically, not variables.
- The correct method is selected based on the actual object the reference points to.
- Enables flexible and reusable code by allowing different object behaviors under a common interface.

Example

```
class A  
  
{  
  
    void display()  
  
    {
```



```
        System.out.println("Display from Class A");
    }
}

class B extends A
{
    void display()
    {
        System.out.println("Display from Class B");
    }
}

class C extends A
{
    void display()
    {
        System.out.println("Display from Class C");
    }
}

public class Main
{
    public static void main(String[] args)
```



```
{  
  
    A ref;    // Reference of superclass A  
  
    ref = new B(); // ref refers to object of class B  
  
    ref.display(); // Calls B's display method  
  
  
    ref = new C(); // ref refers to object of class C  
  
    ref.display(); // Calls C's display method  
  
}
```

Output

Display from Class B

Display from Class C

In this example, class A is the superclass, and classes B and C are its subclasses that override the display() method. A reference variable ref of type A is used to refer to objects of both B and C. When ref refers to an object of B, the overridden display() method in class B is called. Similarly, when it refers to an object of C, the method in class C is executed. This behavior is called dynamic method dispatch, where the method call is determined at runtime based on the actual object, allowing Java to support runtime polymorphism and enabling flexible and extensible code.

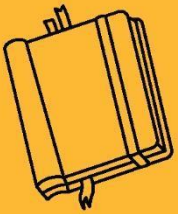
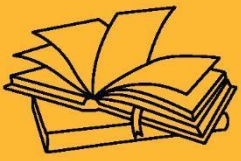
CONCLUSION:

- Dynamic method dispatch allows Java to decide which overridden method to call at runtime, based on the actual object being referred to.
- It supports runtime polymorphism, enabling developers to write more flexible, reusable, and scalable code.



KEY TAKEAWAYS

- Works only with overridden methods, not with data members.
- Requires a superclass reference to point to a subclass object.
- Ensures correct method is executed based on the object's actual type, not the reference type.



INHERITANCE & INTERFACE



SUB LESSON 4.4

SUPER KEYWORD

In Java, inheritance is a core concept that allows one class (subclass) to inherit fields and methods from another class (superclass). The super keyword plays an important role in inheritance, acting as a reference to the parent class object.

SUPER KEYWORD

The super keyword in Java is a reference variable used inside a subclass to refer to its immediate parent class object. It plays a key role in inheritance by allowing access to parent class members (variables, methods, and constructors) that may be overridden or hidden in the child class. It helps in resolving naming conflicts, reusing code, and enhancing clarity in inheritance-based programs.

When a subclass contains a method or variable with the same name as its parent class, ambiguity arises. The super keyword resolves this by clearly specifying that the member from the parent class is to be used. Additionally, when a subclass constructor needs to invoke a constructor from its superclass, super() is used for this purpose and must appear as the first statement in the constructor.

The super keyword enhances code reusability, clarity, and modularity by allowing child classes to efficiently interact with inherited functionality without redefining it.

ACCESSING PARENT CLASS VARIABLE:

Used when the child class has a variable with the same name as the parent class.

Example

```
class A
```

```
{
```




```
int num = 100;

}

class B extends A

{

    int num = 200;

    void show()

    {

        System.out.println("Value of num in B: " + num);    // 200

        System.out.println("Value of num in A: " + super.num); // 100

    }

}

public class Main

{

    public static void main(String[] args)

    {

        B obj = new B();

        obj.show();

    }

}
```



Output:

Value of num in B: 200

Value of num in A: 100

In the above example, class A contains a variable num. Class B extends A and also defines its own num variable. Inside the show() method of class B, the super.num accesses the num variable from class A, while num refers to class B's own variable. This clearly shows how the super keyword helps to avoid ambiguity and access hidden members of the parent class.

CALLING PARENT CLASS METHODS

Allows calling the overridden method of the parent class from the subclass.

Example

class A

```
{  
  
    void display()  
  
    {  
  
        System.out.println("This is the parent class method.");  
  
    }  
  
}
```

class B extends A

```
{  
  
    void display()  
  
    {
```



```
        System.out.println("This is the child class method.");

    }

    void show()

    {

        super.display(); // Call parent class method

        display();      // Call child class method

    }

}

public class Main

{

    public static void main(String[] args)

    {

        B obj = new B();

        obj.show();

    }

}
```

Output:

This is the parent class method.

This is the child class method.

CALL PARENT CLASS CONSTRUCTO:

`super()` is used to invoke the parent class constructor, and it must be the first statement in the subclass constructor.

Example

class A

```
{  
  
    A()  
  
    {  
  
        System.out.println("Parent class constructor called.");  
  
    }  
  
}
```

Class B extends A

```
{  
  
    B()  
  
    {  
  
        super(); // Call parent class constructor  
  
        System.out.println("Child class constructor called.");  
  
    }  
  
}
```



```
public class Main

{

    public static void main(String[] args) {

        B obj = new B();

    }

}
```

Output:

Parent class constructor called.

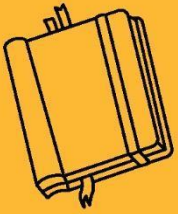
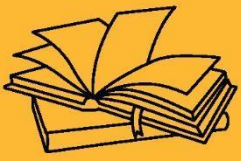
Child class constructor called.

CONCLUSION:

- The super keyword provides a way for subclasses to access and reuse members of their immediate parent class.
- It is a useful tool to maintain clear, readable, and organized inheritance-based code in Java.

KEY TAKEAWAYS

- Must be the first statement in a constructor when calling super().
- Used to access overridden methods and shadowed variables from the parent class.
- Helps in achieving code reusability and avoiding duplication in subclass definitions.



INHERITANCE & INTERFACE





SUB LESSON 4.5

FINAL KEYWORD

In Java, the final keyword is a non-access modifier used to restrict the user.

DYNAMIC METHOD DISPATCH

Final keyword can be applied to variables, methods, and classes, and its behavior changes based on the context. The final keyword helps in writing secure and well-structured programs by preventing changes to the defined element.

FINAL VARIABLE

- A variable declared with final cannot be changed after it is initialized.
- It behaves like a constant.

Syntax:

```
final datatype vname = value;
```

Example:

```
final int MAX = 100;
```

```
MAX = 200; // Error: cannot assign a value to final variable
```

Here, MAX variable become constant, we can't change the value of it after assigning.

FINAL METHOD

- A method declared as final cannot be overridden by subclasses.
- Ensures that the method behavior remains unchanged in inheritance.

Example:

```
class A
```



```
{  
  
    final void show()  
  
    {  
  
        System.out.println("Final method in A");  
  
    }  
  
}
```

```
class B extends A  
{  
    // void show() {} // Error: Cannot override final method  
}
```

FINAL CLASS

- A class declared as final cannot be extended (no subclass can be created).
- It is used to prevent inheritance.

Example:

```
final class Vehicle  
{  
  
    void run()  
  
    {  
  
        System.out.println("Running...");  
  
    }  
  
}
```




```
}
```

```
// class Car extends Vehicle {} // Error: Cannot inherit from final class
```

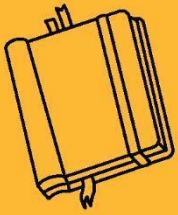
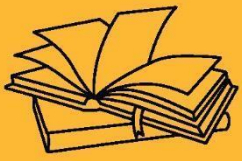
Final class can not be inherited , so we can not create child of final class.

CONCLUSION:

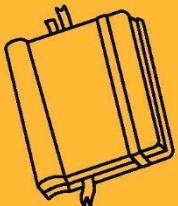
- The final keyword is used for security, stability, and control in Java programs.
- It is especially useful in API development to prevent method overriding and class extension.

KEY TAKEAWAYS

- final variables must be initialized only once.
- final methods provide a fixed implementation that cannot be changed by subclasses.
- final classes ensure that no other class can inherit or modify their behavior.



INHERITANCE & INTERFACE





SUB LESSON 4.6

ABSTRACT CLASS

Abstract classes help in defining a base structure for subclasses while allowing some implementation details.

ABSTRACT CLASS

An abstract class in Java is a class that is declared using the keyword `abstract`. It cannot be instantiated directly, meaning you cannot create an object of an abstract class. It is used to provide a base for subclasses, often containing abstract methods that must be implemented by derived classes.

- Declared using the `abstract` keyword.
- Can have abstract methods (methods without body) and non-abstract methods (methods with body).
- Can include constructors, static methods, fields, and final methods.
- Cannot be instantiated directly.
- Must be inherited by a subclass which implements the abstract methods.

Syntax

```
abstract class ClassName

{

    // abstract method (no body)

    abstract void method1();

    // regular method (with body)
```



```
        void method2() {  
  
            System.out.println("This is a regular method.");  
  
        }  
  
    }
```

Example

```
abstract class Animal  
{  
  
    // Abstract method  
  
    abstract void sound();  
  
  
    // Concrete method  
  
    void sleep()  
  
    {  
  
        System.out.println("Sleeping...");  
  
    }  
  
}  
  
class Dog extends Animal  
{  
  
    void sound()
```



```
{  
  
    System.out.println("Dog barks");  
  
}  
  
}  
  
public class Main  
  
{  
  
    public static void main(String[] args)  
  
    {  
  
        Dog d = new Dog();  
  
        d.sound(); // Output: Dog barks  
  
        d.sleep(); // Output: Sleeping...  
  
    }  
  
}
```

Output

Dog barks

Sleeping...

CONCLUSION:

- An abstract class provides a way to create a blueprint for other classes.
- It allows code reuse through regular methods and enforces rules through abstract methods.
- It is a vital part of Java's object-oriented features when designing large, extensible applications.



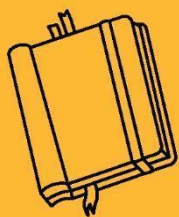
KEY TAKEAWAYS

- If a class contains even one abstract method, it must be declared abstract.
- Subclasses must override all abstract methods of the abstract class.
- You can extend only one abstract class .
- Abstract classes can have constructors, which are called when a subclass object is created.

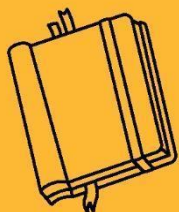
OUTCOMES OF CHAPTER 4

After completing this unit, students will be able to:

- Understand the concept of inheritance and its various types like single, multilevel, and hierarchical inheritance for code reusability.
- Apply method overriding to provide specific implementations in subclasses.
- Demonstrate the use of dynamic method dispatch to achieve runtime polymorphism in Java.
- Use the super keyword to access parent class methods, constructors, and variables.
- Apply the final keyword to restrict class inheritance, method overriding, and variable modification.



INHERITANCE & INTERFACE





SUB LESSON 5.1

DEFINING INTERFACE, INHERITANCE ON INTERFACES, IMPLEMENTING INTERFACE, MULTIPLE INHERITANCE USING INTERFACE

OBJECTIVES OF CHAPTER 5

5.1 Defining Interface, Inheritance on Interfaces, Implementing Interface, Multiple Inheritance using Interface

- Understand how to define an interface and declare abstract methods in Java.
- Learn how to implement interfaces in classes and extend one interface from another.
- Explore how Java uses interfaces to achieve multiple inheritance and design loosely coupled systems

Interfaces in Java provide a way to achieve full abstraction by defining a set of abstract methods that implementing classes must follow.



INTERFACE

In Java, an interface is like a blueprint for a class. It contains method declarations but no method bodies, meaning it tells what a class should do, but not how to do it. Interfaces are used to achieve abstraction and multiple inheritance in Java. When a class implements an interface, it must provide the actual implementation for all the methods declared in the interface.

Interfaces help in building flexible and loosely-coupled code by allowing different classes to follow the same contract or behavior, even if they are not related by inheritance.

An interface in Java is a reference type similar to a class, but it can only contain abstract methods, default methods, static methods, and constants (fields marked as public, static, and final).

- Interfaces use the keyword `interface`.
- All methods in an interface are implicitly public and abstract (Java 8+ can include default/static).
- Interfaces cannot be instantiated.
- Used to define a contract for implementing classes.

Syntax

```
interface InterfaceName

{

    void method1(); // abstract method

}
```

A class uses the `implements` keyword to use an interface. The class must provide concrete implementations of all the abstract methods defined in the interface.

Example

```
interface Drawable

{
```



```
        void draw();
    }

    class Circle implements Drawable
    {
        public void draw()
        {
            System.out.println("Drawing Circle");
        }
    }

    public class Main
    {
        public static void main(String[] args)
        {
            Circle c = new Circle();

            c.draw();
        }
    }
```

Output

Drawing Circle

INHERITANCE ON INTERFACE

Just like classes, interfaces can extend other interfaces using the `extends` keyword. A child interface inherits all methods from its parent interface.



Example

interface A

```
{  
  
    void methodA();  
  
}
```

interface B extends A

```
{  
  
    void methodB();  
  
}
```

class Test implements B

```
{  
  
    public void methodA()  
  
    {  
  
        System.out.println("Method A");  
  
    }  
  
    public void methodB()  
  
    {  
  
        System.out.println("Method B");  
  
    }  
  
}
```



```
public class Main

{

    public static void main(String[] args)

    {

        Test obj = new Test();

        obj.methodA();

        obj.methodB();

    }

}
```

Output

Method A

Method B

MULTIPLE INHERITANCE USING INTERFACE

Java does not support multiple inheritance with classes to avoid ambiguity. But it supports multiple inheritance using interfaces, since there is no method body and thus no conflict.

Example

```
interface Printable
```

```
{

    void print();

}
```

```
interface Showable
```



```
{  
  
    void show();  
  
}  
  
class Demo implements Printable, Showable  
  
{  
  
    public void print()  
  
    {  
  
        System.out.println("Printing...");  
  
    }  
  
    public void show()  
  
    {  
  
        System.out.println("Showing...");  
  
    }  
  
}  
  
public class Main  
  
{  
  
    public static void main(String[] args)  
  
    {  
  
        Demo d = new Demo();  
  
        d.print(); // Output: Printing...
```



```
d.show(); // Output: Showing...  
  
}  
  
}
```

Output

Printing...

Showing...

CONCLUSION:

- Interfaces in Java are powerful tools that allow for abstraction and flexibility in application design.
- Interface enable multiple inheritance and promote clean architecture through contracts that ensure consistency across classes.

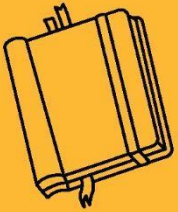
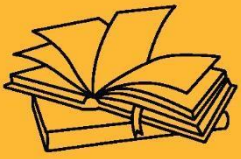
KEY TAKEAWAYS

- Interface provides complete abstraction.
- Can be used to achieve multiple inheritance.
- Promotes loose coupling in design.

OUTCOMES OF CHAPTER 5

After completing this unit, students will be able to:

- Understand how to define and implement interfaces, and use them to achieve abstraction and contract-based programming in Java.
- Apply the concept of multiple inheritance using interfaces and interface inheritance to design flexible and reusable programs.



PACKAGE





SUB LESSON 6.1

CREATING PACKAGE, IMPORTING PACKAGE

OBJECTIVES

6.1 Creating Package, Importing Package

- Understand how to create and organize packages to group related classes and interfaces in Java.
- Learn how to import user-defined and built-in packages using the import statement.
- Explore the benefits of using packages for modularity, code reuse, and namespace management.

6.2 Access Rules for Packages, CLASSPATH

- Learn how Java access modifiers (public, protected, default) work across packages.
- Understand the role of the CLASSPATH environment variable in locating class files.
- Explore how package access control enhances security and encapsulation in Java applications.



Packages in Java are a way to group related classes and interfaces together, providing better structure and organization to code. They help avoid name conflicts and make it easier to locate and use classes within large software projects.

PACKAGE

In Java, a package is a namespace that organizes a set of related classes and interfaces. Conceptually, it is similar to folders on your computer.

- A package act as “containers” for classes.
- A package represents a directory that contain related group of classes and interface.
- A package provides a mechanism for grouping a variety of similar types of classes, interfaces and sub-packages.
- The main advantage of package are:
 - Reusability of code & class
 - provides a way to hide classes

There are basically only 2 types of Java package as follow.

- System Package or Java API
- User Define Package

SYSTEM PACKAGE OR JAVA API

These are the packages that are already available in Java. They are a part of the Java API (Application Programming Interface) and contain various prewritten classes and interfaces to perform different tasks. It also known as inbuilt package.

Example of System Package:

Package Name	Description
--------------	-------------



java.lang	Contains fundamental classes like String, Math, Integer, etc. (imported automatically).
java.util	Contains utility classes like ArrayList, HashMap, Collections, Date, etc.
java.io	Used for input and output operations like reading and writing files.
java.net	Provides classes for networking like sockets, URL, etc.
java.sql	Used for database connectivity with JDBC.
java.awt	Used for building graphical user interfaces (GUI).
javax.swing	Provides more advanced GUI components than AWT.

User Define Package

These are the packages created by the programmer to organize classes logically and avoid name conflicts in larger projects. A user-defined package is a package created by the programmer to group related classes together. It helps organize code better and avoid class name conflicts. To create a package, use the package keyword at the top of the java file. To use the class from the user, define package, use the import statement.

CREATE AND IMPORT THE PACKAGE

- Declare the package at the beginning of a file using the form

```
package packagename;
```

- Define the class that is to be put in the package and declare it public.

Syntax:

```
package name_Package;
```

```
public class Demo
```



```
{  
  
    Body of the class  
  
}
```

- import package in other file using following syntax:

```
import name_package;
```

Step 1: Create a Package and a Class

(Myclass.java)

```
package mypackage;  
  
public class MyClass  
{  
  
    public void displayMessage()  
  
    {  
  
        System.out.println("Hello from MyClass in mypackage!");  
  
    }  
  
}
```

Step 2: Use the Package in Another Class

(TestPackage.java)

```
import mypackage.MyClass;  
  
public class TestPackage
```



```
{  
  
    public static void main(String[] args)  
  
    {  
  
        MyClass obj = new MyClass();  
  
        obj.displayMessage();  
  
    }  
  
}
```

Step 3: Compile and Run

```
javac -d . MyClass.java // Compiles and places class in folder structure
```

```
javac TestPackage.java // Compiles main class
```

```
java TestPackage // Runs the program
```

Output

Hello from MyClass in mypackage!

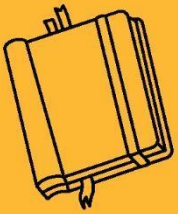
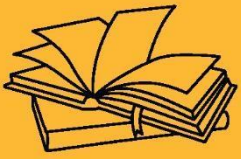
CONCLUSION

- Java supports both built-in and user-defined packages.
- Built-in packages make programming easier by providing ready-made tools.
- User-defined packages help organize and manage large projects efficiently.



KEY TAKEAWAYS

- Java provides two types of packages: Built-in (predefined) and User-defined.
- Built-in packages are part of the Java API, offering ready-to-use classes for various tasks like input/output, data structures, networking, and GUI.
- User-defined packages are created by developers to organize classes logically and avoid naming conflicts.
- Use the package keyword to define a package and the import keyword to access it in another class.



PACKAGE



SUB LESSON 6.2

ACCESS SPECIFIER, CLASSPATH

In Java, packages help organize classes and interfaces logically. Along with this organization, access specifiers determine the visibility of classes and their members across packages. Understanding access rules and class path configuration is crucial for smooth project development and execution.

ACCESS SPECIFIER

In Java, access specifiers (also known as access modifiers) are keywords used to define the scope or visibility of classes, methods, variables, and constructors. They help enforce the concept of encapsulation, which is one of the key principles of Object-Oriented Programming (OOP). By controlling access, you can hide internal object details and expose only what's necessary.

	No modifier	Private	Protected	Public
Same class	Yes	Yes	Yes	Yes
Same package within sub class	Yes	No	Yes	Yes
Same package non sub class	Yes	No	Yes	Yes
Different package subclass	No	No	Yes	Yes



Different package sub class	No	No	No	Yes
-----------------------------	----	----	----	-----

1. public Access Specifier

- Members (class, method, or variable) marked as public are accessible from anywhere in the program.
- There are no restrictions.

2. private Access Specifier

- Members marked as private are accessible only within the same class.
- Not visible to other classes, even within the same package.
- Commonly used for data hiding.

3. default Access Specifier *(No Modifier)*

- When no access specifier is mentioned, the default is package-private.
- Members are accessible only within the same package.
- Not accessible outside the package.

4. protected Access Specifier

- Members are accessible:
- Within the same package.
- In subclasses (even if they are in different packages).

Example

```
package mypackage;

public class TestAccess {
```




```
public int a = 10;

protected int b = 20;

int c = 30;    // default

private int d = 40;

public void show() {

    System.out.println(a + " " + b + " " + c + " " + d);

}

}
```

Another class in same package:

java

CopyEdit

```
class SamePackage {

    public static void main(String[] args) {

        TestAccess obj = new TestAccess();

        System.out.println(obj.a); // accessible

        System.out.println(obj.b); // accessible

        System.out.println(obj.c); // accessible

        // System.out.println(obj.d); // Error: private

    }

}
```



Access specifiers are powerful tools in Java that help manage visibility and access of class members. Using them correctly leads to better data protection and cleaner program structure.

CLASSPATH

In Java, the CLASSPATH is an important environment variable or command-line option that tells the Java compiler and Java Virtual Machine (JVM) where to look for user-defined classes and packages. It plays a key role when your program uses external or custom packages located outside the standard Java library.

- CLASSPATH is used by the compiler (javac) and the interpreter (java) to locate class files (compiled .class files).
- By default, Java looks for classes in the current directory.
- If your .class files or packages are located in another directory, you must specify that location using the CLASSPATH.

Example

File A.java

```
import p1.*;
```

```
Package p2;
```

```
Import java.io.*;
```

```
public class A
```

```
{
```

```
    public static void main(String args[])
```

```
    {
```

```
        B obj = new B();
```



```
        obj.hi();  
    }  
}
```

File B.java

```
Package p1;  
  
public class B  
{  
  
    public void hi()  
    {  
  
        System.out.println("Hello");  
  
    }  
  
}
```

Output

```
C:\Windows\system32\cmd.exe  
Microsoft Windows [Version 6.1.7601]  
Copyright (c) 2009 Microsoft Corporation. All rights reserved.  
  
C:\Users\HETAL>cd Desktop  
C:\Users\HETAL\Desktop>E:  
E:\>cd "Java Program"  
E:\Java Program>cd Package  
E:\Java Program\Package>javac -d . B.java  
E:\Java Program\Package>javac -d . A.java  
E:\Java Program\Package>java p2.java  
Error: Could not find or load main class p2.java  
E:\Java Program\Package>java p2.A  
Hello  
E:\Java Program\Package>_
```



CONCLUSION:

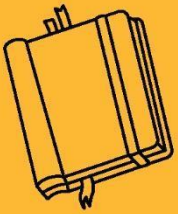
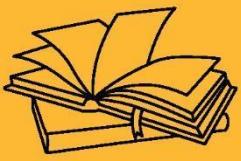
- Understanding access specifiers and the CLASSPATH is crucial for writing secure, well-structured, and modular Java programs.
- Access specifiers help control how data and methods can be accessed across different classes and packages, improving encapsulation and reducing errors.
- On the other hand, CLASSPATH helps the compiler and JVM locate the required classes, especially when working with custom packages or external libraries.

KEY TAKEAWAYS

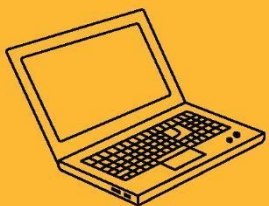
- Access specifiers (private, default, protected, public) control the visibility of classes, methods, and variables, ensuring encapsulation and modularity in Java programs.
- CLASSPATH is used by the Java compiler and JVM to locate class files and packages, especially when working with user-defined or external libraries.
- Proper understanding of access control and classpath setup helps prevent runtime errors and improves code organization across packages.

OUTCOMES OF CHAPTER 6

- Students will understand how to group related classes and interfaces using packages, promoting modularity and reusability in Java programs.
- Learners will be able to implement proper access control in Java by applying private, default, protected, and public keywords to classes, methods, and variables.
- Students will learn how to set and manage the classpath to successfully compile and run Java programs that use packages and external classes.



PACKAGE, MULTITHREADING & EXCEPTION HANDLING





SUB LESSON 7.1

CREATING THREAD, EXTENDING THREAD CLASS, IMPLEMENTING RUNNABLE INTERFACE, LIFE CYCLE OF A THREAD

OBJECTIVES OF CHAPTER 7

7.1 Creating thread, extending Thread class, implementing Runnable interface, life cycle of a thread

- Understand how to create and run threads using both Thread class and Runnable interface.
- Explain and illustrate the life cycle of a thread including its different states like new, runnable, running, blocked, and terminated.

7.2 Thread priority & methods

- Learn how to assign priority to threads and understand how thread priority influences execution.
- Explore commonly used thread methods such as start(), sleep(), join(), yield(), and understand their impact on thread behavior.

7.3 Thread synchronization & messaging

- Understand the concept of synchronization to manage multiple threads accessing shared resources and prevent data inconsistency.
- Demonstrate inter-thread communication using wait/notify mechanisms to facilitate message passing and coordination between threads.

7.4 communication, Suspending, Resuming and stopping thread

- Learn how threads communicate and coordinate through wait-notify, and manage their flow in multi-threaded applications.

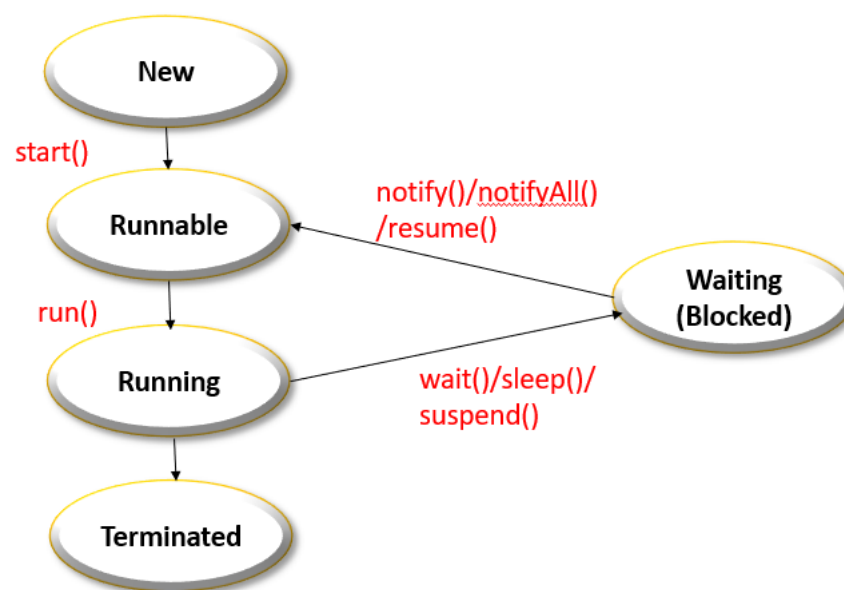


- Understand deprecated thread control methods like `suspend()`, `resume()`, and `stop()`, and learn modern alternatives to safely manage thread states.

In Java, multithreading allows concurrent execution of two or more threads, enabling efficient CPU utilization. Java provides built-in support for creating and managing threads through the `Thread` class and the `Runnable` interface.

THREAD

- A thread is a lightweight subprocess, the smallest unit of processing.
- It is a separate path of execution, and multiple threads can run in a program to perform different tasks simultaneously.
- A thread is an independent path of execution within a program.
- Threads are independent, if there occurs exception in one thread, it doesn't affect other threads.
- It shares a common memory area.





LIFE CYCLE OF THREAD

In Java, a thread goes through several states from its creation to completion. These states are defined by the Thread State machine and managed by the JVM (Java Virtual Machine).

- **New State**
 - A thread is in the New state when it is created but not yet started.
 - It is created by creating an object of a class that extends Thread or implements Runnable.
- **Runnable State**
 - The thread moves to the Runnable state when the start() method is called.
 - In this state, the thread is ready to run, but actual execution depends on the thread scheduler (handled by the JVM).
- **Running State**
 - When the thread scheduler selects the thread, it enters the Running state.
 - The thread's run() method starts executing.
- **Blocked/Waiting State**
 - A thread enters this state when it is temporarily inactive.
 - It may be waiting for resources, sleeping, or waiting for another thread to finish.
- **Terminated (Dead) State**
 - A thread enters the Terminated (or Dead) state when:
 - Its run() method completes execution, or
 - It is forcefully terminated (not recommended)

CREATE THREAD

In Java, a thread is a lightweight process that allows multiple tasks to run concurrently. There are two main ways to create a thread:

By Extending the Thread class

- Create a class that extends the Thread class.



- Override the run() method.
- Create an object of your class.
- Call the start() method to begin execution.

Example

```
class MyThread extends Thread
```

```
{
```

```
    public void run()
```

```
    {
```

```
        System.out.println("Thread running using Thread class");
```

```
    }
```

```
    public static void main(String[] args)
```

```
    {
```

```
        MyThread t1 = new MyThread(); // New thread created
```

```
        t1.start();           // Starts the thread
```

```
    }
```

```
}
```

Output

Thread running using Thread class

By Implementing the Runnable Interface

- Create a class that implements Runnable.
- Override the run() method.



- Create an object of your class and pass it to a Thread object.
- Start the thread using start().

Example

class MyRunnable implements Runnable

```
{  
  
    public void run()  
  
    {  
  
        System.out.println("Thread running using Runnable interface");  
  
    }  
  
    public static void main(String[] args)  
  
    {  
  
        MyRunnable obj = new MyRunnable();  
  
        Thread t1 = new Thread(obj); // Pass Runnable object to Thread  
  
        t1.start();           // Starts the thread  
  
    }  
  
}
```

Output

Thread running using Runnable interface

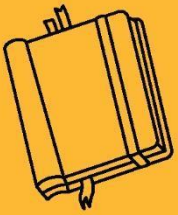
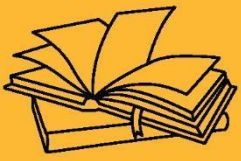


CONCLUSION

- Creating threads in Java allows programs to perform multiple tasks simultaneously, improving efficiency and responsiveness. Java supports multithreading through two primary mechanisms: extending the Thread class and implementing the Runnable interface.
- Both methods achieve the same goal, using the Runnable interface is generally preferred in real-world applications due to its flexibility and better object-oriented design support.

KEY TAKEAWAYS

- Threads allow concurrent execution of tasks in Java.
- Java provides two ways to create a thread: by extending Thread or implementing Runnable.
- The Runnable interface is preferred when a class needs to extend another class.
- Use start() method to begin thread execution, not run().
- Proper thread management enhances program performance and user experience.



PACKAGE, MULTITHREADING & EXCEPTION HANDLING





SUB LESSON 7.2

THREAD PRIORITIES & METHOD

Multithreading in Java allows concurrent execution of two or more parts of a program to maximize CPU utilization. Thread priorities help in deciding the order of execution when multiple threads are in the ready state. Java provides several methods to control thread behavior and status during execution.

THREAD PRIORITIES & METHODS

THREAD PRIORITIES

- Thread priority helps the thread scheduler determine the execution order of threads.
- Higher priority threads are more likely to run before lower priority threads.
- In java, each thread assigned a priority ,which affects the order in which it is schedule for running.

Thread Priorities like,

CONSTANT	VALUE	DESCRIPTION
THREAD.MIN_PRIORITY	1	Lowest priority
THREAD.NORM_PRIORITY	5	Default/Normal priority
THREAD.MAX_PRIORITY	10	Highest priorit

Example

```
class MyThread extends Thread {  
  
    public void run() {
```



```
        System.out.println(Thread.currentThread().getName() + " - Priority: " +
Thread.currentThread().getPriority());

    }

}

public class ThreadPriorityExample {

    public static void main(String[] args) {

        MyThread t1 = new MyThread();

        MyThread t2 = new MyThread();

        MyThread t3 = new MyThread();

        t1.setPriority(Thread.MIN_PRIORITY);

        t2.setPriority(Thread.NORM_PRIORITY);

        t3.setPriority(Thread.MAX_PRIORITY);

        t1.start();

        t2.start();

        t3.start();

    }

}
```

Output

Thread-0 - Priority: 1

Thread-1 - Priority: 5

Thread-2 - Priority: 10



THREAD METHODS

- `sleep()` : The `sleep()` method of `Thread` class is used to sleep a thread for the specified amount of time.

Syntax:

- `public static void sleep(long miliseconds)throws InterruptedException`
- `public static void sleep(long miliseconds, int nanos)throws InterruptedException`
- `join()` : The `join()` method waits for a thread to die. In other words, it causes the currently running threads to stop executing until the thread it joins with completes its task.

Syntax:

- `public void join()throws InterruptedException`
- `public void join(long milliseconds)throws InterruptedException`
- `currentThread()` : The `currentThread()` method returns a reference to the currently executing thread object.

Syntax:

- `public static Thread currentThread()`
- `getName()` : The `getName()` method returns a name of thread.

Syntax:

- `public final String getName()`
- `isAlive()`: The `isAlive()` method Tests if this thread is alive.

Syntax:

- `public final boolean isAlive()`
- `getPriority()` : The `getPriority()` method returns a priority of thread.

Syntax:



➤ `public final int getPriority()`

Example

```
class MyThread extends Thread
```

```
{
```

```
    public MyThread(String name, int priority) {
```

```
        super(name);
```

```
        setPriority(priority); // Setting thread priority
```

```
    }
```

```
    public void run() {
```

```
        System.out.println(getName() + " is running with priority " + getPriority()); }  
}
```

```
public class SimpleThreadDemo
```

```
{
```

```
    public static void main(String[] args) {
```

```
        MyThread t1 = new MyThread("High-Priority-Thread", Thread.MAX_PRIORITY);
```

```
        MyThread t2 = new MyThread("Low-Priority-Thread", Thread.MIN_PRIORITY);
```

```
        t1.start();
```

```
        t2.start();
```

```
        System.out.println("Current thread: " + Thread.currentThread().getName());
```

```
    }
```




```
}
```

Output

High-Priority-Thread is running with priority 10

Low-Priority-Thread is running with priority 1

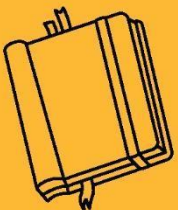
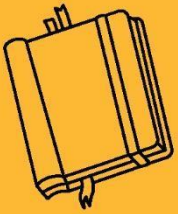
Current thread: main

CONCLUSION

- Thread priorities and methods play a crucial role in managing thread execution in Java. By assigning priorities, developers can influence the order of thread execution, although actual scheduling depends on the JVM.
- Java provides several built-in methods to manage thread behavior and coordination effectively. Java provides two types of packages: Built-in (predefined) and User-defined.

KEY TAKEAWAYS

- Thread priority helps in setting the importance of a thread but does not guarantee execution order.
- Proper use of thread methods and priorities improves application performance and responsiveness.
- JVM's thread scheduler decides execution based on priority and other system factors.



PACKAGE, MULTITHREADING & EXCEPTION HANDLING



SUB LESSON 7.3

THREAD SYNCHRONIZATION & MESSAGING

In a multithreaded environment, threads often share common resources like data or memory, which may lead to conflicts or inconsistent results. To handle such issues, thread synchronization and messaging are used to coordinate the execution and communication between threads safely and efficiently.

THREAD SYNCHRONIZATION

When multiple threads access shared resources concurrently, there's a risk of data inconsistency and conflicts. Thread synchronization helps manage thread access, while messaging enables safe inter-thread communication to coordinate tasks efficiently.

- Synchronization is the process of controlling the access of multiple threads to shared resources.
- It ensures that only one thread can access a resource at a time, preventing race conditions and maintaining data integrity.
- Threads often share common resources like variables, files, or objects.
- If threads are not synchronized, it may lead to unexpected results and bugs.
- Synchronization is especially needed in multithreaded environments where resource sharing occurs.

Java provides different ways to achieve synchronization:

SYNCHRONIZED METHODS

- If you declare any method as synchronized, it is known as synchronized method.
- Synchronized method is used to lock an object for any shared resource.
- When a thread invokes a synchronized method, it automatically acquires the lock for that object and releases it when the thread completes its task.



- Declaring a method as **synchronized** ensures that only one thread can execute it at a time.

Syntax

```
public synchronized void show()

{

    // synchronized code

}
```

Example

```
class Table_Demo

{

synchronized void printT(int n){

    //synchronized method

    for(int i=1;i<=5;i++){

        System.out.println(n*i);

    try{

        Thread.sleep(400);

    }catch(Exception e){System.out.println(e);}

    }

}

}
```

```
class MyThread1 extends Thread
```



```
{  
  
    Table t;  
  
    MyThread1(Table t){  
  
        this.t=t;  
  
    }  
  
    public void run(){  
  
        t.printT(5);  
  
    }  
}  
  
class MyThread2 extends Thread  
  
{  
  
    Table t;  
  
    MyThread2(Table t){  
  
        this.t=t;  
  
    }  
  
    public void run(){  
  
        t.printT(10);  
  
    }  
}  
  
public class TestSynchronization2
```



```
{  
  
    public static void main(String args[])  
  
    {  
  
        Table_Demo obj = new Table_Demo();  
  
        MyThread1 t1=new MyThread1(obj);  
  
        MyThread2 t2=new MyThread2(obj);  
  
        t1.start();  
  
        t2.start();  
  
    }  
}
```

Output

5

10

15

20

25

10

20

30

40

SYNCHRONIZED BLOCK

- Synchronized block can be used to perform synchronization on any specific resource of the method.
- Suppose you have 50 lines of code in your method, but you want to synchronize only 5 lines, you can use synchronized block.
- If you put all the codes of the method in the synchronized block, it will work same as the synchronized method.
- Instead of synchronizing the whole method, a specific block of code can be synchronized.

SYNTAX:

```
synchronized (object reference expression)

{

    //code block

}
```

Example

```
class Table_Demo

{

    void printT(int n)

    {

        synchronized(this){

            for(int i=1;i<=5;i++){

                System.out.println(n*i);

            }

        }

    }

}
```



```
        try{

            Thread.sleep(400);

        }catch(Exception e){System.out.println(e) }

    } }

}

}

class MyThread1 extends Thread

{

    Table t;

    MyThread1(Table t){

        this.t=t;

    }

    public void run()

    {

        t.printT(5);

    }

}

class MyThread2 extends Thread

{

    Table t;
```




```
MyThread2(Table t){

    this.t=t;

}

public void run(){

    t.printT(10);

}

}

public class TestSynchronizedBlock1

{

    public static void main(String args[])

    {

        Table_Demo obj = new Table_Demo();

        MyThread1 t1=new MyThread1(obj);

        MyThread2 t2=new MyThread2(obj);

        t1.start();

        t2.start();

    }

}
```

Output

5



10

15

20

25

10

20

30

40

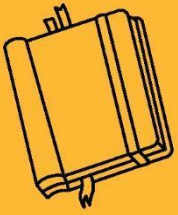
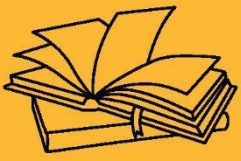
50

CONCLUSION

- Thread synchronization ensures that only one thread accesses a shared resource at a time, avoiding inconsistency. Messaging helps threads to communicate and coordinate execution, enhancing inter-thread cooperation.

KEY TAKEAWAYS

- Synchronization avoids race conditions and ensures thread safety.
- synchronized methods or blocks are used to control thread access. Proper use of synchronization and messaging leads to stable and bug-free multithreaded applications.



PACKAGE, MULTITHREADING & EXCEPTION HANDLING





SUB LESSON 7.4

COMMUNICATION, SUSPENDING, RESUMING AND STOPPING THREAD

Multithreading in Java not only enables concurrent execution but also requires proper coordination between threads. Java provides mechanisms to allow communication and control actions like suspending, resuming, and stopping threads to maintain efficient execution.

THREAD COMMUNICATION

Thread communication is required when multiple threads share resources and need to coordinate their actions. Java provides methods like `wait()`, `notify()`, and `notifyAll()` from the `Object` class for inter-thread communication.

Methods for thread communication

- `wait()`: Causes the current thread to wait until another thread invokes `notify()` or `notifyAll()`.
- Syntax:
 - `public final void wait()throws InterruptedException`
 - `public final void wait(long timeout)throws InterruptedException`
- `notify()`: Wakes up a single thread that is waiting on the object's monitor.
- Syntax:
 - `public final void notify()`
- `notifyAll()`: Wakes up all the threads that are waiting on the object's monitor.

Syntax:

- `public final void notifyAll()`

Example



```
class SharedResource

{

    boolean flag = false;


    synchronized void produce()

    {

        if (flag) {

            try {

                wait();

            } catch (Exception e) {}

        }

        System.out.println("Producing...");

        flag = true;

        notify();

    }

    synchronized void consume() {

        if (!flag) {

            try {

                wait();

            } catch (Exception e) {}

        }

    }

}
```



```
    }  
  
    System.out.println("Consuming...");  
  
    flag = false;  
  
    notify();  
  
}  
  
}
```

Output

```
Producing...  
Consuming...  
Producing...  
Consuming...  
Producing...  
Consuming...  
Producing...  
Consuming...  
Producing...  
Consuming...
```

CONCLUSION

- Thread communication is an essential concept in Java for coordinating actions between multiple threads that share resources.



- By using synchronization along with `wait()`, `notify()`, and `notifyAll()`, threads can communicate effectively without creating resource conflicts or inconsistent data.
- This ensures smooth execution and avoids problems like deadlocks or race conditions in multithreaded environments.

KEY TAKEAWAYS

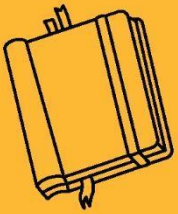
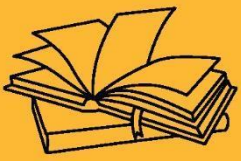
- `wait()`, `notify()`, and `notifyAll()` are used to enable communication between threads in synchronized blocks.
- Thread communication helps maintain proper sequence and data consistency when multiple threads access shared resources.
- Effective communication prevents deadlocks and ensures that resources are used efficiently.



OUTCOMES OF CHAPTER 7

After completing this topic, students will be able to:

- Understand how threads work in Java and create them using both the Thread class and the Runnable interface.
- Explain and manage the life cycle of a thread, including states like New, Runnable, Running, Blocked, and Terminated.
- Set and utilize thread priorities to control execution order when multiple threads compete for CPU time.
- Use various thread methods such as start(), run(), sleep(), join(), and understand their practical applications.
- Implement thread synchronization to safely manage shared resources using synchronized methods/blocks.
- Apply inter-thread communication using wait(), notify(), and notifyAll() for effective coordination between threads.
- Understand messaging and thread control, such as suspending, resuming, and stopping threads (though deprecated methods should be avoided).
- Develop multi-threaded applications with efficient resource sharing, avoiding data inconsistency and ensuring program stability.



PACKAGE, MULTITHREADING & EXCEPTION HANDLING





SUB LESSON 8.1

ERRORS, EXCEPTIONS, INBUILT EXCEPTION

OBJECTIVES OF CHAPTER 8

8.1 errors, exceptions, Inbuilt Exception

- To understand the difference between errors and exceptions in Java, and to learn about the commonly used inbuilt exception classes provided by Java.

8.2 try..catch statement, multiple catch blocks

- To learn how to handle exceptions using the try..catch statement and how to use multiple catch blocks to manage different types of exceptions effectively.

8.3 throw and throws keywords, finally clause

- To understand the use of throw and throws keywords for throwing exceptions and declaring them. Also, to learn the purpose of the finally clause and how it ensures code execution even if an exception occurs.

8.4 uses of exceptions, user defined exceptions

- To understand the importance of exception handling in programs and how to create and use user-defined exceptions to handle custom error conditions in a program.

In Java, errors and exceptions are problems that can occur during the execution of a program. Errors are serious issues that typically cannot be handled by the program (e.g., `OutOfMemoryError`, `StackOverflowError`) and usually indicate system-level problems. On the other hand, exceptions are conditions that a program should handle, such as trying to open a missing file or dividing a number by zero.



EXCEPTIONS

In the Java programming language, runtime problems are categorized as errors and exceptions. These problems may arise due to faulty logic, invalid user input, hardware failures, or software issues. Java provides structured ways to detect and handle these problems so that the program does not crash unexpectedly.

- **Error:** Represents serious problems that an application should not try to handle. Errors are mostly system-level issues, such as memory leaks or stack overflows.
- **Exception:** Represents conditions that an application might want to catch and handle during runtime.

ERROR:

Errors in Java are unchecked and derive from the `java.lang.Error` class. These are critical issues that usually indicate that the JVM is in a problematic state. Since these are generally beyond the control of the program, they should not be caught or handled.

- Errors are not meant to be caught in a try-catch block.
- Typically caused by the environment in which the application is running.
- Usually fatal and unrecoverable.
- There are three categories of errors:
 - **Syntax errors** - arise because the rules of the language have not been followed. They are detected by the compiler.
 - **Runtime errors** - occur while the program is running if the environment detects an operation that is impossible to carry out.
 - **Logic errors** - occur when a program doesn't perform the way it was intended to.

Examples of Errors:

- `OutOfMemoryError`
- `StackOverflowError`
- `VirtualMachineError`



EXCEPTION

Exceptions are events that disrupt the normal flow of the program. Unlike errors, exceptions are conditions that can be anticipated and recovered from in code. All exceptions are derived from the `java.lang.Exception` class.

Types of Exceptions:

- Checked Exceptions: These are checked at compile-time. Example: `IOException`, `SQLException`.
- Unchecked Exceptions: These occur at runtime. Example: `ArithmeticException`, `NullPointerException`.

Example 1

```
int x = 10/2; → Answer: 5
```

```
int x = 10/0; → Arithmetic Divide by Zero
```

Example 2

```
int a[4] = { 2,3,6,4,};
```

```
System.out.println(a[0]); → Answer: 2
```

```
System.out.println(a[7]); → Array Index out of Bound
```

Exception handling in Java is used to manage errors that occur while a program is running. These errors, called exceptions, are unexpected events that can disrupt the normal flow of the program. For example, an exception may occur if you divide a number by zero, try to access an invalid index in an array, or attempt to open a file that does not exist. Without proper handling, such errors can cause the program to crash. Java provides a way to handle these situations using exception handling techniques like the try-catch block, finally block, throw and throws keywords, and custom exceptions.



The Exception Handling in Java is one of the powerful mechanism to handle the runtime errors so that normal flow of the application can be maintained. By using exception handling, developers can write programs that respond to errors gracefully, show meaningful messages to users, and continue running safely instead of stopping abruptly.

- try – Used to wrap the code that might cause an exception.

Example:

```
try {  
    int a = 10 / 0;  
}
```

- catch – placed directly after the try block to handle one or more exception types

Example:

```
try {  
    int a = 10 / 0;  
}  
catch (ArithmeticException e) {  
    System.out.println("Cannot divide by zero.");  
}
```

- finally – optional statement used after a try-catch block A finally block of code always executes, whether or not an exception has occurred.

Example:

```
finally {  
    System.out.println("This will always run.");  
}
```

INBUILT EXCEPTION

Java provides a rich set of predefined exceptions that cover almost all the standard error conditions.



Exception	Description
ArithmeticException	It is thrown when an exceptional condition has occurred in an arithmetic operation.
ArrayIndexOutOfBoundsException	It is thrown to indicate that an array has been accessed with an illegal index. The index is either negative or greater than or equal to the size of the array.
ClassNotFoundException	This exception is raised when we try to access a class whose definition is not found.
FileNotFoundException	An exception that is raised when a file is not accessible or does not open.
IOException	It is thrown when an input-output operation is failed or interrupted.
InterruptedException	It is thrown when a thread is waiting, sleeping, or doing some processing, and it is interrupted.
NoSuchFieldException	It is thrown when a class does not contain the field (or variable) specified.

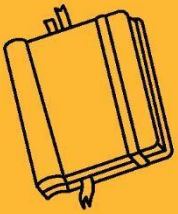
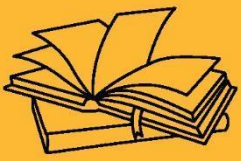
CONCLUSION:

- Errors and exceptions are integral parts of Java programming.
- Java provides a powerful mechanism to handle exceptions.
- Inbuilt exceptions help catch common issues efficiently.
- Proper exception handling improves program reliability.

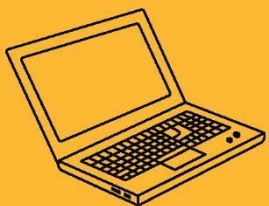


KEY TAKEAWAYS

- Errors are serious problems; exceptions are manageable.
- Java provides try, catch, throw, throws, and finally for exception handling.
- Use inbuilt exceptions to manage common runtime issues.
- Custom exceptions improve code clarity in large systems.
- Exception handling makes applications robust and user-friendly.



PACKAGE, MULTITHREADING & EXCEPTION HANDLING





SUB LESSON 8.2

TRY..CATCH STATEMENT, MULTIPLE CATCH BLOCKS

Before learning try..catch statements, it is important to understand what exceptions are and how they can interrupt the normal flow of a program. The try..catch block allows programmers to handle exceptions and continue program execution without crashing.

TRY...CATCH STATEMENT

The try statement is used to write a block of code that you want to test for possible errors while the program is running. If any error (exception) occurs in the try block, the program does not crash. Instead, control is passed to the catch block. The catch block contains the code that handles the error. This way, Java allows the program to handle problems smoothly without stopping unexpectedly.

Syntax:

```
try
{
    //Code
}
catch(ExceptionName e1)
{
    //catch block
}
```

Example without try..catch:



```
public class Demo

{

    public static void main(String[ ] args)

    {

        System.out.println("start programming");

        int a=5/0;           Can not divide number by zero

        System.out.println("Answer="+a);

        System.out.println("Something went wrong.");

    }

}
```

Output:

start programming

Java.lang.Excpetion: Divide by zero

Example with try..catch:

```
public class Demo

{

    public static void main(String[ ] args)

    {

        System.out.println("start programming");

        try
```



```
    {  
        int a=10/0;  
        System.out.println("Answer="+a);  
    }  
    → catch (Exception e)  
    {  
        System.out.println("Something went wrong.");  
    }  
    System.out.println("End Program");  
}  
}
```

Output:

start programming

Something went wrong

End Program

MULTIPLE CATCH BLOCKS

Multiple catch blocks is used to handle different types of exceptions separately. When an exception occurs in the try block, Java checks each catch block to find a match. As soon as it finds a matching catch block, it executes that block and skips the rest. This is useful when you want to take different actions for different types of errors. For example, you might want to handle arithmetic errors differently from file-related errors. Each catch block should handle a specific exception type, and the more specific exceptions should come before general ones.



Syntax:

```
try
```

```
{// Protected code
```

```
}
```

```
catch (ExceptionType1 e1)
```

```
{// Catch block1
```

```
}
```

```
catch (ExceptionType2 e2)
```

```
{// Catch block2
```

```
}
```

```
catch (ExceptionType3 e3)
```

```
{// Catch block3
```

```
}
```

Example:

```
class Example
```

```
{
```

```
    public static void main(String args[])
```

```
{
```

```
    try
```

```
{
```



```
        int num=120/2;

        System.out.println(num);

        int a= new int[2];

        System.out.println(a[4]);

    }

    catch(ArithmeticException e)

    {

        System.out.println("Number not be divided  by zero");

    }

    catch(ArrayIndexOutOfBoundsException e)

    {

        System.out.println("ArrayIndexOutOfBounds");

    }

}
```

Output:

60

ArrayIndexOutOfBounds

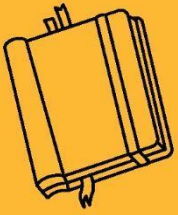
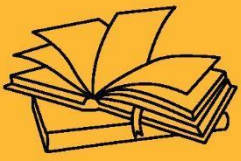
CONCLUSION:

- The try..catch statement is used to handle exceptions and prevent program crashes.

- 
- Multiple catch blocks allow handling different types of exceptions separately, making the program more flexible and accurate in error handling.

KEY TAKEAWAYS

- The try block contains code that may throw an exception.
- The catch block is used to handle the exception thrown in the try block.
- Multiple catch blocks can be used to handle different types of exceptions individually.
- Using try..catch improves program stability and error management.
- The order of catch blocks should be from most specific to most general exception types



PACKAGE, MULTITHREADING & EXCEPTION HANDLING





SUB LESSON 8.3

THROW AND THROWS KEYWORD, FINALLY CLAUSE

Before learning throw, throws, and finally, one should understand basic exception handling using try..catch. These keywords provide more control over how exceptions are created, declared, and cleaned up after execution—helping write clean and predictable error-handling code.

THROW AND THROWS KEYWORD

In Java, throw, throws, and finally are important keywords used in exception handling. The throw keyword is used to explicitly throw an exception from a method or block of code. The throws keyword is declared in the method signature to indicate that a method may throw one or more exceptions. The finally block is used to write code that must be executed regardless of whether an exception occurs or not, such as closing files or releasing resources. Together, these keywords help in building robust and error-resistant Java programs.

THROW:

The throw keyword is used to manually create and throw an exception from a method or a block of code. It lets you signal that something went wrong by creating an exception object and passing it to the runtime. For example, you can throw an `ArithmeticException` if an invalid calculation happens

Syntax:

```
throw new ExceptionType("Error message");
```

Example:

```
public class ThrowExample  
{
```




```
public static void checkNumber(int num) {  
  
    if (num < 0) {  
  
        // Manually throw an exception  
  
        throw new IllegalArgumentException("Number must be positive!");  
  
    } else {  
  
        System.out.println("Number is " + num);  
  
    }  
  
}  
  
public static void main(String[] args) {  
  
    checkNumber(-5);  
  
}  
  
}
```

Output:

Exception in thread "main" java.lang.IllegalArgumentException: Number must be positive!

at ThrowExample.checkNumber(ThrowExample.java:5)

at ThrowExample.main(ThrowExample.java:11)

THROWS:

The throws keyword is used in a method's declaration to declare that the method might throw one or more checked exceptions. It tells the caller of the method that they must handle or declare



these exceptions. This is necessary when the method calls other methods that can throw checked exceptions.

Syntax:

```
// Method declaration with throws

public void methodName() throws ExceptionType1, ExceptionType2 {

    // method body

}
```

Example:

```
public class ThrowsExample

{

    // Method declares that it may throw ArithmeticException

    public static void divide(int a, int b) throws ArithmeticException {

        int result = a / b;

        System.out.println("Result: " + result);

    }

    public static void main(String[] args) {

        try {

            divide(10, 0); // This will cause ArithmeticException

        } catch (ArithmeticException e) {

            System.out.println("Caught Exception: " + e.getMessage());

        }

    }

}
```



```
    }  
}
```

Output:

Caught Exception: / by zero

FINALLY:

The finally block is used to write code that should run no matter what happens in the try and catch blocks. Whether an exception occurs or not, and whether it is handled or not, the code inside the finally block will always be executed. This is especially useful for tasks like closing files, releasing resources, or cleaning up after the program has tried something risky. The finally block comes after the try-catch blocks, and ensures that important code is always run, even if an error occurred.

Syntax:

```
    finally  
{
```

Example:

```
class Demo  
{  
    public static void main(String args[])  
{  
        try  
        {  
            int data=55/5;
```



```
        System.out.println(data);

    }

    catch(NullPointerException e)

    {

        System.out.println(e);

    }

    finally

    {

        System.out.println("finally block is executed");

    }

    System.out.println("remaining code");}

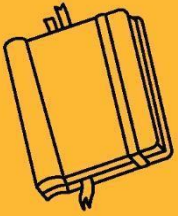
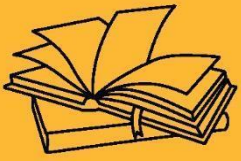
}
```

CONCLUSION:

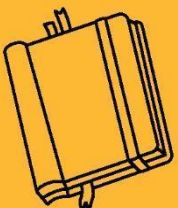
- The throw keyword is used to create and explicitly throw an exception.
- The throws keyword is used to declare exceptions that a method might throw.
- The finally block is always executed, whether an exception occurs or not, and is commonly used to release resources.

KEY TAKEAWAYS

- throw is used to throw a single exception manually.
- throws is used in method declarations to pass the responsibility of exception handling to the calling method.
- finally block is used to write clean-up code like closing files or releasing memory.
- The finally block executes regardless of whether an exception is handled or not.



PACKAGE, MULTITHREADING & EXCEPTION HANDLING





SUB LESSON 8.4

USES OF EXCEPTION, USER DEFINED EXCEPTION

Before learning the uses of exceptions and how to create user-defined exceptions, it's important to understand Java's built-in exception handling mechanism. This knowledge helps you understand when and why custom exceptions are needed for specific error scenarios that inbuilt exceptions cannot describe clearly.

USES OF EXCEPTION

Exception handling in Java is used to manage runtime errors in a structured and reliable way. It allows the program to continue its normal flow even when an error occurs, rather than crashing unexpectedly. By using try-catch blocks, the error-handling code is separated from the main logic, making the code cleaner and easier to understand. It also helps in detecting and handling different types of runtime issues like division by zero, invalid input, file not found, etc. Using keywords like throw, throws, and finally, developers can either throw exceptions manually, pass them to higher methods, or ensure that certain important code always runs. Exception handling improves the reliability and stability of a program and also allows developers to create custom exceptions that match specific needs of the application.

USER DEFINED FUNCTION:

An exception is a problem that arises during the execution of a program and disrupts the normal flow of instructions. When an exception occurs and is not handled properly, the program may terminate unexpectedly, and the remaining code will not be executed. To handle specific situations more effectively, Java allows us to create our own exceptions, known as custom exceptions or user-defined exceptions. This is done by extending the built-in Exception class and writing a custom exception class that represents a specific type of error. Once created, this custom exception can be thrown using the throw keyword, just like any standard Java exception.



This feature helps in building more meaningful and readable error-handling logic in Java applications.

Example:

// Step 1: Create a custom exception class

```
class MyException extends Exception
```

```
{
```

```
    public MyException(String message)
```

```
    {
```

```
        super(message); // Call parent constructor
```

```
    }
```

```
}
```

// Step 2: Use the custom exception in main method

```
public class CustomExceptionExample
```

```
{
```

```
    public static void main(String[] args)
```

```
    {
```

```
        try {
```

```
            int age = 15;
```

```
            if (age < 18) {
```

```
                throw new MyException("Age must be 18 or above.");
```

```
            } else {
```



```
        System.out.println("You are eligible to vote.");

    }

} catch (MyException e) {

    System.out.println("Custom Exception Caught: " + e.getMessage());

}

}

}
```

Output:

Custom Exception Caught: Age must be 18 or above.

A custom exception in Java is an exception defined by the user to handle specific application requirements. These exceptions extend either the `Exception` class (for checked exceptions) or the `RuntimeException` class (for unchecked exceptions).

Why Use User-Defined Exceptions?

To handle specific errors in your application that are not covered by built-in exceptions.

To make the code more meaningful and readable.

To separate business logic errors from system errors

Example:

In a banking app, if the withdrawal amount exceeds the balance, instead of using `IllegalArgumentException`, you can create `InsufficientBalanceException`.

CONCLUSION:

- Exception handling improves the reliability and maintainability of programs by managing errors gracefully.



- User-defined exceptions allow developers to create custom error types suited to specific application needs, improving code clarity and debugging

KEY TAKEAWAYS

- Exception handling helps in detecting and managing runtime errors without crashing the program.
- Custom or user-defined exceptions are created by extending the Exception class.
- They allow developers to define meaningful error types based on specific application logic.
- Using exceptions makes code more readable and professional by separating error-handling code from regular logic.

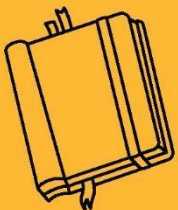
OUTCOMES OF CHAPTER 8

After completing these units, students will be able to:

- Understand the types of errors and exceptions, and handle them using try, catch, throw, throws, and finally.
- Distinguish between inbuilt and user-defined exceptions and apply them in real-world programs for robustness.
- Learn and implement multiple catch blocks for handling various types of exceptions efficiently.
- Use Java's Stream classes (Byte and Character streams) for reading and writing data.
- Perform file handling operations and work with console input/output using relevant Java I/O classes.



INPUT & OUTPUT





SUB LESSON 9.1

STREAM CLASS, BYTE STREAM AND CHARACTER STREAM

OBJECTIVES OF CHAPTER 9

9.1 Stream class, Byte stream and character stream

- Understand the concept of streams in Java and differentiate between byte and character streams.
- Learn to use various InputStream and OutputStream classes for handling byte-oriented data and Reader/Writer classes for character-based data.

9.2 Reading console input, writing console output, Print Writer class, Reading and writing files

- Gain the ability to read data from the console and write output using standard Java I/O classes like Scanner, System.in, System.out, and PrintWriter.
- Learn how to perform file handling operations such as reading from and writing to text files using Java I/O streams.



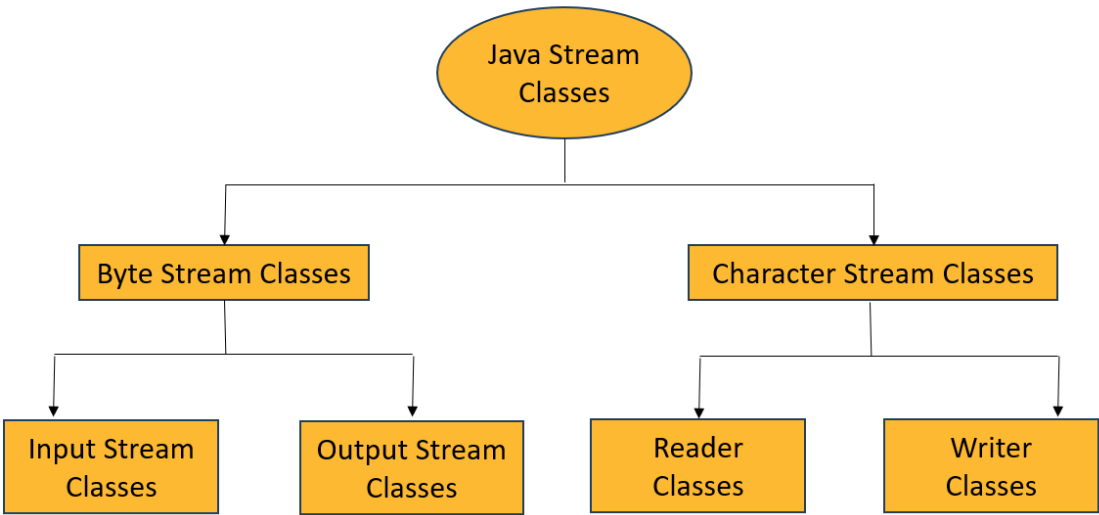
In Java, input and output operations are handled using streams, which represent a flow of data. These streams make it easier to read from and write to different data sources such as files, keyboard input, or network connections.

STREAM CLASS

In Java, a stream is a sequence of data. The Java I/O (Input and Output) system is built on streams which allow you to read data from sources and write data to destinations. Streams in Java represent an ordered sequence of data. Java performs input and output operations in the terms of streams. It uses the concept of streams to make I/O operations fast. For example, when we read a sequence of bytes from a binary file, actually, we're reading from an input stream. Similarly, when we write a sequence of bytes to a binary file, we're writing to an output stream.

Java provides two main types of streams:

- Byte Stream
- Character Stream



BYTE STREAM



- Byte streams in Java are designed to provide a convenient way for handling the input and output of bytes (i.e., units of 8-bits data). We use them for reading or writing to binary data I/O.
- Byte streams are especially used when we are working with binary files such as executable files, image files, and files in low-level file formats such as .zip, .class, .obj, and .exe.
- Binary files are those files that are machine readable. For example, a Java class file is an extension of “.class” and humans cannot read it. It can be processed by low-level tools such as a JVM (executable java.exe in Windows) and java disassembler (executable javap.exe in Windows).
- Byte streams that are used for reading are called input streams and for writing are called output streams.
- They are useful for reading and writing binary data such as images, audio files, etc.
 - InputStream: Base class for reading byte data.
 - OutputStream: Base class for writing byte data.
- Common Byte Stream Classes:
 - FileInputStream
 - FileOutputStream
 - BufferedInputStream
 - BufferedOutputStream

CHARACTER STREAM

- Character streams are more efficient than byte streams. They are mainly used for reading or writing to character or text-based I/O such as text files, text documents, XML, and HTML files.
- Text files are those files that are human readable. For example, a .txt file that contains human-readable text. This file is created with a text editor such as Notepad in Windows.
- Character streams that are used for reading are called readers and for writing are called writers. They are represented by the abstract classes of Reader and Writer in Java.
- Character streams are used for input and output of 16-bit Unicode characters. They are useful for reading and writing text data.



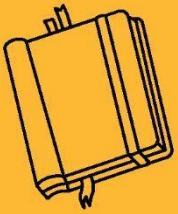
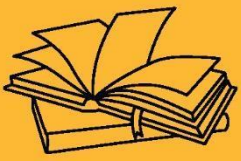
- Reader: Abstract class for reading character streams.
- Writer: Abstract class for writing character streams.
- Common Character Stream Classes:
 - FileReader
 - FileWriter
 - BufferedReader
 - BufferedWriter

CONCLUSION

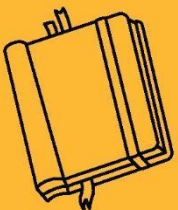
- Streams in Java simplify reading and writing data from different sources. With a rich hierarchy of classes, Java provides both flexibility and control to handle any I/O requirement effectively.

KEY TAKEAWAYS

- Streams are the backbone of Java I/O.
- Byte streams handle raw data, character streams handle textual data.



INPUT & OUTPUT





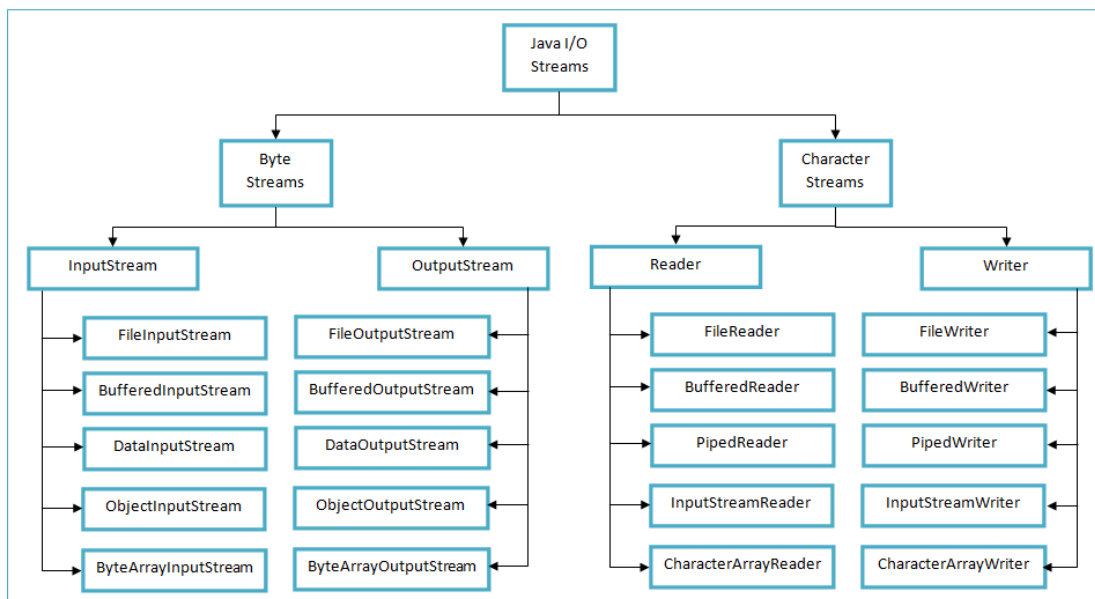
SUB LESSON 9.2

READING CONSOLE INPUT, WRITING CONSOLE OUTPUT, PRINT WRITER CLASS, READING AND WRITING FILES

Reading input and writing output are fundamental tasks in any programming language. Java provides various classes in the `java.io` package to handle user interaction and file operations efficiently.

INPUT OUTPUT STREAM

In Java, Input and Output (I/O) Stream classes are part of the `java.io` package and are used to read data from sources (input) and write data to destinations (output). These streams can handle different types of data such as bytes and characters. Java provides two main types of streams: Byte Streams (like `FileInputStream`, `FileOutputStream`) for binary data and Character Streams (like `FileReader`, `FileWriter`) for text data. These stream classes simplify data handling by providing built-in methods for reading and writing data efficiently.





READING CONSOLE INPUT

Reading input from the console allows the user to interact with a Java program by typing information during execution. Java provides various classes to accept input from the keyboard.

Example

```
import java.io.*;

public class InputExample {

    public static void main(String[] args) throws IOException

    {

        BufferedReader reader = new BufferedReader(new InputStreamReader(System.in));

        System.out.print("Enter your name: ");

        String name = reader.readLine();

        System.out.println("Hello, " + name);

    }

}
```

Output

Enter your name: Hetal

Hello Hetal

WRITING JAVA APPLET

Writing output to the console is one of the fundamental ways to display data to users in Java. It helps developers debug and interact with users during the program's execution. Java provides multiple ways to write output to the console.



These are the most commonly used methods for outputting data.

- `System.out.print()`: Prints the text on the console without moving the cursor to a new line.
- `System.out.println()`: Prints the text and then moves the cursor to a new line.

PRINTWRITER CLASS

The `PrintWriter` class in Java is a part of the `java.io` package and is used for writing formatted text to an output stream. It supports writing both to files and to the console, and it provides methods that make it easier to print formatted representations of objects.

Constructor of Printwriter

```
PrintWriter pw = new PrintWriter(OutputStream out);
```

```
PrintWriter pw = new PrintWriter(OutputStream out, boolean autoFlush);
```

```
PrintWriter pw = new PrintWriter(String fileName);
```

```
PrintWriter pw = new PrintWriter(File file);
```

Commonly Used Methods

Method	Description
<code>print()</code>	Prints text without a newline
<code>println()</code>	Prints text followed by a newline
<code>printf()</code>	Allows formatted printing
<code>close()</code>	Closes the stream
<code>flush()</code>	Flushes the stream

Example

```
import java.io.*;
```

```
public class PrintWriterConsole {
```



```
public static void main(String[] args) {  
  
    PrintWriter pw = new PrintWriter(System.out, true);  
  
    pw.println("Hello from PrintWriter!");  
  
    pw.printf("Formatted number: %.2f", 123.456);  
  
}  
  
}
```

Output

Hello from PrintWriter!

Formatted number: 123.46

READING AND WRITING FILES

File handling in Java is performed using classes from the java.io and java.nio.file packages. Java provides various classes like FileReader, FileWriter, BufferedReader, BufferedWriter, and Scanner to read from and write to files. These operations are essential for storing and retrieving data from the local file system.

Classes used for File Handling

Class	Purpose
FileReader	Reads data from a file (character)
BufferedReader	Reads text efficiently from file
FileWriter	Writes character data to a file
BufferedWriter	Writes text efficiently to file
PrintWriter	Formats and writes text
Scanner	Reads from file using simple syntax



Example(Read the File)

```
import java.io.*;

public class ReadFileExample {

    public static void main(String[] args) {

        try {

            FileReader fr = new FileReader("example.txt");

            BufferedReader br = new BufferedReader(fr);

            String line;

            while ((line = br.readLine()) != null) {

                System.out.println(line);

            }

            br.close();

            fr.close();

        } catch (IOException e) {

            System.out.println("File not found.");

        }

    }

}
```

Example(Write the File)

```
import java.io.*;
```



```
public class WriteFileExample {  
  
    public static void main(String[] args) {  
  
        try {  
  
            FileWriter fw = new FileWriter("output.txt");  
  
            BufferedWriter bw = new BufferedWriter(fw);  
  
            bw.write("This is a line written to the file.");  
  
            bw.newLine();  
  
            bw.write("Second line written.");  
  
            bw.close();  
  
            fw.close();  
  
        } catch (IOException e) {  
  
            System.out.println("Error writing to file.");  
  
        }  
  
    }  
  
}
```

CONCLUSION:

- Reading and writing files in Java is straightforward and powerful with classes from the java.io package.
- By using these tools properly, we can perform various file operations efficiently and securely.
-



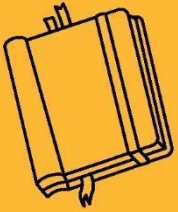
KEY TAKEAWAYS

- Console input in Java can be taken using `Scanner`, `BufferedReader`, or `System.in` for reading data from the keyboard.
- Console output is handled using `System.out.print()`, `System.out.println()`, and `System.out.printf()` for formatted output.
- The `PrintWriter` class offers convenient and formatted output to files and console, similar to `System.out`.

OUTCOME OF CHAPTER 9

By the end of these sections, students will be able to:

- Understand the concept of Streams in Java, and differentiate between Byte Streams and Character Streams.
- Apply input/output stream classes to efficiently manage console and file I/O operations.
- Use `Scanner`, `BufferedReader`, and `System.in` to read console input; and `System.out`, `PrintWriter`, and `BufferedWriter` for output.
- Build Java programs that can interact with users or external files, making applications more dynamic and data-driven.



APPLET





SUB LESSON 10.1

INTRODUCTION TO APPLETS, LIFECYCLE OF APPLET, WRITING JAVA APPLETS

OBJECTIVES OF CHAPTER 10

10.1 Introduction to Applets, Lifecycle of Applet, Writing Java Applets

- Understand what a Java Applet is and how it differs from Java applications.
- Learn the various stages in the lifecycle of an applet (init(), start(), paint(), stop(), destroy()).
- Gain the ability to write and execute basic Java applet programs using Applet or JApplet classes.

10.2 Graphics Class in Applet

- Understand how the Graphics class is used for drawing shapes and text in an applet window.
- Learn to override the paint(Graphics g) method to implement custom drawings.

10.3 Image and Sound in Applet

- Learn how to load and display images in an applet using getImage() and drawImage().
- Understand how to play sound clips using getAudioClip() and play() methods.

10.4 Passing Parameters to Applet

- Understand how to pass parameters to applets via HTML tags.
- Learn to access and use those parameters inside the applet using getParameter() method.



An applet is a Java program that runs inside a web browser and is used to create interactive web applications.

APPLET

An applet is a special kind of Java program designed to be embedded in a web page and executed within a web browser or an applet viewer. Applets are part of Java's Abstract Window Toolkit (AWT) and are used primarily for creating interactive features on web pages, such as animations, games, calculators, or dynamic forms.

Unlike standalone Java applications that start with a `main()` method, applets do not use the `main()` method. Instead, they rely on a predefined lifecycle, including methods like `init()`, `start()`, `stop()`, and `destroy()`, which are automatically invoked by the browser or applet viewer during the execution of the applet.

Applets are lightweight and platform-independent. They are embedded into HTML pages using the `<applet>` or `<object>` tag (older method), though modern browsers now have limited or no support for running applets due to security concerns.

Applets must import the `java.applet.*` and `java.awt.*` packages for basic functionality and graphics support. A simple applet can display text, draw shapes, accept user input, and more.

Advantage

- It works at client side so less response time.
- Secured
- It can be executed by browsers running under many platforms, including Linux, Windows, Mac Os etc.

Disadvantage

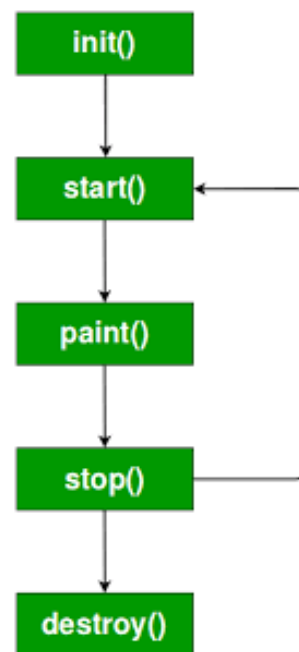
- Plugin is required at client browser to execute applet.
- `java.applet` package has been deprecated in Java 9 and later versions, as applets are no longer widely used on the web.



LIFECYCLE OF APPLET

An applet in Java goes through various stages from loading to destruction. The life cycle of an applet refers to the process and sequence of events that occur from the moment the applet is loaded into memory until it is removed. It defines how an applet is initialized, started, executed, stopped, and finally destroyed. In simple terms, it's the entire journey of an applet's existence, managed by the browser or applet viewer, through specific predefined methods.

The Java Applet Life Cycle refers to the series of methods that define the stages an applet goes through from creation to destruction.



init()

- This is the initialization method, called once when the applet is first loaded.
- It is used to initialize variables, load resources, or perform setup tasks.

Syntax

```
public void init()
```



```
{  
  
    // initialize objects  
  
}
```

start()

- Called after init() and every time the applet becomes active or visible again (e.g., user revisits the page).
- Used to resume execution such as restarting animations or threads.

Syntax

```
public void start()  
  
{  
  
    // code to start the applet  
  
}
```

paint(Graphics g)

- Called automatically after start() and whenever the applet needs to redraw its content.
- Used to draw output on the applet window (text, images, shapes, etc.).

Syntax

```
public void paint( Graphics graphics )  
  
{  
  
    // draw the applet shapes  
  
}
```



stop()

- Invoked when the applet is no longer visible, such as when the user navigates away from the page.
- Used to pause activities, like stopping animations or background tasks.

Syntax

```
public void stop()

{

    // code to stop the applet

}
```

destroy()

- Called just before the applet is unloaded from memory.
- Used to release resources, such as closing files or terminating threads.

Syntax

```
public void destroy()

{

    // code to destroy the applet

}
```

WRITING JAVA APPLET

Writing a Java applet involves creating a class that extends the Applet class and overriding some of its lifecycle methods like `init()` and `paint()`. Unlike regular Java programs, applets do not have



a main() method. Instead, the browser or applet viewer calls these lifecycle methods automatically.

Syntax

```
import java.applet.Applet;
```

```
import java.awt.*;
```

```
class AppletName extends Applet
```

```
{
```

```
    public void init() {
```

```
        // initialized objects
```

```
    }
```

```
    public void start() {
```

```
        // code to start the applet
```

```
    }
```

```
    public void paint(Graphics graphics) {
```

```
        // draw the shapes
```

```
    }
```

```
    public void stop() {
```

```
        // code to stop the applet
```

```
    }
```

```
    public void destroy() {
```

```
        // code to destroy the applet
```



```
}
```

```
}
```

Example

```
<html>
```

```
<body>
```

```
<applet code="HelloWorld.class" width="200" height="200" > </applet>
```

```
</body>
```

```
</html>
```

```
import java.applet.*;
```

```
import java.awt.*;
```

```
public class HelloWorld extends Applet
```

```
{
```

```
    public void paint(Graphics g)
```

```
    {
```

```
        g.drawString("Hello World", 20, 20);
```

```
    }
```

```
}
```

Output

```
javac HelloWorld.java
```

```
appletviewer HelloWorld.java
```

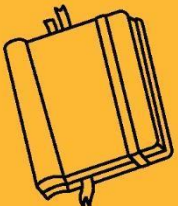
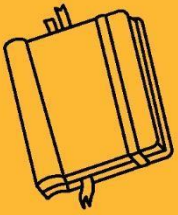


CONCLUSION:

- Java applets allow developers to create interactive and dynamic content that runs inside web browsers, enhancing user experience on web pages.
- The applet lifecycle methods (`init()`, `start()`, `paint()`, `stop()`, and `destroy()`) manage the applet's execution and resource handling effectively.

KEY TAKEAWAYS

- Applets extend the `Applet` class and do not use the `main()` method.
- Lifecycle methods control initialization, execution, drawing, pausing, and termination of applets.
- Applets are embedded into HTML pages using `<applet>` or `<object>` tags (though modern browser support is limited).



APPLET



SUB LESSON 10.2

GRAPHICS CLASS IN APPLET

The Graphics class in Java is a part of the java.awt package and is widely used to draw shapes, text, and images within an applet or other GUI components like JFrame, Canvas, or Panel.

GRAPHICS CLASS

In Java, the Graphics class is used to draw shapes, text, and images on a component like an applet, frame, panel, or canvas. It belongs to the java.awt package. In applets, we use the paint(Graphics g) method to draw something. The g is an object of the Graphics class, and we use it to call different drawing methods. In an applet, we don't use System.out.println() to print things on the screen like we do in console programs. Instead, we use graphics methods to draw things directly onto the applet window.

Following are the common methods of the Graphics Class

Method	What It Does
<code>drawString(String s, int x, int y)</code>	Draws a string at (x, y)
<code>drawLine(int x1, int y1, int x2, int y2)</code>	Draws a line from point (x1, y1) to (x2, y2)
<code>drawRect(int x, int y, int width, int height)</code>	Draws a rectangle
<code>fillRect(int x, int y, int width, int height)</code>	Draws a filled rectangle
<code>drawOval(int x, int y, int width, int height)</code>	Draws an oval
<code>fillOval(int x, int y, int width, int height)</code>	Draws a filled oval
<code>setColor(Color c)</code>	Sets the color of drawing
<code>setFont(Font f)</code>	Sets the font for drawing text



Method	What It Does
--------	--------------

Example

```
<html>
```

```
    <body>
```

```
        <applet code="DrawingExample.class" width="200" height="200" ></applet>
```

```
    </body>
```

```
</html>
```

```
import java.applet.*;
```

```
import java.awt.*;
```

```
public class DrawingExample extends Applet
```

```
{
```

```
    public void paint(Graphics g) {
```

```
        // drawString
```

```
        g.drawString("Hello Applet!", 20, 20);
```

```
        // drawRect(x, y, width, height)
```

```
        g.drawRect(20, 40, 100, 50);
```

```
        // fillRect(x, y, width, height)
```

```
        g.setColor(Color.BLUE);
```

```
        g.fillRect(140, 40, 100, 50);
```



```
// drawLine(x1, y1, x2, y2)

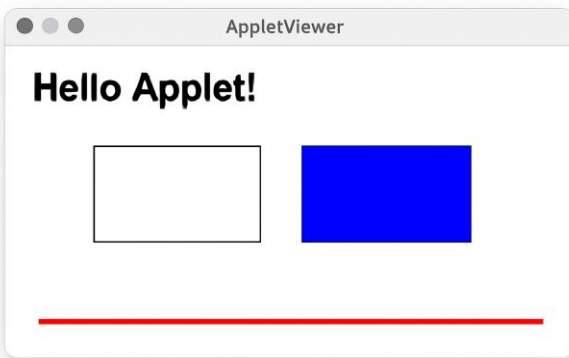
g.setColor(Color.RED);

g.drawLine(20, 110, 240, 110);

}

}
```

Output



Example

```
<html>
```

```
<body>
```

```
<applet code="ShapeExample.class" width="200" height="200" ></applet>
```

```
</body>
```

```
</html>
```

```
import java.applet.*;
```

```
import java.awt.*;
```

```
public class DrawingExample extends Applet
```



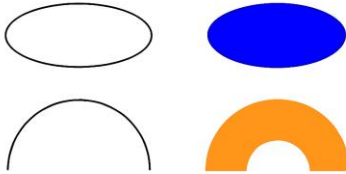
```
{  
  
    public void paint(Graphics g) {  
  
        // Set custom font  
  
        Font myFont = new Font("Serif", Font.BOLD, 18);  
  
        g.setFont(myFont);  
  
        g.drawString("Simple Applet Example", 20, 30);  
  
        // drawOval(x, y, width, height)  
  
        g.drawOval(20, 50, 100, 60);  
  
        // fillOval(x, y, width, height)  
  
        g.setColor(Color.BLUE);  
  
        g.fillOval(140, 50, 100, 60);  
  
        // drawArc(x, y, width, height, startAngle, arcAngle)  
  
        g.setColor(Color.BLACK);  
  
        g.drawArc(20, 130, 100, 60, 0, 180);  
  
        // fillArc(x, y, width, height, startAngle, arcAngle)  
  
        g.setColor(Color.RED);  
  
        g.fillArc(140, 130, 100, 60, 0, 270);  
  
    }  
  
}
```

Output



AppletViewer

Simple Applet Example

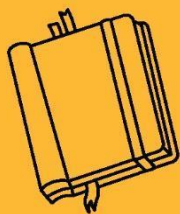


CONCLUSION:

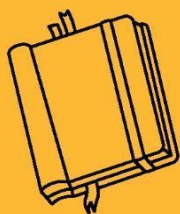
- The Graphics class helps in drawing everything you see in a GUI.
- It makes Java applets interactive and colorful.
- Learning Graphics is essential for creating visual programs in Java.

KEY TAKEAWAYS

- Graphics class is used to draw in Java Applet.
- Use the paint(Graphics g) method to start drawing.
- You can draw strings, lines, rectangles, ovals, etc.
- Use setColor() to change color.
- Coordinates decide the drawing position.



APPLET





SUB LESSON 10.3

IMAGE AND SOUND IN APPLET

Multimedia elements like images and sound enhance the interactivity of Java Applets. This chapter explains how to load and display images and play sound in Java Applets using built-in classes.

IMAGE AND SOUND IN APPLET

In Java Applets, images can be displayed using the `getImage()` method of the `Applet` class. This method loads an image from a specified URL or file path. The loaded image is then drawn on the applet window using the `drawImage()` method of the `Graphics` class, typically inside the `paint()` method. The image must be in a compatible format like `.jpg` or `.gif`. This allows Applets to visually enhance their output using graphics and multimedia.

Steps to Display an Image

- Import necessary packages:

```
import java.applet.Applet;
```

```
import java.awt.*;
```

- Load image using `getImage()`:

```
Image img = getImage(getDocumentBase(), "image.jpg");
```

Display the image inside `paint()`:

```
public void paint(Graphics g) {  
  
    g.drawImage(img, 10, 10, this);  
  
}
```



Example

```
import java.applet.Applet;

import java.awt.*;

public class ImageApplet extends Applet

{

    Image img;

    public void init()

    {

        img = getImage(getDocumentBase(), "image.jpg");

    }

    public void paint(Graphics g)

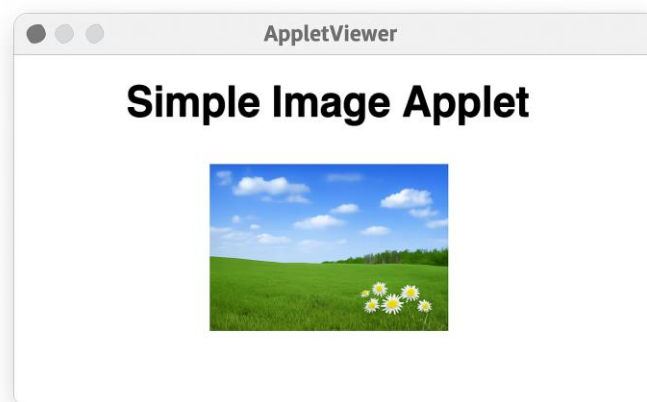
    {

        g.drawString("Simple Image Applet ", 20, 20);

        g.drawImage(img, 50, 50, this);

    }

}
```





```
}
```

Output

In Java Applets, sound can be played using the AudioClip interface from the java.applet package. The sound file, usually in .wav format, is loaded using the getAudioClip() method. Once loaded, it can be played using the play() method, looped continuously with loop(), or stopped with stop(). These methods are typically called within the init() or start() lifecycle methods of the applet. This feature allows applets to provide audio feedback or background sound, enhancing user interaction.

Steps to Play a Sound

- Import required classes:

```
import java.applet.*;
```

```
import java.net.*;
```

- Load sound using getAudioClip():

```
AudioClip sound = getAudioClip(getDocumentBase(), "sound.wav");
```

- Play the sound using:

```
sound.play(); // To play once
```

```
sound.loop(); // To play in loop
```

```
sound.stop(); // To stop playing
```

Example

```
import java.applet.*;
```

```
import java.net.*;
```



```
public class SoundApplet extends Applet

{

    AudioClip sound;


    public void init() {

        sound = getAudioClip(getDocumentBase(), "sound.wav");

    }


    public void start() {

        sound.play();

    }

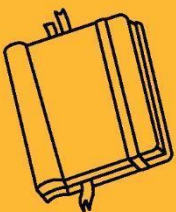
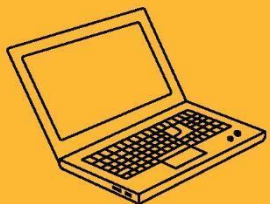
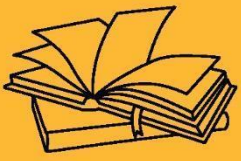
}
```

CONCLUSION:

- Java Applets can be enhanced with multimedia by displaying images and playing sounds.
- Using methods like `getImage()` and `getAudioClip()`, developers can make applets more interactive and visually appealing.

KEY TAKEAWAYS

- To display images, use `getImage()` to load and `drawImage()` to render them within the `paint()` method.
- To play sounds, use `getAudioClip()` to load the sound and control playback using `play()`, `loop()`, and `stop()` methods.



APPLET





SUB LESSON 10.4

PASSING PARAMETER

In Java Applets, input values can be passed from an HTML page using the `<param>` tag.

This allows applets to receive dynamic input at runtime, making them more interactive and flexible.

PASSING PARAMETER

In Java Applets, values can be passed from an HTML file to an applet using the `<param>` tag. This feature is useful when dynamic input is required for applets, such as names, numbers, or messages. These parameters are accessed in the applet using the `getParameter()` method.

- Parameters are passed using the `<param>` tag inside the `<applet>` tag.
- The `getParameter(String name)` method is used to retrieve values.
- Parameters are always retrieved as strings.
- Applet must be compiled first using `javac`.
- HTML or `appletviewer` is used to view output since `main()` is not used in Applets.

Syntax

```
<applet code="MyApplet.class" width="300" height="200">
```

```
  <param name="username" value="Hetal">
```

```
  <param name="roll" value="101">
```

```
</applet>
```

Here

- `name` – is the name of the parameter.
- `value` – is the value assigned to the parameter.
- `getParameter("username")` will return "Hetal".



Example

```
<applet code="MyApplet.class" width="300" height="200">

    <param name="username" value="Hetal">

    <param name="roll" value="101">

</applet>

import java.applet.Applet;

import java.awt.*;

/* <applet code="ParamApplet.class" width="300" height="150">

    <param name="name" value="Hetal">

    <param name="age" value="21">

</applet> */

public class ParamApplet extends Applet {

    String studentName;

    String studentAge;

    public void init() {

        studentName = getParameter("name");

        studentAge = getParameter("age");

    }

    public void paint(Graphics g) {

        g.drawString("Name: " + studentName, 50, 60);
```



```
        g.drawString("Age: " + studentAge, 50, 80);  
  
    }  
  
}
```

Output



CONCLUSION:

- Passing parameters in an applet provides a flexible way to customize applet behavior without changing code.
- It separates input from logic, making applets more dynamic and reusable.

KEY TAKEAWAYS

- The <param> tag is used in HTML to pass values from the webpage to the Java applet.
- Applets use `getParameter(String name)` method to retrieve parameter values inside the `init()` method.
- This technique allows customization of applet behavior without modifying its source code.



OUTCOMES OF CHAPTER 10

After studying this unit, students will be able to:

- Understand the concept and lifecycle of Java Applets, including how they differ from Java applications.
- Learn to write and execute Java Applets, including embedding them in HTML.
- Use the Graphics class to draw shapes and text within applets for visual content.
- Incorporate images and sounds into applets to enhance multimedia interaction.
- Understand how to pass parameters to applets using the `<param>` tag for dynamic content.